SEMINAR PROPOSAL

PENGEMBANGAN SISTEM ANALISIS TEKS TERPADU BERBASIS *WEB* UNTUK PENGECEKAN EJAAN BAHASA INDONESIA MENGGUNAKAN ALGORITMA SYMSPELL DAN DETEKSI PLAGIARISME MENGGUNAKAN TF-IDF SERTA *COSINE SIMILARITY*

Diajukan Untuk Memenuhi

Salah Satu Syarat Memperoleh Gelar Sarjana Teknik

MUH. IKBAL

E1E122066

JURUSAN INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS HALUOLEO

2026

LEMBAR PENGESAHAN

SEMINAR PROPOSAL

PENGEMBANGAN SISTEM ANALISIS TEKS TERPADU BERBASIS *WEB* UNTUK PENGECEKAN EJAAN BAHASA INDONESIA MENGGUNAKAN ALGORITMA SYMSPELL DAN DETEKSI PLAGIARISME MENGGUNAKAN TF-IDF SERTA *COSINE SIMILARITY*

Adalah benar dibuat oleh saya sendiri dan belum pernah dibuat atau diserahkan sebelumnya baik sebagian ataupun seluruhnya, baik oleh saya maupun orang lain, baik di Universitas Halu Oleo ataupun institusi pendidikan lainnya.

Kendari, Januari 2026

MUH. IKBAL

E1E122066

Pembimbing I	Pembimbing II
Sutardi, S.Kom.,M.T.	**Jumadil Nangi, S.Kom., M.T.**
NIP: 197602222010121001	**NIP: 198702062015041003**

Mengetahui,
Ketua Jurusan Informatika,
Fakultas Teknik, Universitas Haluoleo.

Isnawaty, S.Si., MT.

NIP: 197611172008122001

INTISARI

MUH. IKBAL,E1E122066

PENGEMBANGAN SISTEM ANALISIS TEKS TERPADU BERBASIS *WEB* UNTUK PENGECEKAN EJAAN BAHASA INDONESIA MENGGUNAKAN ALGORITMA SYMSPELL DAN DETEKSI PLAGIARISME MENGGUNAKAN TF-IDF SERTA *COSINE SIMILARITY*

Proposal, Fakutas Teknik, 2026

Kata Kunci: *Cosine Similarity*, Pengecekan Ejaan, Plagiarisme, Symspell, *Term Frequency-Inverse Document*, *web*

Perkembangan teknologi digital telah meningkatkan produksi karya tulis ilmiah berbahasa Indonesia yang dipublikasikan melalui jurnal nasional dan jurnal akses terbuka. Kondisi ini menimbulkan tantangan dalam menjaga kualitas bahasa dan integritas konten ilmiah, terutama terkait kesalahan ejaan dan potensi plagiarisme yang dapat terjadi baik secara sengaja maupun tidak sengaja. Proses validasi teks yang masih dilakukan secara terpisah menyebabkan alur kerja penulisan menjadi kurang efisien serta berpotensi meningkatkan risiko terlewatnya kesalahan bahasa dan kemiripan konten antar dokumen.

Penelitian ini bertujuan mengembangkan sistem analisis teks terpadu berbasis *web* untuk melakukan pengecekan ejaan dan deteksi plagiarisme pada naskah ilmiah berbahasa Indonesia. Algoritma *Symmetric Delete Spelling Correction* (SymSpell) digunakan untuk koreksi ejaan secara efisien dengan memanfaatkan kamus frekuensi, sedangkan metode *Term Frequency–Inverse Document Frequency* (TF-IDF) dan *Cosine Similarity* diterapkan untuk mengukur tingkat kemiripan antar dokumen sebagai dasar pendeteksian plagiarisme. Pengujian Sistem menggunakan *dataset* berupa 100 jurnal ilmiah berbahasa Indonesia sebagai data referensi pengecekan plagiarisme serta kamus frekuensi untuk mendukung proses pengecekan ejaan.

Hasil penelitian berupa sistem yang mampu mengintegrasikan proses pengecekan ejaan dan deteksi plagiarisme dalam satu alur kerja, sehingga diharapkan dapat

meningkatkan efisiensi validasi naskah serta mendukung kualitas bahasa dan integritas publikasi ilmiah nasional.

ABSTRACT

MUH. IKBAL, E1E122066

DEVELOPMENT OF A WEB-BASED INTEGRATED TEXT ANALYSIS SYSTEM FOR INDONESIAN SPELLING CHECKING USING THE SYMSPELL ALGORITHM AND PLAGIARISM DETECTION USING TF-IDF AND COSINE SIMILARITY

Proposal. Faculty Of Engineering, 2026

Keywords : *Cosine Similarity, Spell Checking, Plagiarism, Symspell, Term Frequency-Inverse Document, web*

The rapid development of digital technology has led to a significant increase in the production of Indonesian-language scientific texts published through national journals and open-access journals. This condition presents challenges in maintaining language quality and content integrity, particularly regarding spelling errors and potential plagiarism, whether intentional or unintentional. Text validation processes that are still conducted separately result in inefficient workflows and increase the risk of overlooked language errors and content similarity between documents.

This research aims to develop a web-based integrated text analysis system for spelling checking and plagiarism detection in Indonesian-language scientific manuscripts. The Symmetric Delete Spelling Correction (SymSpell) algorithm is employed to perform efficient spelling correction using a frequency dictionary, while the Term Frequency–Inverse Document Frequency (TF-IDF) and Cosine Similarity methods are applied to measure document similarity as the basis for plagiarism detection. The system testing uses a dataset of 100 Indonesian-language scientific journals as reference data for checking plagiarism and a frequency dictionary to support the spelling checking process.

The result of this research is an integrated system that combines spelling checking and plagiarism detection within a single workflow. This system is expected to improve the efficiency of manuscript validation and support the maintenance of language quality and the integrity of national scientific publications.

DAFTAR ISI

LEMBAR PENGESAHAN.....ii

INTISARI.....iii

ABSTRACT.....iv

DAFTAR ISI.....v

DAFTAR GAMBAR.....viii

DAFTAR TABEL.....x

BAB I PENDAHULUAN.....1

1.1 Latar Belakang.....1

1.2 Rumusan Masalah.....3

1.3 Batasan Masalah.....3

1.4 Tujuan Penelitian.....4

1.5 Manfaat Penelitian.....4

1.6 Sistematika Penulisan.....5

1.7 Tinjauan Pustaka.....6

BAB II TINJAUAN PUSTAKA.....13

2.1 Pengolahan Bahasa Alami (*Natural Language Processing*) 13

2.2 Teks *Preprocessing*.....15

2.3 Plagiarisme.....16

2.4 *Typographical Error*.....20

2.5 *Spelling Correction*.....21

2.6 Symspell.....21

2.7 *Cosine Similarity*.....23

2.8 *Ter-Frequency Inverse Document Frecuency* (TF-IDF)..25

2.9 *Facebook AI Similarity Search* (FAISS).....27
2.10 Python.....27
2.11 FastAPI.....28
2.12 *React*.js.....29
2.13 *Unified Modeling Languange (UML)*.....30
2.11.1 *Use Case* Diagram.....30
2.11.2 *Activity Diagram*.....31
2.11.3 *Class* Diagram.....32
2.11.4 *Sequence Diagram*.....33
2.14 *Rational Unified Process* (RUP).....34
2.15 Pengujian *Confusion Matrix*.....35
2.16 *User Acceptance Testing* (UAT).....37
2.17 Skala Likert.....37
BAB III METODOLOGI PENELITIAN.....39
3.1. Metode Pengumpulan Data.....39
3.1.1 Studi Literatur.....39
3.1.2 Pengumpulan *Dataset*.....39
3.2. Metode Pengembangan Sistem.....40
3.3. Waktu dan Tempat Penelitian.....41
3.3.1 Waktu.....41
3.3.2 Tempat Penelitian.....42
3.4. Analisis Kebutuhan Sistem.....42
3.4.1. Kebutuhan Fungsional.....42
3.4.2. Kebutuhan *Non*-fungsional.....44
3.5. Analisis Perancangan Sistem.....45

3.5.1 Perancangan Alur Sistem Secara Umum.....45
3.5.2 Perancangan Alur Kerja Algoritma Symspell.....46
3.5.3 Perancangan Alur Kerja Algoritma *Cosine Similarity* dan Metode TF-IDF..... 49
3.6. Perancangan *Unified Modeling Language* (UML).....52
3.6.1 *Use Case* Diagram.....52
3.6.2 *Class* Diagram.....54
3.6.3 *Activity Diagram*.....57
3.6.4 *Sequence Diagram*.....66
3.7. Analisis dan Perancangan Antarmuka (*User Interface*). 75
3.8. Studi Kasus.....83
3.8.1. Studi Kasus Pengecekan Ejaan Menggunakan Algoritma Symspell..... 83
3.8.2. Studi Kasus Pegecekan Plagiarisme Menggunakan Algoritma TF-IDF dan *Cosine Similarity*.....87
3.9. Pengujian Kinerja Algoritma.....92
3.9.1. Skenarion Pengujian Kinerja Algoritma SymSpell dalam Pengecekan Ejaan.....92
3.9.2. Skenario Pengujian Kinerja Algoritam *Cosine Similarity* dan TF-IDF dalam Pengecekan Plagiarisme.....94
3.10. Pengujian *User Acceptance Testing*.....96
DAFTAR PUSTAKA.....99
LAMPIRAN.....103

DAFTAR GAMBAR

Gambar 2. 1 Proses penghapusan kata.....22

Gambar 3. 1 Alur sistem secara umum 46
Gambar 3. 2 Alur pembuatan kamus algoritma symspell.....47
Gambar 3. 3 Alur proses Pencarian Kandidat Kata yang Benar.....48
Gambar 3. 4 Alur kerja algoritma Cosine Similarity dan metode TF-IDF.....50
Gambar 3. 5 *Use Case* diagram.....53
Gambar 3. 6 Class diagram.....54
Gambar 3. 7 *Activity Diagram* login.....58
Gambar 3. 8 *Activity Diagram* input data jurnal referensi.....59
Gambar 3. 9 *Activity Diagram* input kamus frekuensi.....60
Gambar 3. 10 *Activity Diagram* riwayat pengguna aplikasi.....61
Gambar 3. 11 *Activity Diagram* mengatur max delete distance.....62
Gambar 3. 12 *Activity Diagram* mengatur threshold similarity.....63
Gambar 3. 13 *Activity Diagram* pengecekan ejaan.....64
Gambar 3. 14 *Activity Diagram* pengecekan plagiarism dokumen 65
Gambar 3. 15 *Sequence Diagram* login.....67
Gambar 3. 16 *Sequence Diagram* input jurnal referensi.....68
Gambar 3. 17 *Sequence Diagram* input kamu frekuensi.....69
Gambar 3. 18 *Sequence Diagram* Monitoring Riwayat Penggunaan Aplikasi.....70
Gambar 3. 19 *Sequence Diagram* mengatur max delete distance. 71
Gambar 3. 20 *Sequence Diagram* mengatur treshold similarity....72
Gambar 3. 21 *Sequence Diagram* pengecekan ejaan.....73
Gambar 3. 22 *Sequence Diagram* pengecekan plagiarisme.....74
Gambar 3. 23 Antarmuka landing page.....76
Gambar 3. 24 Antarmuka dashboard admin.....77
Gambar 3. 25 Antarmuka kelola database jurnal.....77

Gambar 3. 26 Antarmuka kelola kamus frekuensi.....78
Gambar 3. 27 Antarmuka riwayat pengecekan plagiarism.....79
Gambar 3. 28 Antarmuka konfigurasi sistem.....80
Gambar 3. 29 Antarmuka pengecekan plagiarisme.....81
Gambar 3. 30 Antarmuka hasil pengecekan plagiarism.....81
Gambar 3. 31 Antarmuka pengecekan ejaan.....82
Gambar 3. 32 Antarmuka hasil pengecekan ejaan.....82

DAFTAR TABEL

Tabel 1. 1 Penelitian Terdahulu.....6

Tabel 2. 1 Proses pencocokan kata menggunakan algoritma symspell 23
Tabel 2. 2 Simbol diagram *Use Case*.....31
Tabel 2. 3 Notasi acitvity diagram.....32
Tabel 2. 4 Simbol class diagram.....33
Tabel 2. 5 Simbol *Sequence Diagram*.....33
Tabel 2.6 Komponen confusion matrix.....36

Tabel 3. 1 Timeline penelitian 42
Tabel 3. 2 Kebutuhan Perangkat Keras.....44
Tabel 3. 3 Kebutuhan Perangkat Lunak.....45
Tabel 3. 4 Relasi antara *class* diagram.....55
Tabel 3. 5 Tokenisasi.....83
Tabel 3. 6 Kata salah.....84
Tabel 3. 7 Daftar kata hasil penghapusan kata pada kamus.....85
Tabel 3. 8 Hasil *delete distance* kata yang salah.....86
Tabel 3. 9 Hasil pencocokan kombinasi penghapusan karakter antara kata input dan kamus frekuensi.....86
Tabel 3. 10 Database referensi.....88
Tabel 3. 11 Hasil pre-processing.....89
Tabel 3. 12 Perhitungan term frequency.....89
Tabel 3. 13 Perhitungan *document frequency*.....90
Tabel 3. 14 Perhitungan IDF.....90
Tabel 3. 15 Perhitungan TF-IDF.....91
Tabel 3. 16 Perhitungan Cosine Similarity.....92
Tabel 3. 17 Contoh data dokumen plagiarisme.....95
Tabel 3. 18 Skenarion Confusion matrix.....95

Tabel 3. 19 Pertanyaan *user acceptance testing*.....96
Tabel 3. 20 Kriteria jawaban.....97
Tabel 3. 21 Kriteria penilaian skala likert.....98

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi digital telah mendorong peningkatan produksi konten tekstual secara masif, mencakup karya ilmiah, artikel daring, hingga dokumen profesional. Indonesia mengalami peningkatan yang didominasi oleh karya tulis ilmiah berbahasa Indonesia yang dipublikasikan melalui jurnal nasional dan jurnal akses terbuka. Fenomena ini membawa tantangan serius dalam proses penulisan, khususnya terkait validasi kualitas dan integritas teks, agar informasi yang disampaikan tetap akurat, jelas, dan sesuai dengan kaidah akademik.

Tantangan kualitas dan integritas teks umumnya muncul secara bersamaan dalam proses penulisan ilmiah berbahasa Indonesia. Kesalahan ejaan dan tipografi, meskipun terlihat sederhana, dapat mengganggu pemahaman pembaca serta menurunkan persepsi terhadap profesionalisme dan ketelitian penulis(Rosdiana, 2020). Di sisi lain, plagiarisme yang dilakukan dengan disengaja maupun tidak sengaja menjadi persoalan yang semakin kompleks seiring dengan kemudahan akses dan distribusi referensi digital, khususnya pada ekosistem jurnal ilmiah nasional yang terus berkembang(Sukowati dan Suciptaningsih, 2024).

Urgensi permasalahan tersebut semakin nyata ketika dikaitkan dengan posisi Indonesia dalam publikasi jurnal akses terbuka di tingkat global. Berdasarkan data *Directory of Open Access Journals* (DOAJ), hingga Maret 2025 Indonesia tercatat sebagai negara dengan jumlah jurnal terindeks terbanyak di dunia, yaitu sebanyak 2.473 jurnal (DOAJ,2025). Capaian ini

menunjukkan pesatnya pertumbuhan publikasi ilmiah nasional sekaligus menegaskan meningkatnya volume naskah ilmiah berbahasa Indonesia yang diproduksi dan dipublikasikan. Kondisi tersebut secara langsung menuntut adanya sistem pendukung yang mampu menjaga kualitas bahasa dan integritas konten ilmiah secara konsisten dan efisien.

Dalam praktiknya, proses validasi teks pada naskah jurnal berbahasa Indonesia masih dilakukan secara terpisah. Penulis umumnya harus menggunakan
aplikasi pengecekan ejaan secara mandiri, kemudian berpindah ke sistem lain untuk melakukan deteksi plagiarisme. Alur kerja yang terfragmentasi ini tidak efisien, memakan waktu, serta meningkatkan risiko terlewatnya kesalahan ejaan bahasa Indonesia maupun indikasi kemiripan konten antar dokumen jurnal.

Untuk mengatasi permasalahan tersebut, diperlukan dukungan teknologi yang mampu melakukan pengecekan kualitas bahasa secara cepat dan akurat. Pengecekan ejaan memerlukan algoritma yang mampu bekerja efektif pada kamus berukuran besar serta sesuai dengan karakteristik Bahasa Indonesia. Oleh karena itu, algoritma *Symmetric Delete Spelling Correction* (SymSpell) dipilih dalam penelitian ini karena kemampuannya melakukan koreksi ejaan secara efisien melalui operasi penghapusan (*delete*), sehingga sesuai untuk pengecekan ejaan naskah jurnal berbahasa Indonesia secara *real-time* pada sistem berbasis *web* (zaky ramadhan dan Lhaksmana, 2020).

Selain kualitas bahasa, integritas karya tulis ilmiah juga perlu dijaga melalui pengukuran tingkat kemiripan dokumen secara objektif, meskipun terdapat perbedaan struktur kalimat.

Pendekatan *Term Frequency–Inverse Document Frequency* (TF-IDF) dan *Cosine Similarity* digunakan dalam penelitian ini untuk menganalisis kesamaan substansial antar dokumen jurnal ilmiah berbahasa Indonesia, sehingga memungkinkan pendeteksian plagiarisme berbasis konten secara efisien dan akurat(Widianto et al., 2024).

Meskipun teknologi SymSpell untuk pengecekan ejaan serta metode TF-IDF dan *Cosine Similarity* untuk deteksi plagiarisme memiliki fungsi dan mekanisme kerja yang berbeda, keduanya diimplementasikan sebagai fitur yang saling melengkapi dalam satu aplikasi analisis teks terpadu. Setiap fitur bekerja secara independen sesuai dengan tujuan masing-masing, namun tetap berada dalam satu sistem yang terintegrasi sehingga pengguna dapat melakukan pemeriksaan kualitas bahasa dan evaluasi integritas konten secara bersamaan dalam satu alur kerja. Integrasi ini memberikan kemudahan dan efisiensi, khususnya dalam proses penulisan dan evaluasi jurnal ilmiah berbahasa Indonesia.

Berdasarkan permasalahan tersebut, penelitian ini mengusulkan Pengembangan "**Pengembangan Sistem Pengecekan Ejaan dan Deteksi Plagiarisme Berbasis *Web* Menggunakan Algoritma SymSpell, TF-IDF, dan *Cosine Similarity***". Sistem ini diharapkan mampu meningkatkan efisiensi proses penulisan, mempercepat validasi teks, serta mendukung peningkatan kualitas dan integritas publikasi ilmiah nasional di tengah pesatnya pertumbuhan jurnal akses terbuka di Indonesia.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini yaitu:

1. Bagaimana merancang dan mengimplementasikan sistem analisis teks berbasis *web* yang mampu melakukan pengecekan ejaan bahasa Indonesia secara cepat dan akurat menggunakan algoritma SymSpell?
2. Bagaimana merancang dan mengimplementasikan fitur deteksi plagiarisme pada dokumen teks dengan memanfaatkan kombinasi metode Term *Frequency–Inverse Document Frequency* (TF-IDF) dan *Cosine Similarity*?
3. Bagaimana mengintegrasikan kedua fungsionalitas tersebut (pengecekan ejaan dan deteksi plagiarisme) ke dalam satu sistem analisis teks terpadu berbasis *web* untuk menciptakan alur kerja yang efisien dan mengatasi masalah fragmentasi alat bantu yang dihadapi pengguna saat ini?

1.3 Batasan Masalah

Berdasarkan rumusan masalah yang ada, adapun batasan masalah dalam penulisan tugas akhir ini adalah :

1. Objek penelitian berfokus pada analisis dokumen teks berbahasa Indonesia. Pengecekan ejaan mengacu pada kosakata baku dalam Kamus Besar Bahasa Indonesia (KBBI) dan tidak mencakup pemeriksaan tata bahasa (*grammar*).
2. Mekanisme deteksi plagiarisme menggunakan pendekatan *Local Repository Checking*, yaitu membandingkan dokumen uji terhadap *dataset* korpus lokal yang tersimpan dalam *database* sistem, dan tidak melakukan pencarian (*crawling*) data secara *real-time* dari *internet*.
3. Sistem dibangun berbasis *website*. *Input* data yang diproses dibatasi pada teks yang dimasukkan langsung (*copy-paste*) atau unggahan dokumen dengan format .docx dan .pdf yang diekstraksi ke dalam bentuk teks murni (*plain* teks).

4. Deteksi plagiarisme menggunakan metode *Cosine Similarity* berbasis bobot *TF-IDF* dibatasi pada deteksi kemiripan leksikal (kesamaan kata), dan tidak menjangkau deteksi plagiarisme makna (semantik) atau parafrase kompleks.

1.4 Tujuan Penelitian

Adapun tujuan dari penelitian ini adalah sebagai berikut :

1. Merancang dan mengimplementasikan sistem analisis teks berbasis *web* yang mampu melakukan pengecekan ejaan bahasa Indonesia secara cepat dan akurat menggunakan algoritma SymSpell.
2. Merancang dan mengimplementasikan fitur deteksi plagiarisme pada dokumen teks dengan memanfaatkan kombinasi metode *Term Frequency–Inverse Document Frequency* (TF-IDF) dan *Cosine Similarity.*
3. Menghasilkan sebuah sistem analisis teks terpadu berbasis *web* yang mengintegrasikan fitur pengecekan ejaan dan deteksi plagiarisme dalam satu platform tunggal, sehingga memberikan solusi alur kerja yang lebih efisien bagi pengguna.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik secara teoritis maupun praktis. Secara teoritis, penelitian ini berkontribusi dalam pengembangan ilmu pengetahuan di bidang pemrosesan bahasa alami (*Natural Language Processing*/NLP), khususnya dalam konteks bahasa Indonesia, serta menjadi referensi akademik mengenai penerapan algoritma SymSpell, TF-IDF, dan *Cosine Similarity* pada sistem analisis teks terpadu. Selain itu, penelitian ini menambah literatur mengenai

penggabungan algoritma pengecekan ejaan dan deteksi plagiarisme dalam satu kerangka sistem berbasis *web*.

Secara praktis, hasil penelitian ini dapat membantu penulis, mahasiswa, dan peneliti dalam meningkatkan kualitas tulisan melalui alat bantu yang cepat, akurat, dan mudah digunakan untuk memeriksa ejaan serta mendeteksi kemiripan teks. Sistem yang dikembangkan juga diharapkan mampu mempermudah proses validasi keaslian dan kebersihan bahasa dalam karya ilmiah tanpa perlu menggunakan beberapa aplikasi terpisah, serta dapat menjadi solusi terpadu yang efisien untuk diterapkan maupun dikembangkan lebih lanjut di lingkungan akademik dan institusi pendidikan.

1.6 Sistematika Penulisan

Adapun sistematika penulisan laporan proposal penelitian ini yaitu sebagai berikut :

BAB I PENDAHULUAN

Bab ini berisi tentang deskripsi umum isi laporan yang meliputi latar belakang, rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, sistematika penulisan dan tinjauan pustaka.

BAB II LANDASAN TEORI

Bab ini berisi pembahasan teori-teori yang berhubungan dengan penelitian, meliputi penjelasan tentang *Natural Language Processing (NLP)*, algoritma SymSpell untuk pengecekan ejaan, konsep TF-IDF (*Term Frequency–Inverse Document Frequency*) dan *Cosine Similarity* untuk deteksi plagiarisme, serta teori pendukung lainnya seperti pengolahan teks, *tokenization*, *stemming*, dan *word segmentation*. Selain itu, bab ini juga membahas teori mengenai sistem berbasis *web*, arsitektur *client–server*, serta teknologi yang digunakan dalam

pengembangan sistem seperti *framework*, bahasa pemrograman, dan basis data.

BAB III METODE PENELITIAN

Bab ini menjelaskan mengenai metode yang digunakan dalam penelitian, meliputi waktu dan tempat penelitian, metode pengumpulan data, serta pendekatan pengembangan sistem yang diterapkan. Pada bagian ini juga dijelaskan tahapan penelitian yang mencakup analisis kebutuhan, perancangan sistem, implementasi, dan pengujian. Metode pembangunan sistem yang digunakan dalam penelitian ini disesuaikan dengan pendekatan pengembangan perangkat lunak berbasis *web* yang mendukung integrasi algoritma SymSpell dan TF-IDF serta *Cosine Similarity*.

BAB IV ANALISIS DAN PERANCANGAN SISTEM

Bab ini berisi analisis terhadap sistem yang akan dibangun, mencakup analisis kebutuhan fungsional dan nonfungsional, serta perancangan sistem secara menyeluruh. Perancangan meliputi rancangan arsitektur sistem, rancangan alur proses pengecekan ejaan dan deteksi plagiarisme, rancangan algoritma, rancangan basis data, serta rancangan antarmuka pengguna (*User interface*). Pada bagian ini juga dijelaskan alur integrasi antara modul pengecekan ejaan dan modul deteksi plagiarisme dalam satu platform analisis teks terpadu berbasis *web*.

BAB V IMPLEMENTASI DAN PENGUJIAN SISTEM

Bab ini menjelaskan implementasi dari sistem yang telah dirancang, termasuk tahapan instalasi, konfigurasi, serta penerapan algoritma SymSpell dan TF-IDF serta *Cosine Similarity* ke dalam sistem berbasis *web*. Selain itu, dilakukan

pengujian sistem untuk memastikan bahwa seluruh fungsi berjalan sesuai kebutuhan. Pengujian meliputi pengujian akurasi hasil pengecekan ejaan, tingkat kemiripan teks pada deteksi plagiarisme, serta evaluasi kinerja sistem berdasarkan waktu proses dan efisiensi penggunaan sumber daya.

1.7 Tinjauan Pustaka

Penelitian ini diperkuat oleh kajian literatur yang merujuk pada sejumlah studi sebelumnya terkait pengembangan sistem pendeteksi kesalahan ejaan dan plagiarisme dengan memanfaatkan metode SymSpell, TF-IDF, serta *Cosine Similarity*. Rangkuman kajian literatur tersebut disajikan pada Tabel 1.1.

Tabel 1. *1* Penelitian Terdahulu

No	Judul	Metode	Hasil
1	Analisis Perbandingan Metode *Burkhard Keller Tree* dan *SymSpell* dalam *Spell Correction* Bahasa Indonesia (Ferdiansyah dan Nuryana, 2023).	*Burkhard Keller dan SymSpell*	Metode BK Tree bekerja lebih lambat dengan waktu pemrosesan hingga 52 detik, tetapi membutuhkan sumber daya sistem yang kecil. Sebaliknya, SymSpell jauh lebih cepat dengan waktu maksimum 0,05 detik, namun memerlukan kapasitas sistem lebih besar. Dari sisi akurasi, kedua metode menunjukkan kinerja yang sama, dengan nilai

Tabel 1. *1* Penelitian Terdahulu

No	Judul	Metode	Hasil
			rata-rata *accuracy* 0,95, *precision* 0,89, dan *recall* 0,73.
2	*A Comparison Between SymSpell and a Combination of Damerau-Levenshtein Distance with the Trie Data Structure*(Auda h, Yuliawati dan Alfina, 2023)	*SymSpell* dan *Damerau-Levenshtein Distance*	Penelitian ini membandingkan dua metode koreksi non-word error Bahasa Indonesia: SymSpell (varian *weighted* dan *unweighted*) dan DLTrie. Dengan kamus berisi 91.557 kata dan *dataset* 58.532 salah eja, kinerja keduanya diuji pada kasus *isolated-word* dan *conteks-dependent*. Hasilnya, untuk kasus *isolated-word*, SymSpell unggul karena menghasilkan *best-match accuracy* lebih tinggi dan waktu proses lebih cepat dibandingkan DLTrie.
3	*Improving Document Retrieval with Spelling*	*SymSpell*	Penelitian ini membuktikan bahwa menambahkan koreksi ejaan SymSpell pada

Tabel 1. *1* Penelitian Terdahulu

No	Judul	Metode	Hasil
	Correction for Weak and Fabricated Indonesian-Translated Hadith (zaky ramadhan dan Lhaksmana, 2020)		sistem pencarian hadis palsu meningkatkan akurasi pencarian. Setelah koreksi ejaan digunakan, nilai *precision* naik dari 73% menjadi 81%, dan *recall* meningkat dari 80% menjadi 89%. Artinya, sistem menjadi lebih tepat dan lebih banyak menemukan hadis palsu yang benar. Ini menunjukkan bahwa koreksi ejaan sangat membantu karena banyak teks hadis di internet mengandung *typo*.
4	*Automatic Error Correction: Evaluating performance Of Spell Checker Tools* (Tolegenova, 2024)	SymSpell, NeuSpell, dan Hunspell	Penelitian ini mengevaluasi tiga alat pemeriksa ejaan—NeuSpell, SymSpell, dan Hunspell—untuk mengetahui efektivitasnya dalam mendeteksi dan memperbaiki kesalahan ejaan. Hasilnya

Tabel 1. *1* Penelitian Terdahulu

No	Judul	Metode	Hasil
			menunjukkan bahwa SymSpell memiliki kecepatan pemrosesan tertinggi dengan nilai 2480, menandakan performa yang jauh lebih cepat dibandingkan alat lainnya. Sementara itu, NeuSpell menghasilkan tingkat kesalahan terendah, yaitu 0,80%, sehingga lebih unggul dalam akurasi. Secara keseluruhan, masing-masing algoritma memiliki kelebihan dan kekurangan, dan masih terdapat peluang untuk pengembangan lebih lanjut.
5	*Evaluating Hunspell, SymSpell, Norvig, and N-gram Spellcheckers for Azerbaijani*	Hunspell, SymSpell, metode Norvig, dan model N-gram	Penelitian ini membandingkan empat algoritma pemeriksa ejaan untuk bahasa Azerbaijan yaitu Hunspell, SymSpell, metode Norvig, dan

Tabel 1. *1* Penelitian Terdahulu

No	Judul	Metode	Hasil
	Teks (Gasim, 2025).		model N-gram. Hunspell mencapai akurasi tertinggi (84,5%) karena mampu menangani morfologi dengan baik. SymSpell memiliki kecepatan terbaik dengan akurasinya (81,4%) yang bergantung pada ketersediaan kamus besar. Metode Norvig (78,3%) dan N-gram (82,1%) juga memiliki keterbatasan terkait korpus dan efisiensi.
6	"*Cosine Similarity-based Plagiarism Detection on Electronic Documents* (Sari, 2023)	*Cosinie Similarity dan* TF-IDF (*Term Frequency-Inverse Document Frequency*)	Penelitian ini berhasil membuktikan bahwa metode *Cosine Similarity* yang dikombinasikan dengan TF-IDF mampu mendeteksi tingkat kemiripan antar dokumen skripsi secara efektif**.** Sistem yang dikembangkan dapat mengidentifikasi kesamaan teks pada

Tabel 1. *1* Penelitian Terdahulu

No	Judul	Metode	Hasil
			berbagai bagian dokumen yang sebelumnya berpotensi luput dari pengawasan dosen pembimbing. *Cosine Similarity* terbukti akurat karena tidak dipengaruhi oleh panjang dokumen dan mampu mengukur kemiripan berdasarkan makna kata.
7	*Document Plagiarism Detection Application Using Web-Based TF-IDF and Cosine Similarity Methods*(Halim dan Lasut, 2024)	*Cosinie Similarity dan* TF-IDF (*Term Frequency-Inverse Document Frequency*)	Penelitian ini mengusulkan aplikasi berbasis *web* yang mengimplementasikan metode TF-IDF (*Term* Frequency-*Inverse Document Frequency*) dan *Cosine Similarity*. Proses deteksi di dalam aplikasi meliputi tiga tahap utama: *preprocessing* data, perhitungan bobot kata menggunakan *TF-IDF*, dan kalkulasi *Cosine Similarity* untuk

Tabel 1. *1* Penelitian Terdahulu

No	Judul	Metode	Hasil
			menentukan persentase kemiripan antara dokumen pengguna dan dokumen yang ada di *database*
8	*Design and Implementation of a Final Project Plagiarism Detection System Using Cosine Similarity Method*(Fauzi, Iqbal dan Haryanti, 2022)	*Cosine Similarity*	Penelitian ini menunjukkan bahwa sistem Kipcheck yang dikembangkan dengan metode *Cosine Similarity* mampu mendeteksi tingkat plagiarisme tugas akhir secara efektif**.** Sistem dapat menghitung persentase kemiripan antara dokumen yang diuji dan dokumen pembanding di dalam basis data dengan hasil yang konsisten dan akurat. Pengujian membuktikan bahwa sistem mampu memproses dokumen dengan jumlah kata tertentu, menghasilkan nilai kesamaan yang

Tabel 1. *1* Penelitian Terdahulu

No	Judul	Metode	Hasil
			sesuai dengan perhitungan manual, serta memberikan hasil yang stabil meskipun menggunakan skema perhitungan yang berbeda. Dengan demikian, Kipcheck layak digunakan sebagai alat bantu untuk mencegah plagiarisme dan mendukung penegakan integritas akademik**.**
9	*A Proposed Approach for Plagiarism Detection in Article Documents*(Saeed dan Taqa, 2022)	K-Means *Clustering*, TF-IDF, dan *Cosine Similarity*	Penelitian ini membuktikan bahwa penggabungan metode K-Means *Clustering*, TF-IDF, dan *Cosine Similarity* mampu mendeteksi tingkat plagiarisme dokumen artikel secara efektif**.** Sistem dapat mengelompokkan dokumen yang relevan, menghitung kemiripan teks secara akurat, serta

Tabel 1. *1* Penelitian Terdahulu

No	Judul	Metode	Hasil
			menghasilkan rasio plagiarisme yang digunakan untuk menentukan kelayakan dokumen berdasarkan ambang batas tertentu. Hasil pengujian menunjukkan tingkat plagiarisme yang bervariasi, sehingga sistem dinilai mampu mendukung evaluasi keaslian dokumen secara objektif.

Tabel 1. *1* Penelitian Terdahulu

No	Judul	Metode	Hasil
10	*Document Similarity Using Term Frequency-Inverse Document Frequency Representation and Cosine Similarity*(Widianto et al., 2024).	*Cosine Similarity* dan *Term Frequency-Inverse Document*(TF-IDF)	Penelitian ini membahas penerapan metode *document similarity* untuk dokumen panjang dengan menggunakan pendekatan TF-IDF dan *Cosine Similarity*. Hasil eksperimen pada *Document Similarity Triplets Dataset* menunjukkan bahwa metode ini mampu memberikan kinerja yang tinggi, dengan tingkat akurasi mencapai 93,6% ketika menggunakan representasi bigram saja. Temuan ini mengindikasikan bahwa TF-IDF cukup efektif dalam merepresentasikan karakteristik unik dokumen panjang serta mengurangi pengaruh kata-kata umum.

BAB II

TINJAUAN PUSTAKA

2.1 Pengolahan Bahasa Alami (*Natural Language Processing*)

Pemrosesan Bahasa Alami (Natural Language Processing) merupakan serangkaian teknik komputasional yang berlandaskan teori dan digunakan untuk menganalisis serta merepresentasikan teks-teks alami pada satu atau lebih tingkat

analisis linguistik, dengan tujuan mencapai pemrosesan bahasa yang menyerupai manusia untuk berbagai tugas atau aplikasi(Qi et al., 2020). Proses *natural language processing* (NLP) menggabungkan linguistik komputasi, pemodelan berbasis aturan bahasa manusia, pemodelan statistik, pembelajaran mesin, dan pembelajaran mendalam untuk memungkinkan komputer dan perangkat digital untuk mengenali, memahami, dan menghasilkan teks dan ucapan.

Penelitian dalam bidang NLP telah menjadi fondasi penting bagi hadirnya era AI *generatif*. Teknologi ini memungkinkan kemampuan komunikasi pada model bahasa besar (LLM) serta membuat model pembuat gambar dapat memahami instruksi pengguna. NLP kini telah hadir dalam kehidupan sehari-hari banyak orang, digunakan pada mesin pencari, chatbot layanan pelanggan berbasis suara, sistem navigasi GPS dengan perintah suara, hingga asisten digital di ponsel seperti Amazon Alexa, Apple Siri, dan Microsoft Cortana (Hagos, Battle dan Rawat, 2024).

NLP memfasilitasi interaksi dan kolaborasi antara manusia dan mesin dengan memungkinkan komunikasi melalui bahasa alami yang digunakan dalam kehidupan sehari-hari. Penerapannya memungkinkan berbagai peningkatan, seperti otomatisasi tugas yang bersifat repetitif, peningkatan kemampuan analisis serta penggalian wawasan dari data, optimalisasi proses pencarian informasi, dan mendukung proses generasi konten secara lebih efektif(Zhu et al., 2025).

Natural Language Processing (NLP) beroperasi dengan mengintegrasikan berbagai teknik komputasi untuk menganalisis, memahami, dan menghasilkan bahasa manusia dalam bentuk yang dapat diproses mesin. Secara umum, alur

kerja NLP meliputi beberapa tahap berikut(Darmanto, Pradasari dan Wahyudi, 2024):

1. Pemrosesan Teks

Pada tahap awal, data teks mentah dipersiapkan melalui proses prapemrosesan. Langkah ini mencakup tokenisasi untuk memisahkan teks menjadi unit dasar seperti kata atau kalimat, normalisasi huruf kecil untuk menyeragamkan penulisan, penghapusan *stopwords* yang dianggap tidak bermakna secara signifikan, serta *stemming* atau *lemmatization* untuk mengembalikan kata ke bentuk dasar. Pembersihan karakter non-esensial seperti tanda baca dan angka juga dilakukan agar teks lebih terstruktur dan siap diolah model *machine learning*.

2. Ekstraksi Fitur

Selanjutnya, teks diubah menjadi representasi numerik yang dapat dianalisis secara komputasional. Pendekatan ini dapat berupa metode tradisional seperti *Bag of Words* atau TF-IDF yang mengukur frekuensi dan bobot kata, hingga metode lanjutan seperti *word embeddings* (misalnya Word2Vec atau GloVe) dan penyematan kontekstual yang menangkap makna semantik berdasarkan konteks kemunculan kata.

3. Analisis Teks

Tahap ini bertujuan menafsirkan isi teks melalui teknik seperti *part-of-speech tagging*, *named entity recognition*, dependency parsing, analisis sentimen, serta topic modeling untuk menemukan tema dalam dokumen. Natural Language Understanding (NLU) memperdalam analisis dengan memfokuskan pada pemaknaan kalimat sehingga sistem mampu memahami konteks dan makna yang kompleks.

4. Pelatihan Model

Data terstruktur kemudian digunakan untuk melatih model machine learning. Selama proses ini, model mempelajari pola dan hubungan dalam data untuk menghasilkan prediksi atau keputusan terhadap teks baru. Kinerja model dievaluasi dan dioptimalkan secara berkelanjutan agar relevan dan akurat dalam penggunaan nyata.

Secara ringkas, NLP telah memberikan kontribusi besar di berbagai sektor industri. Dalam keuangan, teknologi ini mempercepat analisis dokumen untuk mendukung keputusan cepat. Di bidang kesehatan, NLP membantu menelaah catatan medis dan penelitian dengan lebih efisien. Industri asuransi memanfaatkannya untuk mengidentifikasi pola klaim dan meningkatkan proses operasional, sedangkan sektor hukum menggunakannya untuk mempercepat peninjauan dokumen dan pengelolaan informasi. Dengan demikian, NLP menjadi teknologi penting yang mendorong efisiensi dan transformasi digital lintas bidang.

2.2 Teks *Preprocessing*

Teks *preprocessing* merupakan serangkaian langkah untuk mengubah teks mentah yang berantakan menjadi representasi teks yang lebih terstruktur dan konsisten. Tahap ini penting dalam teks *mining*, klasifikasi teks, dan deteksi plagiarisme karena dapat mengurangi noise, menghilangkan kata yang tidak relevan, dan menormalkan bentuk kata sehingga hasil perhitungan kemiripan menjadi lebih akurat(Syam et al., 2024).

Dalam literatur teks *mining*, teks *preprocessing* dikenal sebagai bagian awal dari alur kerja penemuan pengetahuan pada data tekstual (*Knowledge Discovery in* Teks*ual Databases*). Berbagai penelitian yang menggunakan TF-IDF dan *Cosine Similarity* hampir selalu memasukkan tahap *preprocessing*

sebelum pembobotan dan pengukuran kemiripan dokumen (Tanuwijaya et al., 2025). Tahapan teks *preprocessing* umumnya meliputi beberapa langkah berikut :

1. *Cleaning*: proses membersihkan teks dengan menghilangkan elemen yang tidak diperlukan, seperti tanda baca tertentu, angka, URL, tag HTML, serta simbol lain yang tidak relevan dengan analisis.
2. *Case Folding*: langkah menyeragamkan seluruh huruf dalam teks, biasanya menjadi huruf kecil agar variasi penulisan seperti “Plagiat” dan “plagiat” dianggap sebagai kata yang sama.
3. *Tokenizing*: tahap memecah teks menjadi unit-unit kata (token), misalnya memisahkan suatu kalimat menjadi kumpulan kata yang dapat dihitung atau dianalisis lebih lanjut.
4. *Filtering / Stopword Removal*: proses menghilangkan kata-kata yang sering muncul namun tidak memiliki kontribusi makna signifikan, seperti “dan”, “yang”, atau “atau”, sehingga hanya kata penting yang dipertahankan sebagai indeks.
5. *Stemming / Lemmatization*: mengubah kata ke bentuk dasarnya, misalnya “menulis” dan “penulisan” menjadi “tulis” agar kata yang memiliki makna serupa tidak dianggap sebagai term yang berbeda.

Dalam sistem pengecekan ejaan dan deteksi plagiarisme, teks *preprocessing* berfungsi menyiapkan teks yang konsisten sebelum masuk ke modul SymSpell, TF-IDF, dan *Cosine Similarity*. Misalnya, setelah *cleaning* dan *case folding*, teks dapat dikoreksi terlebih dahulu ejaannya (*spelling correction*), kemudian dilakukan *tokenizing*, *stopword removal*, dan

stemming sehingga kata-kata yang dianalisis benar, relevan, dan dalam bentuk dasar.

2.3 Plagiarisme

Istilah plagiarisme berasal dari bahasa Latin *plagiarius*, yang berarti merampok atau membajak. Dalam konteks akademik, plagiarisme diartikan sebagai bentuk pencurian dan kebohongan intelektual. Kata *plagiarii* sendiri dalam karya epik merujuk pada perompak yang suka mencuri atau menculik anak. Dengan demikian, ketika seseorang melakukan plagiarisme, tindakan tersebut dapat disamakan dengan mencuri hasil pemikiran orang lain. Lebih dari itu, karena pelaku mengakuinya sebagai hasil karyanya sendiri, maka ia juga telah berbohong(Silalahi et al., 2024).

Seiring dengan perkembangan teknologi informasi, bentuk dan cara terjadinya plagiarisme juga mengalami perubahan. Saat ini, kita berada pada era “paperless”, yang ditandai dengan penyebaran hasil penelitian secara terbuka melalui internet (*open access*) menggunakan media digital, tanpa perlu dicetak di atas kertas (Silalahi et al., 2024).Perkembangan era digital dan internet ini membawa dampak terhadap meningkatnya potensi perilaku plagiarisme, karena akses terhadap karya orang lain menjadi semakin mudah(Sukowati dan Suciptaningsih, 2024). Bentuk plagiarisme pun telah berubah dari teks cetak menjadi teks digital yang tersebar di dunia maya. Dengan demikian, jika pada masa sebelum hadirnya internet dan komputer tindakan plagiarisme sulit terdeteksi, kini berbagai layanan daring memudahkan proses identifikasi dan pendeteksian terhadap adanya indikasi plagiarisme.

Plagiarisme bukanlah entitas monolitik. Ia hadir dalam berbagai bentuk, mulai dari yang terang-terangan hingga yang

subtil dan sulit dideteksi. Para ahli mengklasifikasikan plagiarisme berdasarkan modus operandi, objek yang dicuri, dan tingkat kesengajaan. Pemahaman mendalam mengenai tipologi ini krusial bagi deteksi dan pencegahan yang efektif. Berikut adalah jenis-jenis plagiarisme .

1. Plagiarisme Verbatim (*Direct/Cloning*)

Ini adalah bentuk plagiarisme yang paling primitif namun fatal. Plagiarisme verbatim terjadi ketika seseorang menyalin kata-kata secara langsung (*word-for-word transcription*) dari sumber lain tanpa menggunakan tanda kutip dan tanpa atribusi sama sekali. Dalam terminologi digital, ini dikenal sebagai *Copy and Paste*. Karakteristik utamanya adalah identitas teks yang hampir 100% sama dengan aslinya. Meskipun pelaku mengubah satu atau dua kata (misalnya mengganti "namun" dengan "tetapi"), jika struktur mayoritas kalimat tetap sama, tindakan ini tetap dikategorikan sebagai verbatim plagiarism. Pelaku jenis ini biasanya mengambil jalan pintas karena desakan waktu atau kemalasan intelektual(Sukowati dan Suciptaningsih, 2024).

2. Plagiarisme Mosaik (*Patchwork/Patchwriting*)

Plagiarisme mosaik merupakan bentuk kecurangan yang lebih halus dan sistematis. Tindakan ini sering disebut *patchwriting*, yaitu ketika penulis mengambil frasa, klausa, atau ide dari berbagai sumber kemudian merangkainya menjadi satu paragraf baru. Pelaku biasanya mempertahankan struktur kalimat asli, hanya mengganti beberapa kata dengan sinonim atau menyisipkan kata-kata sendiri tanpa memberikan penandaan atau sitasi yang jelas. Akibatnya, batas antara gagasan asli penulis dan sumber menjadi kabur, meskipun teks

tampak seperti hasil tulisan baru(Sukowati dan Suciptaningsih, 2024).

3. Plagiarisme Parafrase (*Inadequate Paraphrasing*)

Parafrase merupakan keterampilan akademik yang membutuhkan kemampuan tinggi, namun jika dilakukan secara tidak tepat dapat berubah menjadi bentuk plagiarisme. Kondisi ini terjadi ketika penulis hanya mengganti kata-kata dalam sumber dengan kalimat sendiri, tetapi mempertahankan alur dan struktur ide yang sama serta tidak memberikan sitasi. Banyak mahasiswa salah memahami bahwa perubahan kata saja sudah cukup untuk menyatakan bahwa ide tersebut menjadi milik pribadi, padahal anggapan ini keliru. Plagiarisme tidak hanya berkaitan dengan penyalinan kata-kata, tetapi juga pengambilan gagasan tanpa pengakuan. Karena itu, tanpa sitasi, parafrase yang baik sekalipun tetap dianggap sebagai tindakan plagiat(Sukowati dan Suciptaningsih, 2024).

4. Self-Plagiarism (*Auto-Plagiarisme dan* Teks *Recycling*)

Konsep ini sering memicu perdebatan karena secara intuitif seseorang merasa tidak mungkin mencuri dari dirinya sendiri. Namun, dalam etika akademik, *self-plagiarism* adalah pelanggaran serius terhadap integritas publikasi. Ini terjadi ketika penulis menggunakan kembali karya, data, atau tulisan mereka sendiri yang telah dipublikasikan sebelumnya dalam karya baru tanpa merujuk ke karya lama. Bentuk-bentuk spesifik dari self-plagiarism meliputi:

a. Teks *Recycling*: Menggunakan kembali porsi besar tulisan dari makalah lama di makalah baru.

b. *Double Submission*: Menyerahkan makalah yang sama untuk dua mata kuliah berbeda atau dua jurnal berbeda secara bersamaan.
c. *Salami Slicing*: Memecah hasil satu penelitian besar menjadi beberapa artikel kecil ("irisan salami") yang tumpang tindih secara signifikan untuk meningkatkan kuantitas publikasi secara artifisial. Hal ini mendistorsi rekam jejak ilmiah dan membuang sumber daya review sejawat.

5. Plagiarisme Ide (*Idea Plagiarism*)

Jenis plagiarisme ini merupakan bentuk yang paling abstrak dan sulit diidentifikasi oleh perangkat lunak deteksi kesamaan teks. Plagiarisme terjadi ketika seseorang mengadopsi gagasan orisinal, seperti kesimpulan, hipotesis, atau teori yang dirumuskan oleh pihak lain, kemudian menyajikannya seolah merupakan hasil pemikirannya sendiri. Proses pembuktian terhadap tindak plagiarisme ini biasanya memerlukan analisis kualitatif yang mendalam oleh ahli pada bidang ilmu terkait (Risparyanto, 2020).

6. Plagiarisme Struktur dan Gaya (*Structural Plagiarism*)

Penulis mereplikasi pola penalaran, alur penyajian data, atau struktur metodologi milik pihak lain secara hampir identik, meskipun topik yang dibahas berbeda. Praktik ini sering dikategorikan sebagai peniruan terhadap kerangka pikir atau *thinking template* orang lain(Pratiwi dan Aisya, 2021).

7. *Cyber Plagiarism* dan *Image Plagiarism*

Pada era digital, muncul fenomena *cyber plagiarism*, yaitu tindakan mengunduh dan menggunakan makalah secara utuh

dari *paper mill* atau menyalin konten daring tanpa mencantumkan atribusi. Selain teks, pengambilan elemen visual seperti grafik, diagram, foto, maupun video tanpa izin dan sitasi juga termasuk dalam kategori *image plagiarism*. Bahkan, tindakan memodifikasi gambar ilmiah untuk memperkuat hasil penelitian atau menghilangkan kekurangan tertentu dapat diklasifikasikan sebagai bentuk manipulasi data yang berkaitan dengan plagiarisme visual.

Plagiarisme merupakan permasalahan kompleks yang mengancam integritas pendidikan tinggi. Perkembangannya sejalan dengan kemajuan teknologi, bergeser dari penyalinan sederhana menjadi praktik lebih rumit seperti mosaik, *cyber plagiarism*, dan teks *recycling*. Fenomena ini tidak disebabkan oleh satu faktor, melainkan kombinasi antara tekanan akademik, peluang sistemik, serta rasionalisasi individu, yang sering diperburuk oleh rendahnya kemampuan menulis ilmiah.

Upaya penanggulangannya harus bersifat menyeluruh. Penegakan aturan melalui sanksi administratif dan hukum berperan sebagai efek jera, namun tidak cukup tanpa dukungan edukasi keterampilan sitasi dan parafrase, pemanfaatan sistem deteksi plagiarisme, dan penguatan budaya akademik yang menekankan proses. Melalui perpaduan literasi, teknologi, dan regulasi, integritas akademik dapat dipertahankan secara berkelanjutan.

2.4 *Typographical Error*

Typographical error atau yang umum disingkat menjadi *typo* merupakan kesalahan yang muncul pada proses pengetikan, baik berupa kekeliruan dalam penulisan ejaan, salah ketik, maupun kelalaian dalam menuliskan suatu kata secara lengkap. Kesalahan ini umumnya disebabkan oleh

ketidaktelitian atau kurangnya perhatian saat melakukan proses *input* teks. Salah satu bentuk kesalahan ketik yang paling sering ditemukan adalah transposisi huruf, yaitu kondisi ketika seluruh huruf yang diperlukan untuk membentuk sebuah kata telah dituliskan namun tersusun dalam urutan yang tidak semestinya. Fenomena ini biasanya muncul ketika seseorang mengetik dengan cepat sehingga tidak sempat memeriksa ulang susunan karakter yang dihasilkan.

Selain transposisi, substitusi huruf juga merupakan bentuk kesalahan ketik yang kerap terjadi. Substitusi ini umumnya dipicu oleh *slip of the finger*, yaitu terpencetnya tombol *keyboard* yang berdekatan secara tidak sengaja sehingga menghasilkan huruf yang berbeda dari yang dimaksudkan. Kesalahan-kesalahan tersebut dapat terjadi sebagai akibat dari faktor mekanis perangkat *input* ataupun ketidaksesuaian koordinasi antara gerakan tangan dan kehendak pengetik pada saat memasukkan karakter (Prianto, Markuci dan Pane, 2023).

Contoh *typographical error* dapat berupa duplikasi karakter, hilangnya salah satu huruf, maupun tertukarnya posisi huruf dalam suatu kata . Misalnya, pada kasus kekurangan huruf, kata “nasi” dapat terketik menjadi “nsi”; pada kasus kelebihan huruf, kata yang sama dapat berubah menjadi “nasii”; sedangkan pada kasus kesalahan penempatan atau transposisi huruf, kata “nasi” dapat terketik menjadi “nsai” . Ketiga bentuk kesalahan tersebut merupakan pola *typo* yang sangat sering dijumpai dalam aktivitas pengetikan sehari-hari, baik dalam konteks profesional, akademik, maupun komunikasi digital informal .

2.5 *Spelling Correction*

Spelling correction bertujuan mengidentifikasi kata yang salah tulis (*misspelled word*) dan memberikan satu atau beberapa kandidat kata yang benar berdasarkan data kamus atau korpus. Teknik ini umum digunakan pada aplikasi penulisan, mesin pencari, maupun sistem teks mining untuk meningkatkan kualitas data teks sebelum dianalisis lebih lanjut(Prianto, Markuci dan Pane, 2023).

Kesalahan ejaan umumnya dibagi menjadi *non-word error* (kata tidak ada di kamus, misalnya “plagiarime”) dan *real-word error* (kata valid tetapi salah konteks, misalnya “di” vs “d i”). *Spelling correction* biasanya lebih fokus dulu pada *non-word error* dengan mengecek keberadaan kata dalam kamus dan mencarikan koreksi yang paling mirip.

Secara umum, terdapat tiga kelompok pendekatan utama dalam spelling correction:

1. *Dictionary lookup*: setiap kata dicek ke kamus; jika tidak ditemukan, sistem mencari kata terdekat di kamus sebagai kandidat koreksi.
2. *Similarity* / *minimum edit distance*: kemiripan antara kata salah dan kata kandidat diukur menggunakan jarak edit seperti Levenshtein atau Damerau–Levenshtein.
3. *Rule-based dan similarity key*: menggunakan aturan linguistik (misalnya pola huruf yang sering tertukar) atau kunci fonetik/visual untuk membuat kelompok kata yang mungkin saling menggantikan.

Pada sistem modern, *spelling correction* sering digabungkan dengan informasi frekuensi kata (seberapa sering sebuah kata muncul di korpus) agar kandidat yang paling umum dan relevan dipilih sebagai koreksi utama.

2.6 Symspell

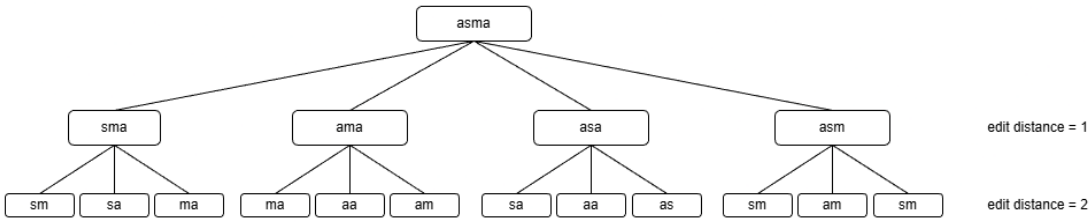
Algoritma SymSpell (*Symmetric Delete Spelling Correction*) diperkenalkan oleh Wolf Garbe pada 2012–2015 dan menjadi populer karena efisiensinya yang tinggi dibanding metode seperti Norvig, BK-*tree*, dan Levenshtein tradisional. Pendekatan ini menggunakan pra-komputasi daftar hapusan (*delete dictionary*) dari setiap kata dalam kamus, sehingga pencarian koreksi ejaan dapat dilakukan dalam waktu hampir konstan (*O(1)*).

Algoritma SymSpell menurunkan kompleksitas pencarian dengan menggunakan operasi *deletion* saja. Tidak seperti pendekatan Peter Norvig yang menghitung seluruh kemungkinan *edit-distance* pada kata yang diuji, metode Wolf Garbe menerapkan penghapusan pada kata uji dan pada seluruh entri kamus. Karena penghapusan pada kamus dilakukan satu kali saat pra-pemrosesan, proses pencocokan menjadi jauh lebih cepat.

Secara umum, algoritma SymSpell terdiri atas lima tahap utama(Shafiera, 2022):

1. Melakukan operasi penghapusan huruf pada seluruh entri kamus dan menyimpan hasilnya sebagai *dataset*.
2. Memuat dataset hasil pra-pemrosesan tersebut.
3. Ketika sebuah kata terdeteksi sebagai *typo*, dilakukan operasi penghapusan huruf pada kata tersebut.
4. Setiap hasil penghapusan kemudian dicocokkan dengan dataset.
5. Jika terdapat kecocokan, kata yang terkait ditambahkan sebagai kandidat koreksi.

SymSpell menetapkan nilai *edit distance* sebagai batas maksimum jumlah iterasi operasi penghapusan pada suatu kata. Nilai *edit distance* tertinggi biasa yang digunakan oleh SymSpell adalah 2. Dengan *edit distance* = 2, maka proses penghapusan dilakukan sebanyak dua kali baik pada kata yang diuji maupun pada seluruh entri dalam kamus. Hal yang sama berlaku jika nilai *edit distance* ditetapkan menjadi 1, maka operasi penghapusan hanya dilakukan satu kali pada kata yang diuji maupun pada entri kamus (Shafiera, 2022).



Gambar 2. 1 Proses penghapusan kata

Kata-kata yang telah melalui proses penghapusan beserta bentuk aslinya disimpan sebagai dasar pemeriksaan *typo*. Proses penghapusan kemudian dilakukan pada kata yang diuji. Hasil penghapusan dari kata uji dicocokkan dengan hasil penghapusan pada *dictionary*. Kecocokan antara keduanya akan menghasilkan kata asli dari *dictionary* yang selanjutnya dimasukkan ke dalam daftar *suggestion*. Contohnya proses proses pencarian antara kata yang *typo* (“asms”) dan kata yang berasal dari kamus (“asma”) ditunjukkan pada Tabel 2.1.

Tabel 2. 1 Proses pencocokan kata menggunakan algoritma symspell

Kata yang typo	Kata pada kamus
asms	asma
sms	sma
ams	ama

ass	asa
asm	asm
as	as

Secara keseluruhan, mekanisme penghapusan yang menjadi dasar kerja SymSpell memberikan efisiensi signifikan dalam proses koreksi ejaan. Pendekatan ini mampu mempercepat pencarian kandidat kata yang relevan tanpa mengorbankan ketepatan hasil. Implementasi pra-pemrosesan pada kamus, ditambah pencocokan berbasis hasil penghapusan, menjadikan SymSpell sebagai salah satu algoritma koreksi ejaan yang sangat efektif untuk diterapkan pada sistem pemrosesan teks berskala besar.

2.7 *Cosine Similarity*

Cosine Similarity adalah metode untuk menghitung kesamaan (*similarity*) antara dua objek yang dinyatakan dalam bentuk vektor bernilai riil non-negatif. Dalam konteks teks *mining* dan *Natural Language Processing* (NLP), metode ini digunakan untuk mengukur seberapa mirip dua dokumen teks tanpa memandang ukuran atau panjang dokumen tersebut(Sari, 2023).

Prinsip kerja *Cosine Similarity* didasarkan pada pengukuran kosinus sudut antara dua vektor dalam ruang multidimensi (biasanya disebut *Vector Space Model*). Jika kedua vektor memiliki arah yang sama, maka nilai kosinus sudutnya adalah 1, yang mengindikasikan bahwa kedua dokumen identik. Sebaliknya, jika kedua vektor tegak lurus (membentuk sudut 90

derajat), nilai kosinusnya adalah 0, yang menandakan bahwa kedua dokumen tidak memiliki kemiripan sama sekali.

Keunggulan utama *Cosine Similarity* dibandingkan metode pengukuran jarak lain (seperti *Euclidean Distance*) adalah sifatnya yang menormalisasi panjang dokumen. Artinya, sebuah dokumen pendek dan dokumen panjang tetap dapat dianggap "mirip" jika keduanya menggunakan komposisi kata-kata yang relevan sama, meskipun frekuensi total katanya berbeda jauh. Hal ini sangat krusial dalam deteksi plagiarisme di mana dokumen yang dijiplak mungkin telah mengalami penyuntingan sebagian(Pawestri dan Suyanto, 2024).

Secara matematis, jika diberikan dua dokumen yang direpresentasikan sebagai vektor dan vector , maka nilai *Cosine Similarity* didefinisikan sebagai hasil kali titik (*dot product*) antara kedua vektor dibagi dengan hasil kali besar (*magnitude*) kedua vektor tersebut (Euclidean norm).

Rumus persamaan *Cosine Similarity* dapat dituliskan sebagai berikut:

(2.1)

Keterangan :

1. : *Dot product* (hasil kali titik) antara vektor dokumen A dan dokumen B.
2. : *Magnitude* (panjang/norma) dari vektor A.
3. : *Magnitude* (panjang/norma) dari vektor B.
4. : Bobot kata ke-i dalam dokumen A (menggunakan bobot TF-IDF).
5. : Bobot kata ke-i dalam dokumen B (menggunakan bobot TF-IDF).

6. n : Jumlah kata unik (*term*) yang ada dalam seluruh korpus dokumen.

Nilai keluaran dari perhitungan *Cosine Similarity* berada dalam rentang interval [0, 1] untuk pembobotan kata yang positif (seperti TF-IDF), dengan interpretasi sebagai berikut:

1. Nilai mendekati 1: Menunjukkan sudut antara dua vektor mendekati 0 derajat, yang berarti kedua dokumen memiliki tingkat kemiripan yang sangat tinggi (substansi sama).
2. Nilai mendekati 0: Menunjukkan sudut antara dua vektor mendekati 90 derajat, yang berarti kedua dokumen sangat berbeda atau tidak memiliki kata-kata yang sama (independen).

Dalam implementasi sistem deteksi plagiarisme, nilai *similarity* ini biasanya dikonversi menjadi persentase (dikalikan 100%) untuk memberikan gambaran tingkat kemiripan dokumen uji terhadap dokumen referensi. Kombinasi metode ini dengan pembobotan TF-IDF (*Term Frequency-Inverse Document Frequency*) terbukti efektif karena TF-IDF memberikan bobot lebih pada kata-kata yang unik dan penting, sementara *Cosine Similarity* mengukur kedekatan orientasi vektor dari bobot-bobot tersebut.

2.8 *Ter-Frequency Inverse Document Frecuency* (TF-IDF)

TF-IDF merupakan singkatan dari *term frequency–inverse document frequency*, yaitu ukuran statistik yang banyak digunakan pada temu balik informasi dan teks *mining*. Ide utamanya adalah memberi bobot tinggi pada kata yang sering muncul di suatu dokumen tetapi jarang muncul di dokumen lain, sehingga kata tersebut dianggap representatif terhadap isi dokumen (Widianto et al., 2024).

Secara umum, nilai TF-IDF untuk sebuah term *t* pada dokumen *d* dalam korpus *D* dirumuskan sebagai:

(2.2)

Rumus ini menunjukkan bahwa bobot akhir adalah hasil kali antara frekuensi kemunculan term di dokumen dan tingkat "kelangkaan" *term* di seluruh korpus (Septiani et al., 2022).

Term Frequency (TF) menyatakan seberapa sering suatu *term* muncul dalam satu dokumen tertentu. Bentuk paling sederhana menghitung TF sebagai jumlah kemunculan sebuah kata dibagi dengan jumlah total kata dalam dokumen, sehingga menghasilkan frekuensi relatif (Septiani et al., 2022). Rumus dasar untuk menghitung TF adalah:

(2.3)

Keterangan :

1. : Jumlah kemunculan kata *t* dalam dokumen *d.*
2. : Total jumlah seluruh kata dalam dokumen *d*.

Inverse Document Frequency berfungsi untuk mengurangi bobot kata-kata yang terlalu umum (seperti "yang", "dan", "di") yang muncul di hampir semua dokumen. Kata-kata umum ini dianggap memiliki daya pembeda yang rendah. IDF dihitung dengan membagi total jumlah dokumen dengan jumlah dokumen yang mengandung kata tersebut, kemudian diambil nilai logaritmanya. Rumus IDF adalah sebagai berikut:

(2.4)

Keterangan :

1. *N* : Total jumlah dokumen dalam korpus.
2. : Jumlah dokumen dimana kata *t* muncul.

Dalam konteks sistem pengecekan ejaan dan deteksi plagiarisme berbasis *web*, TF-IDF dapat digunakan untuk

merepresentasikan setiap dokumen sebagai vektor bobot kata. Vektor TF-IDF ini kemudian menjadi *input* bagi perhitungan *Cosine Similarity* untuk mengukur tingkat kemiripan antara dokumen uji dan dokumen pembanding di dalam basis data.

Dengan menggunakan TF-IDF, sistem dapat lebih fokus pada kata-kata yang benar-benar mencerminkan isi dokumen, bukan pada kata-kata umum, sehingga deteksi plagiarisme menjadi lebih sensitif terhadap kesamaan konten bermakna, bukan sekadar kecocokan kata-kata biasa.

2.9 *Facebook AI Similarity Search* (FAISS)

Faiss (*Facebook AI Similarity Search*) merupakan *library* sumber terbuka yang dikembangkan oleh Facebook/Meta untuk melakukan pencarian kemiripan (*similarity search*) dan *clustering* pada vektor berdimensi tinggi. *Library* ini dirancang untuk menangani pencarian *nearest neighbor* pada kumpulan vektor berukuran besar secara efisien dari sisi waktu komputasi dan penggunaan memori, serta mendukung akselerasi GPU. Dengan memanfaatkan teknik pengindeksan vektor, Faiss memungkinkan sistem melakukan pencarian kandidat vektor yang relevan dengan cepat meskipun jumlah data yang diproses sangat besar(Ong, 2024).

Dalam sistem deteksi plagiarisme berbasis *web*, Faiss berperan sebagai mekanisme percepatan pencarian dokumen kandidat yang memiliki kemiripan tinggi. Vektor TF-IDF dari dokumen referensi disimpan dalam indeks Faiss, kemudian vektor TF-IDF dari teks masukan pengguna digunakan sebagai kueri untuk menemukan dokumen paling mirip secara efisien. Selanjutnya, nilai kemiripan akhir tetap dihitung menggunakan *Cosine Similarity* sebagai dasar penentuan indikasi plagiarisme,

sehingga Faiss berfungsi sebagai pendukung efisiensi sistem tanpa menggantikan metode pengukuran kemiripan utama.

2.10 Python

Python merupakan bahasa pemrograman tingkat tinggi yang dibuat dengan tujuan agar mudah dipelajari dan digunakan. Dikembangkan oleh Guido van Rossum pada akhir tahun 1980-an dan resmi dirilis pada 1991, Python kemudian tumbuh menjadi salah satu bahasa pemrograman yang paling serbaguna. Dengan sintaks yang ringkas dan menyerupai bahasa Inggris, Python lebih mudah dipahami dibandingkan sejumlah bahasa pemrograman lainnya(Muis dan Muhammad, 2023).

Sebagai bahasa multi-paradigma, Python mendukung pemrograman prosedural, berorientasi objek, dan fungsional, sehingga fleksibel untuk berbagai kebutuhan pengembangan perangkat lunak. Python juga bersifat lintas platform dan dapat dijalankan pada berbagai sistem operasi seperti Windows, Linux, dan macOS.

Dalam pengembangan aplikasi *web*, Python banyak digunakan karena tersedianya *framework* seperti Django dan Flask yang memudahkan pengaturan routing, basis data, serta autentikasi. *Framework* ini mendukung pengembangan cepat dan struktur kode yang terorganisasi, sehingga sesuai untuk membangun sistem pengecekan ejaan dan deteksi plagiarisme berbasis *web*.

Di bidang *data mining*, teks *mining*, dan *machine learning*, Python menjadi bahasa yang sangat populer berkat pustaka seperti NumPy, Pandas, dan Scikit-Learn yang mendukung pengolahan data numerik maupun tekstual. Dalam konteks skripsi ini, pustaka tersebut dapat digunakan untuk perhitungan

TF-IDF, pembentukan vektor dokumen, dan penghitungan *Cosine Similarity* untuk mengukur tingkat kemiripan teks(Muis dan Muhammad, 2023).

Selain itu, Python juga mendukung implementasi algoritma SymSpell melalui pengolahan *string*, struktur data *dictionary*, serta manajemen korpus kata yang efisien. Dukungan pustaka koneksi basis data dan *framework web* mempermudah integrasi antara modul logika seperti SymSpell, TF-IDF, dan *Cosine Similarity* dengan antarmuka *web*, sehingga pengguna dapat mengunggah dokumen, memeriksa ejaan, dan memperoleh hasil deteksi kemiripan dalam satu sistem berbasis Python.

2.11 FastAPI

FastAPI adalah salah satu *framework* modern untuk membangun API berbasis Python yang dirancang agar cepat, ringan, dan mudah digunakan. Framework ini memanfaatkan tipe data Python (*type hints*) untuk menghasilkan dokumentasi otomatis serta validasi data yang kuat. Keunggulan utama FastAPI terletak pada performanya yang sangat tinggi, mendekati kecepatan *framework* asinkron lain seperti Node.js dan Go, berkat penggunaan *ASGI* (Asynchronous *Server Gateway Interface*)(Khandelwal1, 2024).

FastAPI mendukung penggunaan *async* dan *await*, sehingga sangat sesuai untuk membangun aplikasi yang membutuhkan banyak operasi I/O seperti komunikasi dengan *database* atau layanan eksternal. Dengan sintaks yang sederhana dan struktur yang rapi, FastAPI memungkinkan pengembang membangun aplikasi berskala kecil hingga besar dengan cepat, terukur, dan tetap mudah dipelihara(Azhari, 2022).

Dalam konteks sistem pengecekan ejaan dan deteksi plagiarisme berbasis *web*, FastAPI menawarkan solusi yang

ideal untuk mengintegrasikan algoritma SymSpell, TF-IDF, dan *Cosine Similarity* ke dalam arsitektur *backend* yang efisien dan skalabel.

2.12 *React*.js

React (atau *React* JS) merupakan *library* JavaScript *open-source* yang digunakan untuk membangun antarmuka pengguna, khususnya pada bagian frontend aplikasi *web*. Teknologi ini pertama kali dikembangkan oleh Facebook dan kini memiliki ekosistem serta komunitas yang luas sehingga banyak diadopsi dalam berbagai proyek industri.

React mengadopsi konsep pemecahan UI menjadi komponen-komponen kecil yang berdiri sendiri dan dapat digunakan kembali. Pendekatan berbasis komponen ini membuat struktur kode lebih teratur serta memudahkan pengembangan fitur baru tanpa mengganggu fungsi yang telah berfungsi sebelumnya.

React memiliki beberapa keunggulan sebagai teknologi *frontend*. Pertama, *React* memanfaatkan Virtual DOM untuk mengurangi manipulasi DOM secara langsung, sehingga proses pembaruan tampilan menjadi lebih cepat dan efisien. Kedua, *React* mendukung penggunaan komponen yang dapat dipakai kembali, sehingga satu komponen seperti formulir unggah dokumen atau tabel hasil pemeriksaan plagiarisme dapat digunakan di berbagai halaman. Ketiga, *React* menggunakan sintaks JSX yang menggabungkan *JavaScript* dengan struktur mirip HTML, sehingga penulisan tampilan menjadi lebih intuitif dan mudah dipahami (Nasution dan Iswari, 2021).

Dalam sistem pengecekan ejaan dan deteksi plagiarisme, *React* berfungsi sebagai lapisan *frontend* yang berinteraksi langsung dengan pengguna. *React* dapat digunakan untuk

membangun halaman *input* teks atau unggah dokumen, menampilkan saran perbaikan ejaan secara dinamis, serta menyajikan visualisasi hasil analisis seperti persentase kemiripan dan daftar dokumen terkait secara interaktif tanpa perlu memuat ulang seluruh halaman.

React juga mendukung komunikasi yang efisien dengan *backend* (misalnya API berbasis Python), baik untuk mengirim teks yang akan diproses maupun menerima hasil perhitungan SymSpell, TF-IDF, dan *Cosine Similarity*. Data yang diterima kemudian dapat dirender dengan cepat dan responsif untuk memberikan pengalaman pengguna yang optimal.

2.13 *Unified Modeling Languange (UML)*

Unified Modeling Language (UML) merupakan seperangkat notasi dan diagram yang digunakan untuk merepresentasikan struktur serta perilaku suatu sistem dalam bentuk visual. Penggunaan UML memudahkan pengembang dalam memahami kebutuhan sistem, alur proses, dan hubungan antar komponen sebelum tahap implementasi, sehingga potensi kesalahan dalam perancangan dapat diminimalkan (Ramdany et al., 2024).

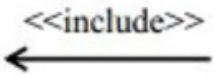
Dalam konteks rekayasa perangkat lunak, UML berperan sebagai *blueprint* yang memfasilitasi komunikasi antara analis, pengembang, dan pemangku kepentingan non-teknis. Pemodelan sistem menggunakan UML turut mendukung proses pemeliharaan serta pengembangan jangka panjang, karena dokumentasi desain tersusun secara sistematis dan mudah dipahami. Adapun jenis-jenis diagram UML antara lain :

2.11.1 *Use Case* Diagram

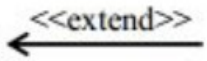
Use Case diagram adalah diagram yang digunakan untuk merepresentasikan interaksi antara sistem dan aktor yang terlibat. Diagram ini menyajikan gambaran umum mengenai

fungsi-fungsi utama sistem tanpa menjelaskan detail proses secara mendalam. Adapun symbol yang digunakan dalam *Use Case* dapar dilihat pada Tabel 2.2 (Ramdany et al., 2024) :

Tabel 2. *2* Simbol diagram *Use Case*

Simbol	Nama	Keterangan
	Actor	Mewakili peran orang, sistem yang lain, atau alat ketika berkomunikasi dengan *Use Case*
	Use Case	Abstraksi dan interaksi antara sistem dan aktor
	Association	Abstraksi dari penghubung antara aktor dengan *Use Case*
	Generalisasi	Menunjukkan spesialisasi aktor untuk dapat berpartisipasi dengan *Use Case*
	*Include*	Menunjukkan bahwa suatu *Use Case* seluruhnya merupakan fungsionalitas dari *Use Case* lainnya

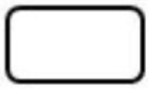
Tabel 2. *2* Simbol diagram *Use Case*

Simbol	Nama	Keterangan
	*Exclude*	Menunjukkan bahwa suatu *Use Case* merupakan tambahan fungsional dari *Use Case* lainnya jika suatu kondisi terpenuhi

2.11.2 *Activity Diagram*

Activity Diagram merupakan salah satu komponen penting dalam *Unified Modeling Language* (UML) yang digunakan untuk memvisualisasikan alur kerja, proses, atau aktivitas di dalam suatu sistem. Diagram ini menunjukkan keterkaitan antar tindakan serta bagaimana sistem berpindah dari satu keadaan ke keadaan lainnya. Dengan memberikan gambaran yang jelas mengenai alur kerja sederhana maupun kompleks, *Activity Diagram* membantu pengembang dan pemangku kepentingan memahami interaksi antar elemen dalam sistem. Adapun notasi yang digunakan *Activity Diagram* dapat dilihat pada Tabel 2.3 (Ramdany et al., 2024):

Tabel 2. *3* Notasi acitvity diagram

Simbol	Nama	Keterangan
	Status Awal	Sebuah diagram aktivitas memiliki sebuah status awal.
	Aktivitas	Aktivitas yang dilakukan sistem, aktivitas biasanya diawali dengan kata kerja.

Tabel 2. *3* Notasi acitvity diagram

Simbol	Nama	Keterangan
	Percabangan / *Decision*	Percabangan dimana ada pilihan aktivitas yang lebih dari satu.
	Penggabungan / *Join*	Penggabungan dimana yang ada lebih dari satu aktivitas lalu digabungkan jadi satu.
	Status Akhir	Status akhir yang dilakukan sistem, sebuah *Activity Diagram* harus memiliki status akhir.
	Swimlane	*Swimlane* memisahkan organisasi bisnis yang bertanggung jawab terhadap aktivitas yang terjadi.

2.11.3 *Class* Diagram

Class diagram merupakan salah satu representasi dalam *Unified Modeling Language* (UML) yang berfungsi memodelkan struktur statis suatu sistem melalui pendefinisian kelas beserta atribut, operasi, dan relasinya. Diagram ini menyediakan deskripsi konseptual terhadap komponen-komponen sistem serta pola keterkaitan yang membentuk arsitektur perangkat lunak. Adapun notasi yang digunakan *class* diagram dapat dilihat pada Tabel 2.4 (Arianti et al., 2022) :

Tabel 2. 4 Simbol *class* diagram

Simbol	Nama	Keterangan

	Association	Relasi antar kelas
	Class	Merepresentasikan sebuah kelas yang ditulis dalam program.

2.11.4 *Sequence Diagram*

Sequence Diagram berfokus pada representasi perilaku suatu sistem dengan menunjukkan bagaimana setiap objek saling berinteraksi. Di dalam *Sequence Diagram*, ditampilkan objek-objek beserta pesan yang dipertukarkan di antara objek tersebut. Diagram ini umumnya dimanfaatkan untuk memodelkan alur interaksi objek dalam suatu *Use Case*. Satu *Use Case* pada prinsipnya membutuhkan satu *Sequence Diagram*; namun apabila *Use Case* tersebut memiliki beberapa skenario, masing-masing skenario dapat digambarkan sebagai fragmen dalam *Sequence Diagram* yang sama(Arianti et al., 2022). Adapun symbol dari *sequence* dapat dilihat pada Tabel 2.5.

Tabel 2. *5* Simbol *Sequence Diagram*

Simbol	Nama	Keterangan
	Actor	*User*, orang, proses, atau sistem lain yang berinteraksi dengan sistem dan mendapatkana
	Activation Box	Mewakili periode dimana suatu elemen melakukan

Tabel 2. *5* Simbol *Sequence Diagram*

Simbol	Nama	Keterangan
		operasi
	Lifeline	Mendeskripsikan sebuah objek dalam sebuah sistem atau salah satu komponennya
	Message	Menggambarkan pesan atau interaksi antar objek
	Message to self	Menggambarkan pesan balikan atau reaksi dari objek sebelumnya

2.14 *Rational Unified Process* (RUP)

Rational Unified Process (RUP) merupakan metodologi pengembangan perangkat lunak yang diperkenalkan oleh Rational Software Corporation, yang kini berada di bawah naungan IBM. RUP berfungsi sebagai kerangka kerja proses yang bersifat terstruktur, iteratif, dan bertahap, serta dirancang untuk mendukung tim dalam menghasilkan perangkat lunak yang berkualitas tinggi. Dalam pengembangan sistem pengecekan ejaan dan deteksi plagiarisme berbasis *web*, RUP memberikan pendekatan yang sistematis sehingga seluruh tahapan pengembangan, mulai dari analisis kebutuhan hingga penerapan sistem—dapat terlaksana dengan terencana dan terkendali.

Terdapat empat fase utama dalam siklus proyek RUP. Keempat fase ini dapat direpresentasikan secara umum menyerupai pendekatan *waterfall*, meskipun prinsip fundamental RUP sebenarnya terletak pada proses iteratif di setiap fasenya. Setiap fase memiliki tujuan utama serta titik

pencapaian tertentu yang digunakan untuk menilai keberhasilan penyelesaian tujuan tersebut. Representasi visual keempat fase ini sepanjang sumbu waktu dikenal sebagai grafik RUP. Adapun penjelasan masing-masing fase adalah sebagai berikut:

1. Fase Insepsi (*Inception*)

Tujuan utama fase ini adalah mendefinisikan ruang lingkup sistem secara memadai sebagai dasar untuk memvalidasi estimasi biaya dan perencanaan anggaran awal. Pada tahap ini, disusun *business case* yang mencakup konteks bisnis, faktor-faktor keberhasilan (misalnya estimasi pendapatan dan strategi masuk pasar), serta proyeksi finansial. Selain itu, dirancang pula model penggunaan (*Use Case model*), rencana proyek, penilaian risiko awal, serta deskripsi proyek secara komprehensif.

2. Fase Elaborasi (*Elaboration*)

Sasaran utama dari fase elaborasi adalah mengurangi atau mengatasi risiko-risiko kritis yang telah diidentifikasi pada tahap sebelumnya. Pada fase ini, bentuk proyek mulai tampak lebih jelas. Analisis domain disusun secara lebih mendalam, sementara arsitektur sistem mulai dihasilkan dalam bentuk kerangka dasar yang akan menjadi fondasi bagi fase-fase berikutnya.

3. Fase Konstruksi (*Construction*)

Fokus utama fase konstruksi adalah mewujudkan sistem perangkat lunak melalui proses pembangunan komponen dan pengembangan fitur inti. Tahap ini merupakan periode dengan intensitas pengkodean yang tinggi. Pada proyek berskala besar, fase ini biasanya mencakup beberapa iterasi konstruksi yang bertujuan memecah *Use Case* ke dalam unit-unit yang lebih

terkelola dan menghasilkan purwarupa fungsional secara bertahap.

4. Fase Transisi (*Transition*)

Tujuan utama fase ini adalah memindahkan sistem dari lingkungan pengembangan ke lingkungan operasional sehingga dapat digunakan oleh pengguna akhir. Kegiatan pada fase transisi meliputi pelatihan pengguna, pelatihan pengelola sistem, serta pengujian beta untuk memastikan bahwa sistem telah sesuai dengan kebutuhan dan ekspektasi pengguna akhir(Arief dan Sugiarti, 2022).

2.15 Pengujian *Confusion Matrix*

Confusion matrix adalah tabel evaluasi standar untuk mengukur performa algoritma dalam machine learning, terutama pada data uji. *Confusion matrix* membandingkan prediksi model dengan label aktual, menunjukkan jumlah klasifikasi benar dan salah (Sathyanarayanan, 2024). *Confusion matrix* memiliki elemen-elemen pada Tabel 2.6.

Tabel 2.6 Komponen *confusion matrix*

Komponen	Keterangan
TP(*True Positive*)	Aktual positif, diprediksi positif
TN (*True Negative*)	Aktual negatif, diprediksi negatif
FP(*False Positive*)	Aktual negatif, diprediksi positif
FN(*False Negative*)	Aktual positif, diprediksi negatif

Berdasarkan *confusion matrix* tersebut, kinerja sebuah algoritma akan diukur menggunakan beberapa metrik evaluasi.

1. Akurasi

Akurasi direncanakan digunakan untuk mengukur tingkat ketepatan sistem dalam mengklasifikasikan seluruh pasangan dokumen (Sathyanarayanan, 2024).

(2.5)

2. *Precision*

Precision direncanakan digunakan untuk mengukur ketepatan sistem dalam mengidentifikasi dokumen yang dikategorikan sebagai plagiarisme (Sathyanarayanan, 2024).

(2.6)

3. *Recall*

Recall direncanakan digunakan untuk mengukur kemampuan sistem dalam mendeteksi seluruh dokumen yang benar-benar termasuk kategori plagiarism (Sathyanarayanan, 2024).

(2.7)

4. *F1-Score*

F1-score direncanakan digunakan untuk memperoleh keseimbangan antara nilai *precision* dan *recall* (Sathyanarayanan, 2024).

(2.8)

Output yang diharapkan dari rencana pengujian ini adalah diperolehnya nilai metrik evaluasi yang dapat menggambarkan kinerja algoritma TF-IDF dan *Cosine Similarity* secara objektif. Hasil pengujian tersebut diharapkan dapat menjadi dasar dalam menilai kelayakan algoritma sebagai metode pendeteksian plagiarisme dokumen teks.

2.16 *User Acceptance Testing* (UAT)

User Acceptance Testing merupakan tahap pengujian yang dilakukan untuk menilai tingkat penerimaan pengguna terhadap sistem yang dikembangkan. Pengujian ini bertujuan untuk memastikan bahwa sistem telah berjalan sesuai dengan kebutuhan pengguna dan tujuan yang telah ditetapkan. Pada tahap UAT, sistem diuji dari sudut pandang pengguna akhir, sehingga fokus pengujian tidak hanya pada fungsi teknis, tetapi juga pada kemudahan penggunaan, kejelasan informasi, serta kelayakan sistem untuk digunakan secara nyata (Wulandari, Nofiyani dan Hasugian, 2023).

Dalam penelitian ini, UAT digunakan untuk mengevaluasi sistem analisis teks terpadu berbasis web yang dikembangkan untuk pengecekan ejaan menggunakan algoritma SymSpell serta deteksi plagiarisme menggunakan metode TF-IDF dan *Cosine Similarity*. Pengujian dilakukan dengan melibatkan pengguna untuk memberikan penilaian berdasarkan pengalaman mereka saat menggunakan sistem.

2.17 Skala Likert

Skala Likert merupakan metode pengukuran yang digunakan untuk mengetahui sikap, pendapat, atau persepsi responden terhadap suatu objek atau sistem. Skala ini digunakan karena mampu memberikan gambaran kuantitatif terhadap tingkat penerimaan pengguna secara sistematis dan mudah dianalisis (Wulandari, Nofiyani dan Hasugian, 2023).

Dalam penelitian ini, skala Likert digunakan untuk menghitung penilaian responden pada *User Acceptance Testing* (UAT) terhadap sistem analisis teks terpadu berbasis web yang dikembangkan. Penilaian diberikan melalui sejumlah pernyataan yang berkaitan dengan aspek kemudahan penggunaan sistem,

kejelasan hasil analisis, kinerja fitur pengecekan ejaan menggunakan algoritma SymSpell, kinerja fitur deteksi plagiarisme menggunakan metode TF-IDF dan *Cosine Similarity*, serta kelayakan sistem secara keseluruhan.

Hasil penilaian responden dihitung dengan cara menjumlahkan skor dari setiap jawaban berdasarkan bobot skala Likert. Skor total yang diperoleh kemudian dibandingkan dengan skor maksimum untuk mendapatkan persentase tingkat penerimaan pengguna (Sasmito et al., 2024). Perhitungan persentase dilakukan menggunakan rumus berikut:

(2,5)

Keterangan :

1. Skor aktual merupakan pilihan dari semua responden dari kuesioner yang telah diberikan.
2. Skor ideal diasumsikan bahwa semua responden memilih skor tertinggi dari semua jawaban.

Persentase kelayakan yang diperoleh dari perhitungan Skala Likert pada pengujian *User Acceptance Testing* (UAT) digunakan sebagai dasar evaluasi untuk menilai tingkat penerimaan dan kelayakan sistem analisis teks terpadu berbasis web yang dikembangkan. Nilai persentase tersebut memberikan gambaran objektif mengenai persepsi pengguna terhadap fungsionalitas, kemudahan penggunaan, serta kinerja fitur utama sistem. Hasil evaluasi ini selanjutnya menjadi acuan dalam menentukan apakah sistem telah memenuhi kebutuhan pengguna dan layak untuk diimplementasikan, sekaligus sebagai bahan pertimbangan dalam melakukan penyempurnaan atau pengembangan lebih lanjut.

BAB III
METODOLOGI PENELITIAN

3.1. Metode Pengumpulan Data

Metode Pengumpulan data yang digunakan dalam penelitian ini adalah sebagai berikut :

3.1.1 Studi Literatur

Studi literatur dilakukan dengan mengumpulkan berbagai referensi yang berkaitan dengan algoritma SymSpell, TF-IDF, dan *Cosine Similarity* sebagai dasar metodologis dalam pembangunan sistem. Proses ini mencakup penelusuran jurnal ilmiah, artikel konferensi, buku, dan sumber daring terpercaya untuk memahami konsep, mekanisme kerja, serta kelebihan dan keterbatasan dari masing-masing algoritma.

3.1.2 Pengumpulan *Dataset*

Pengumpulan *dataset* dilakukan untuk mengumpulkan data yang mendukung fungsi pengecekan ejaan dan deteksi plagiarism. Kebutuhan data dalam penelitian ini mencakup dua jenis dataset yang masing-masing mendukung fungsi utama sistem, yaitu pengecekan ejaan dan deteksi plagiarisme. Kedua jenis data tersebut dikumpulkan dan diolah dengan ketentuan sebagai berikut:

1. Kamus Frekuensi Kata Bahasa Indonesia

Dataset kamus digunakan untuk mengoptimalkan kinerja algoritma SymSpell pada modul koreksi ejaan. Data diperoleh dari *platform* Kaggle dalam format CSV sebanyak 100.000 kata. Kamus ini berfungsi sebagai kamus referensi (*lookup dictionary*) yang memungkinkan SymSpell menentukan kandidat koreksi ejaan dengan probabilitas paling relevan.

2. Dokumen Referensi Deteksi Plagiarisme

Dataset ini digunakan sebagai basis pembanding pada modul deteksi plagiarisme. Dokumen berupa skripsi atau jurnal mahasiswa yang diperoleh dari repositori institusi, serta artikel dari jurnal akses terbuka. Jumlah dokumen yang dikumpulkan sebanyak 100 dokumen. Seluruh dokumen melalui proses praproses berupa pembersihan untuk menghapus metadata atau elemen non-substansial lainnya. Dokumen yang telah bersih kemudian dibobotkan menggunakan TF-IDF dan direpresentasikan sebagai vektor, yang selanjutnya digunakan dalam perhitungan tingkat kemiripan menggunakan *Cosine Similarity*.

3.2. Metode Pengembangan Sistem

Dalam penelitian ini, metode pengembangan sistem yang digunakan yaitu *Rational Unified Process* (RUP), yang terdiri dari 4 tahapan sebagai berikut :

1. *Inception* (Permulaan)

Pada tahap *inception*, peneliti melakukan identifikasi kebutuhan dasar sistem serta ruang lingkup penelitian. Aktivitas utama meliputi analisis masalah yang dihadapi pengguna, yaitu perlunya sebuah sistem berbasis *web* yang mampu melakukan pengecekan ejaan serta mendeteksi plagiarisme secara otomatis dan akurat. Pada tahap ini juga dilakukan pengumpulan kebutuhan fungsional dan non-fungsional, seperti fitur unggah dokumen, proses koreksi ejaan menggunakan algoritma SymSpell, dan deteksi kemiripan dokumen menggunakan TF-IDF serta *Cosine Similarity*. Selain itu, peneliti menetapkan tujuan penelitian, aktor sistem, batasan, serta studi kelayakan awal untuk memastikan bahwa sistem dapat dibangun sesuai dengan sumber daya dan teknologi yang tersedia.

2. *Elaboration* (Perencanaan)

Tahap *elaboration* berfokus pada perencanaan lebih detail, perancangan arsitektur sistem, serta pendefinisian kebutuhan teknis yang lebih spesifik. Peneliti merumuskan struktur basis data yang digunakan untuk menyimpan dokumen, kamus kata, serta hasil perhitungan TF-IDF. Pada tahap ini juga dilakukan perancangan alur proses algoritma, seperti cara SymSpell melakukan pencarian kata dan koreksi ejaan, serta bagaimana TF-IDF digunakan untuk membobot dokumen sebelum akhirnya dihitung tingkat kemiripannya menggunakan *Cosine Similarity*. Risiko pengembangan dianalisis, termasuk potensi kesalahan dalam preprocessing data teks, performa komputasi saat memproses dokumen panjang, dan kebutuhan optimasi algoritma. Hasil utama dari tahap ini adalah desain sistem, rancangan UI/UX, serta pemilihan teknologi untuk implementasi.

3. *Constuction* (Konstruksi)

Tahap *construction* merupakan fase implementasi kode program berdasarkan desain yang telah dibuat. Aktivitas mencakup pembangunan modul pengecekan ejaan menggunakan algoritma SymSpell, termasuk pembuatan *dictionary*, preprocessing teks, dan implementasi proses pencocokan kata. Selanjutnya, peneliti mengembangkan modul deteksi plagiarisme dengan menerapkan TF-IDF untuk pembobotan kata dan *Cosine Similarity* untuk mengukur tingkat kemiripan antar dokumen. Pada tahap ini juga dibangun antarmuka *web* yang memungkinkan pengguna mengunggah dokumen, melihat hasil koreksi ejaan, dan menerima laporan tingkat plagiarisme. Sistem kemudian diuji secara bertahap menggunakan metode *unit testing* dan *user acceptance*

testing(UAT) untuk memastikan seluruh modul berjalan sesuai kebutuhan.

4. *Transition* (Transisi)

Pada tahap *transition*, sistem yang telah dibangun dipersiapkan untuk digunakan oleh pengguna akhir. Peneliti melakukan uji coba sistem secara langsung melalui *User* testing untuk menilai keakuratan koreksi ejaan dan performa deteksi plagiarisme pada berbagai jenis dokumen. Penyempurnaan dilakukan berdasarkan umpan balik pengguna agar sistem lebih mudah digunakan dan memberikan hasil yang lebih relevan. Dokumentasi sistem, panduan penggunaan, serta laporan penelitian akhir juga diselesaikan pada tahap ini. Terakhir, sistem di-*deploy* pada *server* atau lingkungan hosting sehingga dapat diakses melalui *web*, menandai transisi dari tahap pengembangan menuju tahap operasional.

3.3. Waktu dan Tempat Penelitian

3.3.1 Waktu

Penelitian berlangsung dalam kurun waktu 4 bulan yang dimulai dari bulan Januari 2026 – April 2026. Adapun rincian kegiatan yang dilaksanakan disajikan pada Tabel 3.1.

Tabel 3. 1 *Timeline* penelitian

No.	Uraian	Waktu Penelitian (2026)															
		Januari				Ferbuari				Maret				April			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	*inception*																

2	*Elaboration*																
3	*Construction*																
4	*Transition*																

3.3.2 Tempat Penelitian

Penelitian tugas akhir ini bertempat di Laboratorium *Software Enggineering*, Jurusan Teknik Informatika, Fakultas Teknik, Universitas Halu Oleo.

3.4. Analisis Kebutuhan Sistem

3.4.1. Kebutuhan Fungsional

Pada tahap analisis kebutuhan fungsional, terdapat sejumlah kebutuhan pokok yang harus dipenuhi dalam pengembangan sistem, yang meliputi analisis kebutuhan *input*, analisis kebutuhan proses, serta analisis kebutuhan *output*.

1. Analisis Kebutuhan *Input*

Analisis kebutuhan *input* menjelaskan jenis data yang harus dimasukkan oleh pengguna maupun administrator agar sistem dapat menjalankan fungsinya dengan baik. Pada sistem pengecekan ejaan dan deteksi plagiarisme berbasis *web* ini, data masukan utama berupa dokumen teks yang akan dianalisis. Dokumen tersebut dapat dimasukkan dalam bentuk teks langsung melalui editor pada halaman *web* atau melalui unggahan *file* dengan format tertentu, seperti .pdf atau .docx.

Selain dokumen utama, sistem juga memerlukan data pendukung berupa korpus atau basis data dokumen pembanding yang digunakan dalam proses deteksi plagiarisme. Data ini berfungsi sebagai referensi dalam perhitungan tingkat

kemiripan menggunakan metode TF-IDF dan *Cosine Similarity*. Adapun pada proses pengecekan ejaan, sistem memerlukan kamus kata sebagai data *input* awal untuk mendukung algoritma SymSpell dalam mendeteksi dan mengoreksi kesalahan penulisan kata.

2. Analisis Kebutuhan Proses

Analisis kebutuhan proses menjelaskan tahapan pengolahan data yang dilakukan oleh sistem berdasarkan *input* yang telah diberikan. Pada sistem ini, proses diawali dengan tahap prapemrosesan teks, yang meliputi normalisasi huruf, tokenisasi, serta penghapusan karakter yang tidak diperlukan.

Proses pengecekan ejaan dilakukan menggunakan algoritma SymSpell untuk mendeteksi kesalahan penulisan kata dan memberikan saran perbaikan yang sesuai. Setelah proses koreksi ejaan selesai, dokumen akan diproses lebih lanjut untuk keperluan deteksi plagiarisme. Pada tahap ini, sistem menerapkan metode TF-IDF untuk menghitung bobot kata dalam dokumen, kemudian dilanjutkan dengan perhitungan tingkat kemiripan antara dokumen uji dan dokumen pembanding menggunakan algoritma *Cosine Similarity*. Hasil perhitungan tersebut digunakan untuk menentukan tingkat kemiripan teks yang mengindikasikan adanya potensi plagiarisme.

3. Analisis Kebutuhan *Output*

Analisis kebutuhan *output* menjelaskan informasi yang dihasilkan oleh sistem setelah seluruh proses pengolahan data selesai dilakukan. Pada sistem pengecekan ejaan dan deteksi plagiarisme ini, keluaran utama berupa hasil koreksi ejaan yang

menampilkan kata-kata yang terdeteksi salah beserta saran perbaikannya.

Selain itu, sistem juga menghasilkan laporan tingkat kemiripan dokumen dalam bentuk persentase yang menunjukkan indikasi plagiarisme. *Output* ini dapat dilengkapi dengan informasi tambahan, seperti bagian teks yang memiliki tingkat kemiripan tinggi dengan dokumen pembanding. Seluruh hasil analisis ditampilkan melalui antarmuka *web* agar mudah dipahami oleh pengguna dan dapat digunakan sebagai bahan evaluasi terhadap keaslian serta kualitas penulisan dokumen.

3.4.2. Kebutuhan *Non*-fungsional

Analisis kebutuhan nonfungsional merupakan suatu proses pengkajian terhadap sumber daya yang dibutuhkan dalam pengembangan aplikasi agar sistem dapat beroperasi secara optimal. Kebutuhan nonfungsional ini diklasifikasikan ke dalam dua kategori utama. Pertama, kebutuhan perangkat keras yang meliputi komponen fisik atau perangkat yang diperlukan untuk mendukung kinerja sistem. Kedua, kebutuhan perangkat lunak yang mencakup berbagai aplikasi, sistem pendukung, dan *framework* yang berfungsi untuk mengoptimalkan pemanfaatan perangkat keras serta menjamin keberlangsungan operasional sistem secara efektif dan andal.

1. Kebutuhan Perangkat Keras

Perangkat keras yang digunakan dalam pengembangan sistem dapat dilihat pada Tabel 3.2.

Tabel 3. 2 Kebutuhan Perangkat Keras

No	Nama Perangkat	Spesifikasi
1	Laptop	ASUS VivoBook 14
2	Processor	Amd Athlon Gold 3150U

3	Monitor	14 *Inch*
4	Memori	RAM 12 GB
5	*Storage*	512 GB

2. Kebutuhan Perangkat Lunak

Perangkat lunak yang digunakan dalam pengembangan sistem dapat dilihat pada Tabel 3.3.

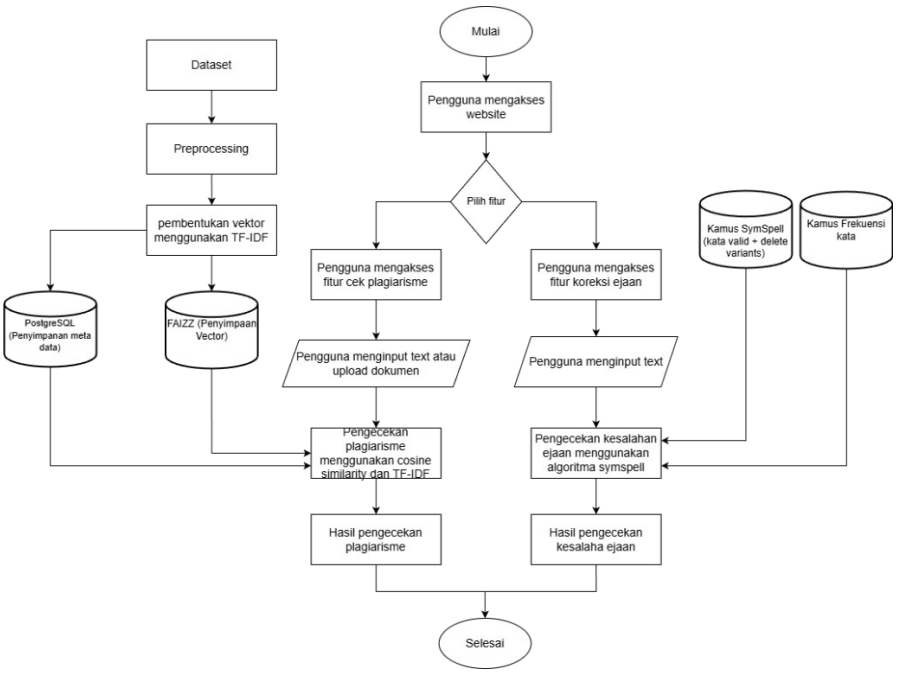
Tabel 3. *3* Kebutuhan Perangkat Lunak

No	Nama Perangkat	Spesifikasi
1	Sistem Operasi	Ubuntu 24.04 LTS
2	Brave	Versi 1.85.118
3	Python	Versi 3.12
4	Postgree SQL	Versi 18.1
5	Docker	Versi 28.5.1
6	FastAPI	Versi 0.128.0
7	*React* JS	Versi 19.2
8	Node JS	Versi 22.21.1
9	Visual Studio Code	Versi 1.107.1

3.5. Analisis Perancangan Sistem

3.5.1 Perancangan Alur Sistem Secara Umum

Bagian ini menjelaskan perancangan alur sistem secara umum pada sistem pendeteksi kesalahan ejaan menggunakan algoritma SymSpell serta pengecekan plagiarisme menggunakan metode TF-IDF dan *Cosine Similarity*. Perancangan alur sistem ini bertujuan untuk menggambarkan tahapan kerja sistem secara menyeluruh, mulai dari proses pengolahan teks masukan, pendeteksian dan koreksi kesalahan ejaan, hingga pengukuran tingkat kemiripan dokumen untuk mendukung proses pengecekan plagiarisme. Dengan perancangan alur yang terstruktur, sistem diharapkan mampu bekerja secara efisien dan menghasilkan keluaran yang akurat sesuai dengan tujuan penelitian.



Gambar 3. 1 Alur sistem secara umum

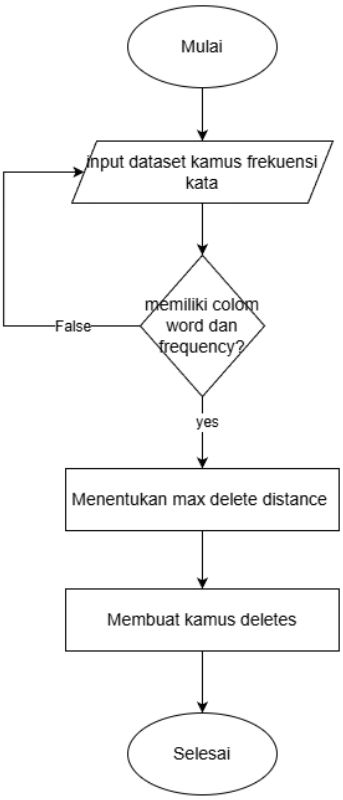
Alur sistem dimulai ketika pengguna mengakses *web*site sistem pengecekan ejaan dan deteksi plagiarisme. Selanjutnya, sistem menyediakan pilihan fitur kepada pengguna, yaitu fitur pengecekan plagiarisme atau fitur koreksi ejaan. Apabila pengguna memilih fitur pengecekan plagiarisme, pengguna dapat meng*input* teks atau mengunggah dokumen yang akan dianalisis, kemudian sistem memproses dokumen tersebut dengan menerapkan metode TF-IDF untuk pembobotan kata dan algoritma *Cosine Similarity* guna menghitung tingkat kemiripan dokumen berdasarkan *database* yang sudah disiapkan. Hasil analisis ditampilkan dalam bentuk persentase tingkat kemiripan sebagai indikator potensi plagiarisme. Sebaliknya, jika pengguna memilih fitur koreksi ejaan, pengguna meng*input* teks yang akan diperiksa, lalu sistem melakukan pengecekan kesalahan penulisan menggunakan algoritma SymSpell dan menampilkan hasil koreksi berupa daftar kesalahan ejaan

beserta saran perbaikannya. Proses sistem diakhiri setelah hasil analisis ditampilkan kepada pengguna.

3.5.2 Perancangan Alur Kerja Algoritma Symspell

Algoritma SymSpell (*Symmetric Delete Spelling Correction*) bekerja melalui dua alur utama, yaitu proses pembuatan kamus sebagai tahap *preprocessing* dan proses pencarian koreksi **(***lookup***)** yang digunakan untuk mendeteksi serta memperbaiki kesalahan ejaan pada kata.

1. Proses Pembuatan Kamus *Deletes*



Gambar 3. 2 Alur pembuatan kamus algoritma symspell

Penjelasan rinci mengenai alur pembuatan kamus SymSpell yang ditampilkan pada gambar 3.2 adalah sebagai berikut:

a. Mulai : Proses diawali dengan inisialisasi sistem untuk membangun kamus koreksi kata.

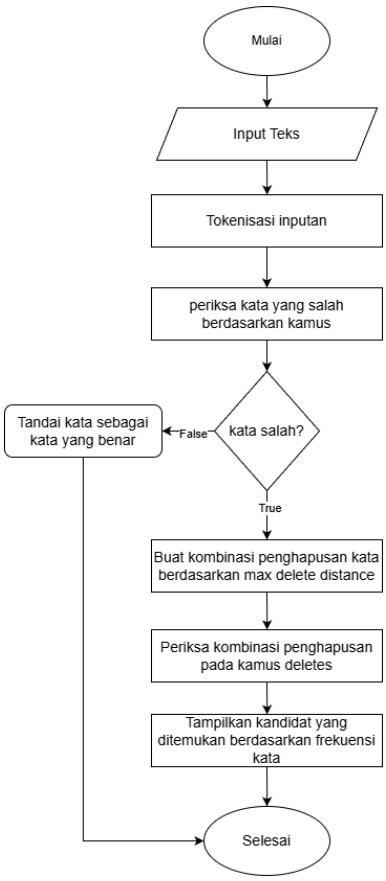
b. *Dataset* : Sistem menerima dataset berupa kumpulan kata yang dilengkapi dengan nilai frekuensi kemunculan.

c. Menentukan *Maximum Delete Distance* : Sistem menetapkan nilai maksimum jarak penghapusan (*maximum delete distance*) yang akan digunakan sebagai batas jumlah karakter yang dapat dihapus dari sebuah kata.

d. Membuat Kamus *Deletes* : Setiap kata dalam kamus utama diproses dengan menghasilkan seluruh kemungkinan variasi kata hasil penghapusan karakter hingga mencapai batas *maximum delete distance*. Variasi hasil penghapusan ini disimpan dalam struktur kamus *deletes,* yang memetakan kata hasil penghapusan terhadap kata asli dalam kamus.

e. Selesai : Proses pembuatan kamus berakhir setelah seluruh kata dan variasi penghapusannya berhasil disimpan dan siap digunakan pada tahap pencarian (*lookup*).

2. Proses Pencarian Koreksi (*Lookup*)



Gambar 3. 3 Alur proses Pencarian Kandidat Kata yang Benar

Penjelasan rinci mengenai alur pencarian kata yang benar menggunakan algoritma SymSpell yang ditampilkan pada gambar 3.3 adalah sebagai berikut:

a. Mulai : Proses *lookup* dimulai ketika sistem menerima teks masukan dari pengguna.

b. *Input* Teks : Sistem menerima kalimat atau teks yang akan dianalisis untuk mendeteksi kemungkinan kesalahan penulisan kata.

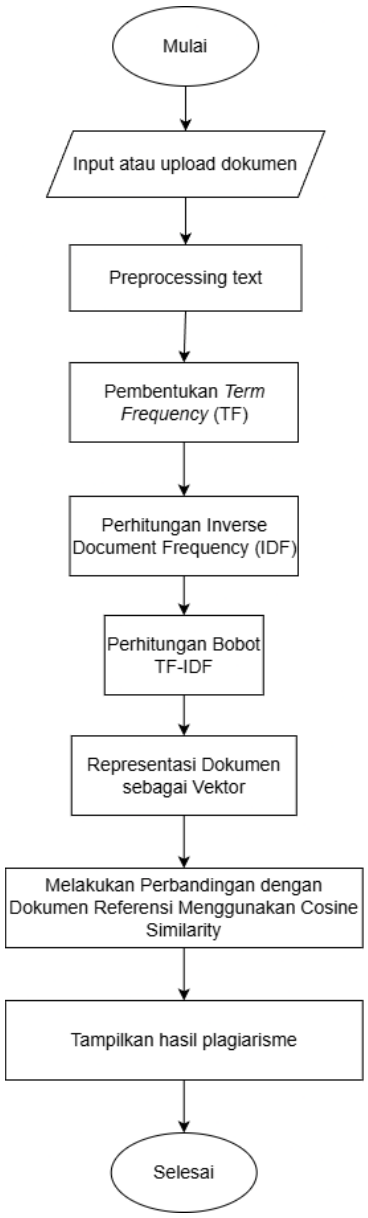
c. Tokenisasi *Input*an : Teks masukan dipecah menjadi unit-unit kata untuk memungkinkan pemrosesan setiap kata secara individual.

d. Pemeriksaan Kata Salah Berdasarkan Kamus : Setiap token dibandingkan dengan kamus utama. Token yang tidak ditemukan dalam kamus dianggap sebagai kata yang berpotensi salah eja dan diproses lebih lanjut.
e. Membuat Kombinasi Penghapusan Kata Salah Berdasarkan *Maximum Delete Distance* : Untuk setiap kata yang terdeteksi salah, sistem menghasilkan kombinasi variasi kata dengan melakukan operasi penghapusan karakter hingga batas *maximum delete distance*.
f. Pemeriksaan Kombinasi Penghapusan pada Kamus *Deletes*: Variasi penghapusan yang dihasilkan dibandingkan dengan entri pada kamus *deletes* untuk menemukan kata-kata dalam kamus utama yang memiliki pola penghapusan yang sama.
g. Mengumpulkan Semua Kandidat yang Ditemukan **:** Seluruh kata kandidat yang berhasil ditemukan melalui pencocokan dengan kamus *deletes* dikumpulkan sebagai kandidat koreksi.
h. Menampilkan Kandidat yang Diurutkan Berdasarkan Frekuensi Kemunculan Kata : Kandidat kata diurutkan berdasarkan nilai frekuensi kemunculannya dalam kamus. Kandidat dengan frekuensi tertinggi diprioritaskan sebagai rekomendasi koreksi yang paling relevan.
i. Selesai : Proses *lookup* berakhir setelah sistem menampilkan hasil rekomendasi kata kepada pengguna.

3.5.3 Perancangan Alur Kerja Algoritma *Cosine Similarity* dan Metode TF-IDF

Bagian ini membahas perancangan alur kerja algoritma *Cosine Similarity* dan metode *Term Frequency–Inverse Document Frequency* (TF-IDF) yang digunakan dalam proses

pengolahan dan analisis data teks. Perancangan alur kerja ini bertujuan untuk menggambarkan tahapan sistematis dalam merepresentasikan dokumen ke dalam bentuk vektor numerik serta mengukur tingkat kemiripan antar dokumen berdasarkan bobot kata yang dihasilkan. Dengan alur kerja yang terstruktur, penerapan metode TF-IDF dan *Cosine Similarity* diharapkan mampu menghasilkan pengukuran kesamaan dokumen yang akurat dan relevan sesuai dengan kebutuhan sistem yang dikembangkan.



Gambar 3. 4 Alur kerja algoritma *Cosine Similarity* dan metode TF-IDF

Penjelasan rinci mengenai alur kerja algoritma *Cosine Similarity* dan metode TF-IDF yang ditampilkan pada gambar 3.3 adalah sebagai berikut:

a. Mulai : Tahap awal proses yang menandai dimulainya sistem dalam melakukan pengolahan dan analisis teks.

Pada tahap ini, sistem dipersiapkan untuk menerima dan memproses data dokumen.

b. *Preprocessing* Teks : *Dataset* teks yang telah dikumpulkan selanjutnya diproses melalui tahap *preprocessing* untuk meningkatkan kualitas data. Tahapan ini umumnya meliputi *case folding*, *tokenisasi*, *stopword removal*, dan *stemming*. Tujuan dari tahap ini adalah mengurangi noise dan menstandarkan bentuk kata dalam dokumen.

c. Pembentukan *Term Frequency* (TF) : Setelah proses *preprocessing*, sistem menghitung nilai *Term Frequency (TF)* untuk setiap kata dalam dokumen. TF merepresentasikan jumlah kemunculan suatu term dalam sebuah dokumen, yang menunjukkan tingkat kepentingan relatif kata tersebut di dalam dokumen.

d. Perhitungan *Inverse Document Frequency* (IDF) : Sistem menghitung nilai *Inverse Document Frequency (IDF)* untuk setiap term berdasarkan distribusi kemunculannya di seluruh dokumen. IDF digunakan untuk menurunkan bobot kata yang sering muncul di banyak dokumen dan meningkatkan bobot kata yang bersifat lebih spesifik.

e. Perhitungan Bobot TF-IDF : Nilai TF dan IDF digabungkan untuk menghasilkan bobot *TF-IDF* bagi setiap term dalam dokumen. Bobot ini mencerminkan tingkat kepentingan suatu kata terhadap sebuah dokumen dalam keseluruhan koleksi dokumen.

f. Representasi Dokumen sebagai Vektor : Setiap dokumen direpresentasikan dalam bentuk vektor numerik yang terdiri dari bobot TF-IDF dari seluruh term. Representasi vektor ini memungkinkan dokumen dianalisis secara matematis.

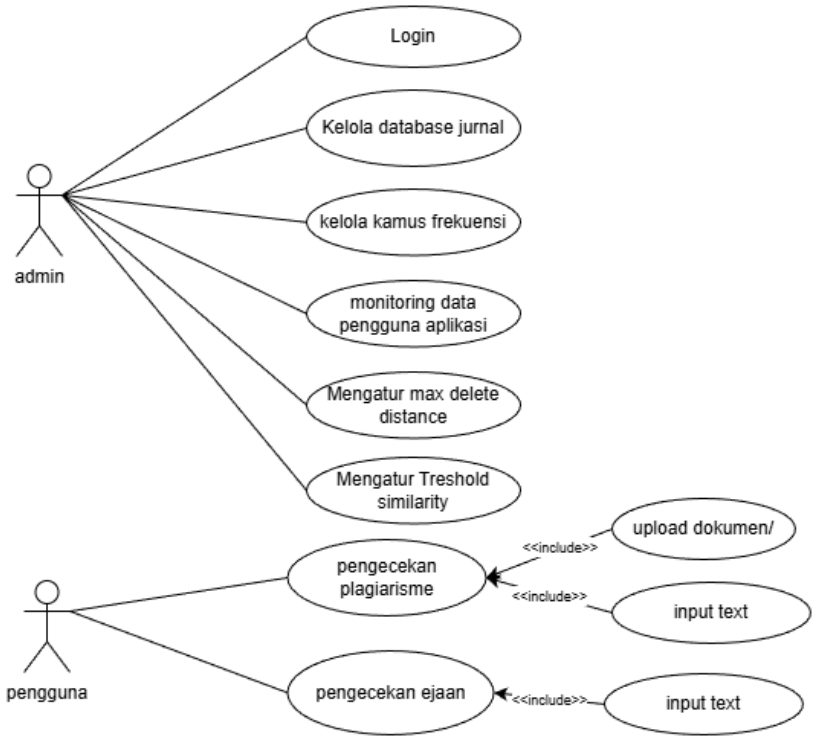
g. Perhitungan *Cosine Similarity* : Sistem menghitung tingkat kemiripan antar dokumen yang di *input User* dan dokumen yang ada dalam *database* dengan menggunakan metode *Cosine Similarity*. Metode ini mengukur sudut kosinus antara dua vektor dokumen, di mana nilai yang lebih tinggi menunjukkan tingkat kemiripan yang lebih besar.
h. Interpretasi Hasil : Hasil perhitungan *Cosine Similarity* diinterpretasikan untuk menentukan hubungan atau tingkat relevansi antar dokumen. Tahap ini dapat digunakan untuk keperluan pencarian dokumen, rekomendasi, atau pengelompokan data teks.
i. Selesai : Tahap akhir proses, di mana sistem menyajikan hasil analisis kepada pengguna dan seluruh rangkaian pemrosesan dinyatakan selesai.

3.6. Perancangan *Unified Modeling Language* (UML)

Dalam proses perancangan, sistem dimodelkan menggunakan *Unified Modeling Language* (UML) melalui beberapa diagram, yaitu *Use Case diagram*, *Activity Diagram*, *class diagram*, dan *Sequence Diagram*.

3.6.1 *Use Case* Diagram

Use Case diagram digunakan untuk menggambarkan fungsionalitas sistem dari sudut pandang pengguna, termasuk interaksi antara aktor dan sistem. Diagram ini membantu mengidentifikasi kebutuhan fungsional sistem tanpa memperhatikan detail implementasi teknis. Adapun *Use Case diagram* untuk sistem ini ditunjukkan pada gambar 3.5.



Gambar 3. 5 *Use Case diagram*

Use Case diagram diatas mendeskripsikan *User* yang menggunakan sistem dan bagaimana cara *User* berinteraksi dengan sistem.

a. Admin dan pengguna diwajibkan melakukan proses *login* dengan memasukkan *Username* dan *password* untuk dapat mengakses sistem. Khusus bagi pengguna, proses registrasi harus dilakukan terlebih dahulu untuk membuat akun yang akan digunakan pada tahap *login*.

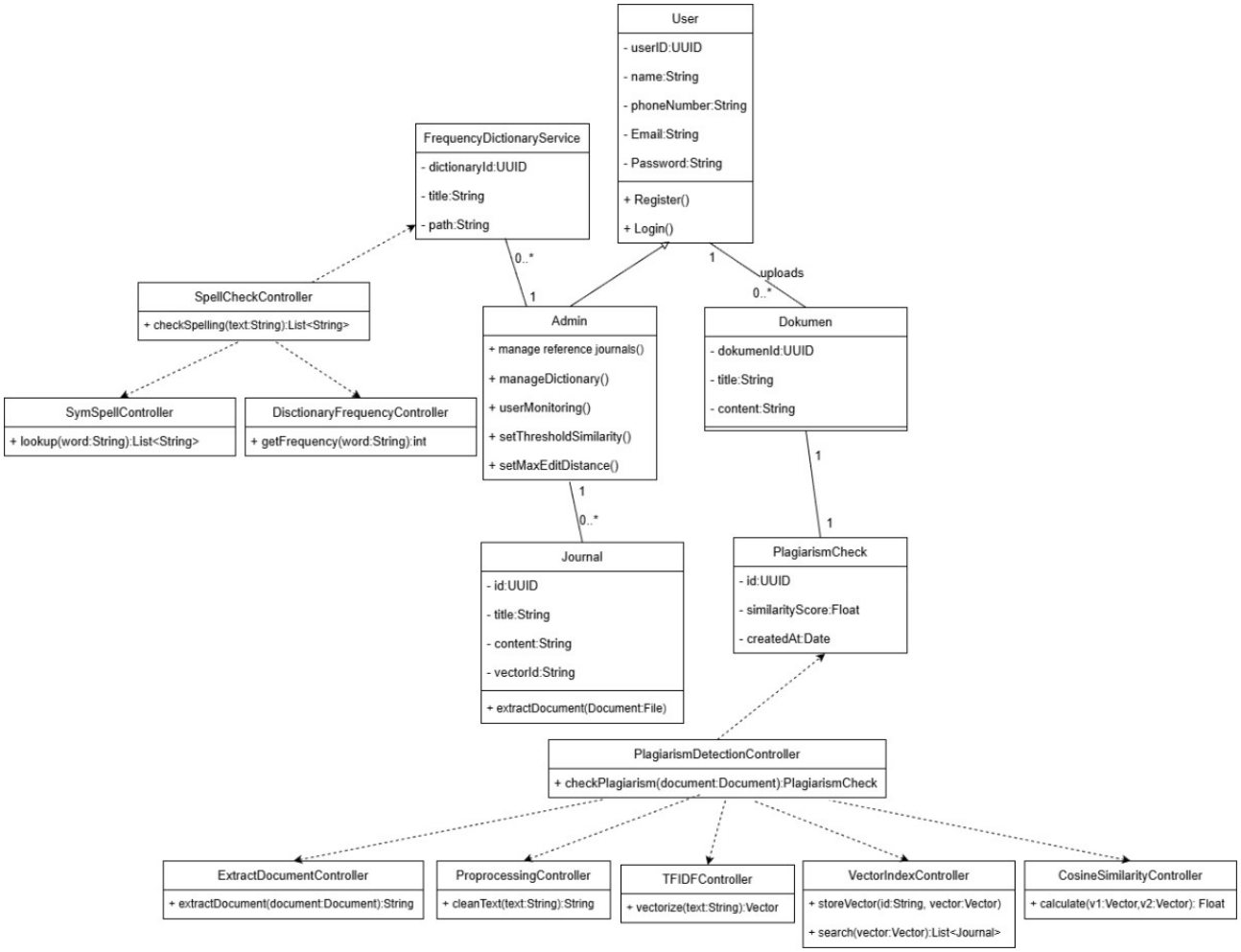
b. Admin memiliki hak akses penuh dalam pengelolaan sistem, yang meliputi pengelolaan *database* jurnal, pengelolaan kamus frekuensi, serta melakukan monitoring terhadap data pengguna aplikasi. Selain itu, Admin juga berwenang dalam pengaturan parameter sistem, seperti nilai *maximum delete distance* dan *threshold similarity* yang digunakan dalam proses analisis.

c. Pada pengecekan ejaan, pengguna memasukkan teks yang akan diperiksa, kemudian sistem memproses teks tersebut dan menampilkan hasil koreksi ejaan berdasarkan kamus yang telah disediakan.

d. Hubungan <<*include*>> pada *Use Case diagram* menunjukkan bahwa proses *input* teks atau unggah dokumen merupakan bagian yang tidak terpisahkan dari proses pengecekan plagiarisme dan pengecekan ejaan, sehingga harus dilakukan agar proses utama dapat dijalankan.

3.6.2 *Class* Diagram

Class diagram digunakan untuk menggambarkan struktur kelas yang terdapat dalam sistem, termasuk atribut dan metode yang dimiliki oleh setiap kelas serta hubungan antar kelas. Diagram ini membantu dalam memahami desain statis sistem



dan menjadi dasar dalam perancangan serta implementasi sistem secara terstruktur dan terorganisasi.

Gambar 3. 6 *Class diagram*

Class Diagram pada sistem ini menggambarkan struktur kelas dan hubungan antar kelas yang mendukung proses pengecekan plagiarisme dan pengecekan ejaan. Sistem melibatkan aktor *User* dan *Admin* yang berinteraksi dengan kelas entitas seperti *Dokumen*, *Jurnal*, dan *PlagiarismCheck*. Adapun penjelasan interaksi antar kelas dijelaskan pada Tabel 3.4 .

Tabel 3. 4 Relasi antara *class* diagram

No	Kelas Sumber	Kelas Tujuan	Jenis Relasi	Penjelasan
1	*User*	Dokumen	*Association*	Satu pengguna dapat mengunggah banyak dokumen, sedangkan setiap dokumen hanya dimiliki oleh satu pengguna.
2	Dokumen	*PlagiarismCheck*	*Association*	Setiap dokumen yang dianalisis menghasilkan satu hasil pengecekan plagiarisme berupa nilai kemiripan.
3	*PlagiarismDetectionController*	*PlagiarismCheck*	*Dependency*	*Controller* menghasilkan objek PlagiarismCheck sebagai hasil dari

Tabel 3. 4 Relasi antara *class* diagram

No	Kelas Sumber	Kelas Tujuan	Jenis Relasi	Penjelasan
				proses deteksi plagiarisme.
4	*PlagiarismDetectionController*	*ExtractDocumentController*	*Dependency*	Digunakan untuk mengekstraksi teks dari dokumen sebelum diproses lebih lanjut.
5	*PlagiarismDetectionController*	*PreprocessingController*	*Dependency*	Digunakan untuk melakukan pembersihan dan normalisasi teks hasil ekstraksi.
6	*PlagiarismDetectionController*	*TFIDFController*	*Dependency*	Digunakan untuk mengubah teks menjadi representasi vektor numerik menggunakan metode TF-IDF.
7	*PlagiarismDetectionController*	*VectorIndexController*	*Dependency*	Digunakan untuk menyimpan dan mencari vektor dokumen jurnal referensi yang relevan.
8	*PlagiarismDetectionController*	*CosineSimilarityController*	*Dependency*	Digunakan untuk menghitung tingkat kemiripan antar

Tabel 3. 4 Relasi antara *class* diagram

No	Kelas Sumber	Kelas Tujuan	Jenis Relasi	Penjelasan
				dokumen menggunakan metode *Cosine Similarity*.
9	*SpellCheck Controller*	*SymSpellController*	*Dependency*	Digunakan sebagai algoritma utama dalam mendeteksi kesalahan ejaan dan memberikan rekomendasi perbaikan kata.
10	*SpellCheck Controller*	*DictionaryFrequencyController*	*Dependency*	Digunakan untuk memperoleh nilai frekuensi kata dari kamus sebagai pendukung algoritma SymSpell.
11	Admin	*Journal*	*Association*	Admin bertanggung jawab dalam mengelola jurnal referensi yang digunakan sebagai dataset pembanding plagiarisme.
12	Admin	*FrequencyDi*	*Associati*	Admin mengelola

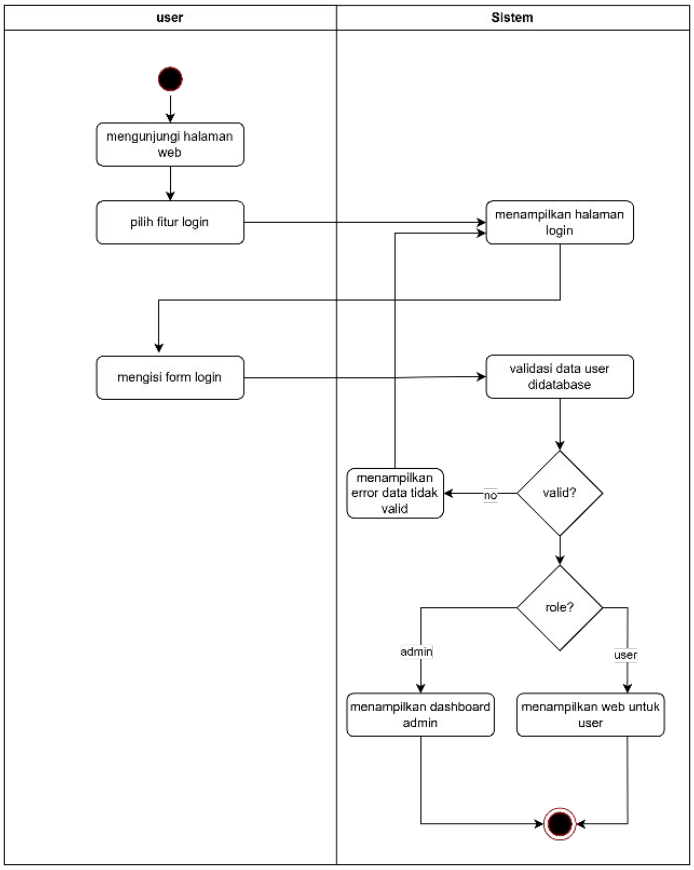
Tabel 3. 4 Relasi antara *class* diagram

No	Kelas Sumber	Kelas Tujuan	Jenis Relasi	Penjelasan
		ctionaryService	*on*	kamus frekuensi yang digunakan dalam proses pengecekan ejaan.
13	Admin	*User*	*Generalization*	Admin merupakan turunan dari *User* yang memiliki hak akses dan fungsi tambahan dalam sistem.

3.6.3 *Activity Diagram*

Activity Diagram digunakan untuk memodelkan alur kerja serta rangkaian aktivitas yang terjadi dalam suatu sistem. Diagram ini menggambarkan urutan proses, pengambilan keputusan, dan kemungkinan alur yang dapat terjadi selama suatu aktivitas berlangsung, sehingga memudahkan pemahaman terhadap proses bisnis atau logika kerja sistem secara menyeluruh.

a. *Activity Diagram Login*

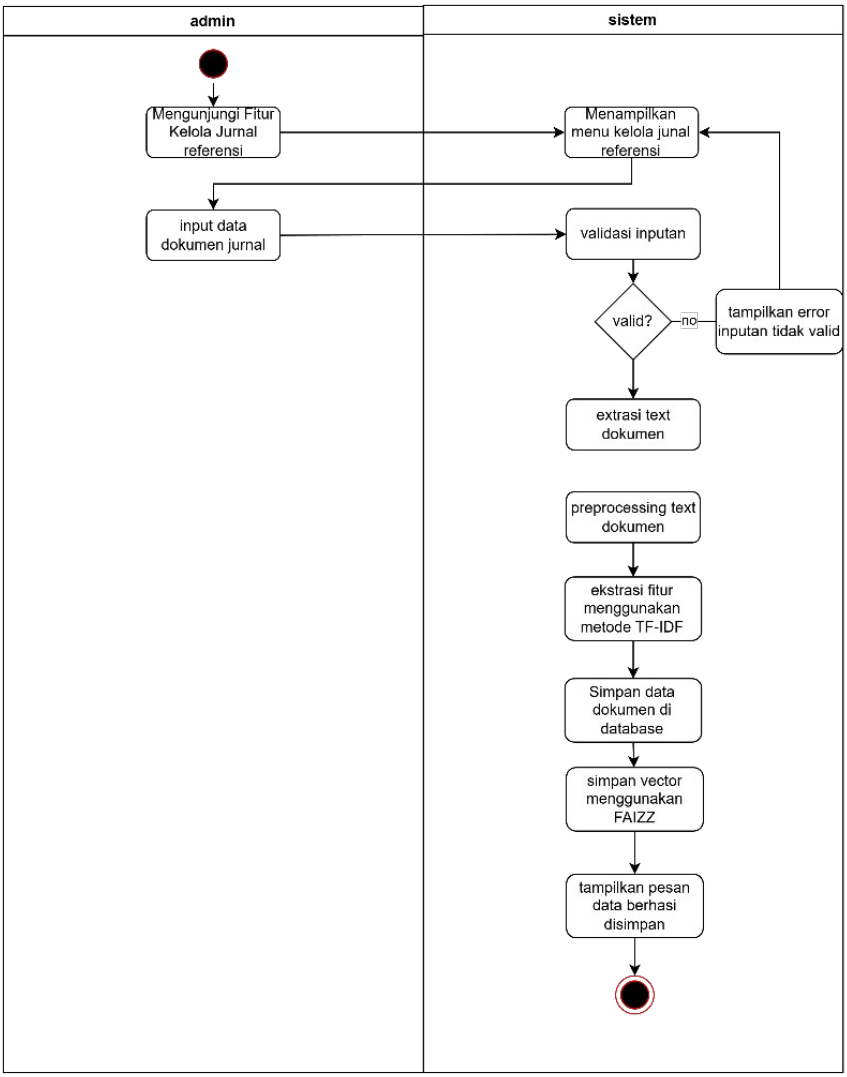


Gambar 3. 7 *Activity Diagram login*

Proses autentikasi dan inisialisasi sistem diawali ketika pengguna mengakses halaman *web* dan memilih fitur *login*, kemudian sistem menampilkan antarmuka halaman *login* yang menyediakan formulir pengisian kredensial. Data yang dimasukkan oleh pengguna selanjutnya divalidasi oleh sistem dengan mencocokkannya terhadap data yang tersimpan di dalam basis data. Pada tahap ini, sistem melakukan pengambilan keputusan, di mana apabila data tidak valid maka sistem menampilkan pesan kesalahan dan mengarahkan pengguna kembali ke halaman *login* untuk melakukan pengisian ulang. Sebaliknya, jika data dinyatakan valid, sistem

melanjutkan proses dengan melakukan pengecekan peran pengguna, sehingga sistem dapat menampilkan antarmuka yang sesuai, yaitu dashboard khusus bagi Admin atau halaman utama bagi pengguna umum. Proses ini berakhir setelah sistem berhasil memberikan akses sesuai dengan hak dan peran masing-masing pengguna.

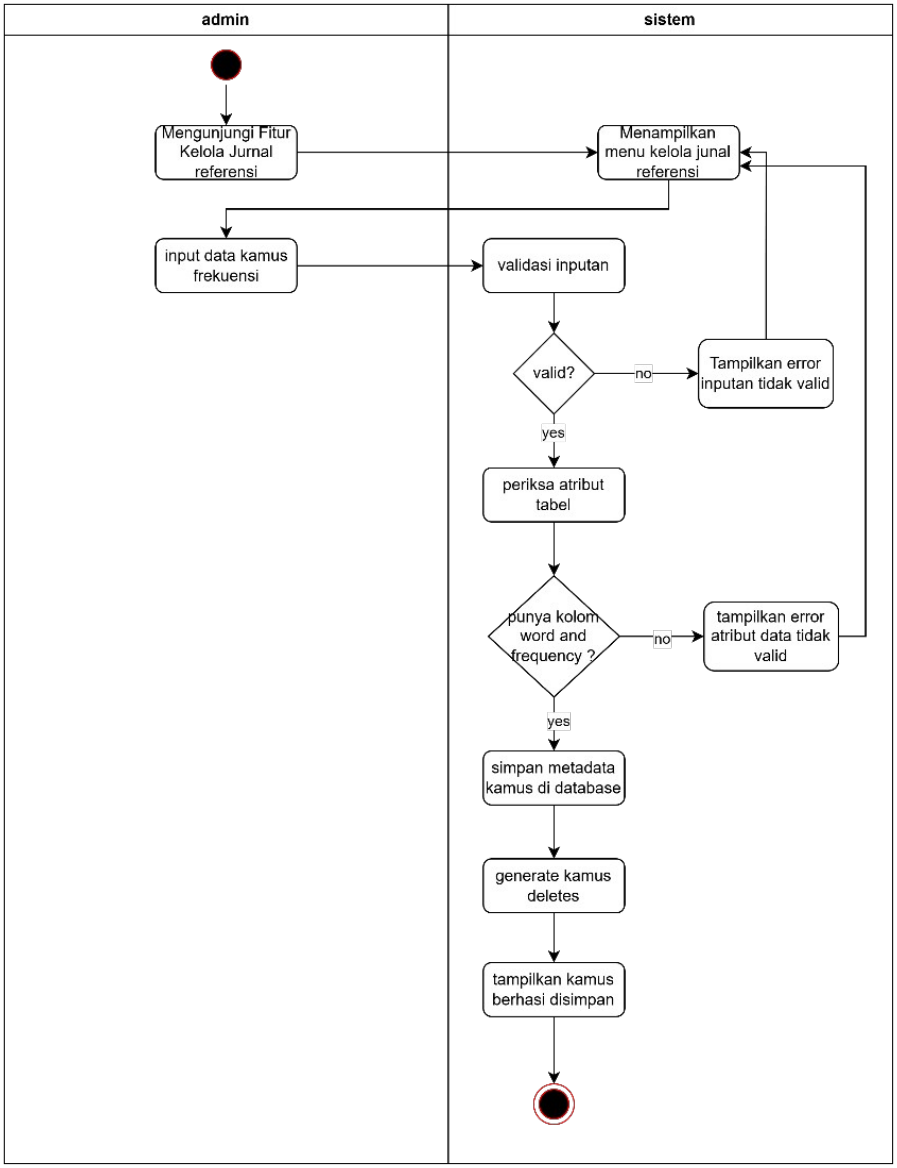
b. *Activity Diagram Input* Data Jurnal Referensi (Level Admin)



Gambar 3. 8 *Activity Diagram input* data jurnal referensi

Activity Diagram ini menggambarkan alur proses *input* data jurnal renferensi oleh Admin pada sistem. Proses dimulai ketika Admin mengakses fitur Kelola Jurnal Referensi, kemudian sistem menampilkan menu pengelolaan jurnal. Admin selanjutnya melakukan *input* data dokumen jurnal, yang kemudian divalidasi oleh sistem. Apabila *input* tidak valid, sistem menampilkan pesan kesalahan dan meminta Admin untuk mengulangi proses *input*. Jika data valid, sistem melanjutkan dengan ekstraksi teks, *preprocessing*, dan pembobotan fitur menggunakan metode TF-IDF untuk membentuk representasi vektor. Selanjutnya, sistem menyimpan data dokumen ke dalam *database* serta menyimpan vektor menggunakan FAISS(*Facebook AI Similarity Search*). Proses diakhiri dengan sistem menampilkan notifikasi bahwa data berhasil disimpan.

c. *Activity Diagram Input* Kamus Frekuensi (Level Admin)

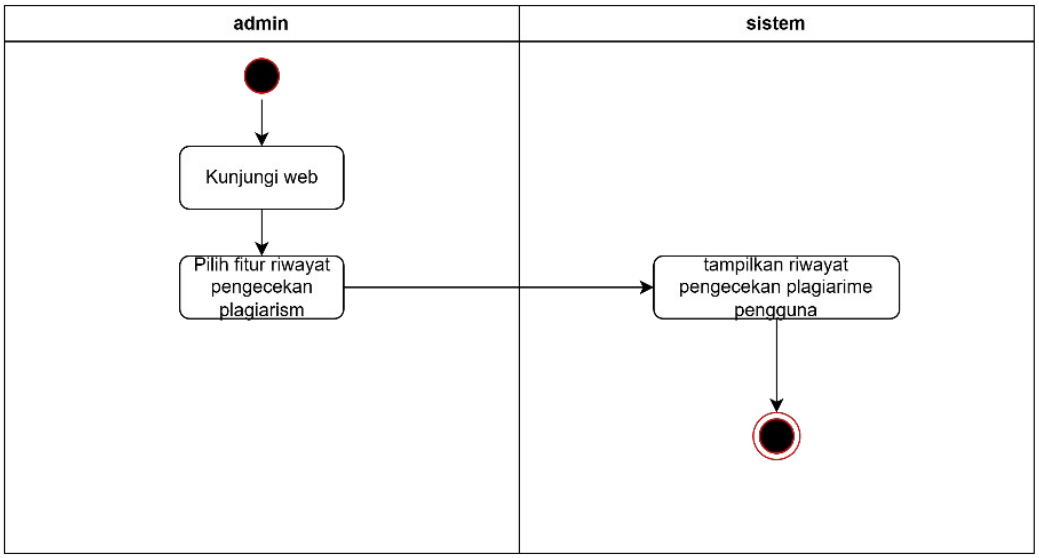


Gambar 3. 9 *Activity Diagram input* kamus frekuensi

Activity Diagram ini menggambarkan alur proses *input* kamus frekuensi oleh Admin pada sistem. Proses dimulai ketika Admin mengunjungi fitur kelola jurnal referensi, kemudian sistem menampilkan menu pengelolaan jurnal referensi yang tersedia. Selanjutnya, Admin melakukan *input* data kamus frekuensi yang akan diproses oleh sistem. Sistem melakukan validasi terhadap data yang di*input*, apabila data tidak valid

maka sistem akan menampilkan pesan kesalahan terkait *input*an yang tidak sesuai. Jika data dinyatakan valid, sistem akan melanjutkan proses dengan memeriksa atribut tabel untuk memastikan bahwa data memiliki kolom *word* dan *frequency* yang sesuai dengan ketentuan. Apabila atribut data tidak valid, sistem akan menampilkan pesan kesalahan, sedangkan jika atribut telah sesuai, sistem akan menyimpan metadata kamus ke dalam *database*. Setelah itu, sistem melakukan proses pembentukan kamus *deletes* sebagai bagian dari penerapan algoritma SymSpell. Proses diakhiri dengan sistem menampilkan notifikasi bahwa kamus frekuensi berhasil disimpan.

d. *Activity Diagram* Monitoring Riwayat Penggunaan Aplikasi (Level Admin)

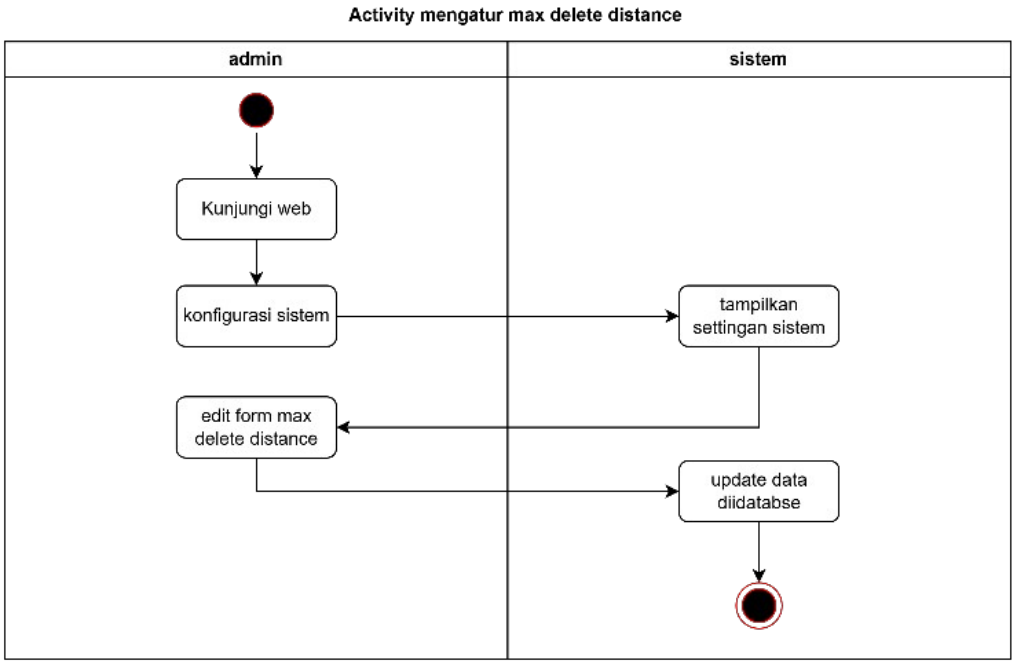


Gambar 3. 10 *Activity Diagram* riwayat pengguna aplikasi

Activity Diagram ini menggambarkan alur proses monitoring data pengguna yang dilakukan oleh Admin pada sistem. Proses diawali ketika Admin mengakses antarmuka berbasis *web* dan berhasil masuk ke dalam sistem. Selanjutnya,

Admin melakukan navigasi dengan memilih fitur Data Pengguna sebagai bentuk permintaan akses terhadap modul Riwayat penggunaan aplikasi. Menanggapi permintaan tersebut, sistem melakukan proses pengambilan data dari basis data dan kemudian menyajikannya dalam bentuk daftar informasi riwayat pengecekan plagiarisme pada antarmuka aplikasi. Alur aktivitas berakhir setelah sistem berhasil menampilkan data pengguna yang diminta oleh Admin.

e. *Activity Diagram* Mengatur *Max Delete Distance* Algoritma SymSpell (Level Admin)

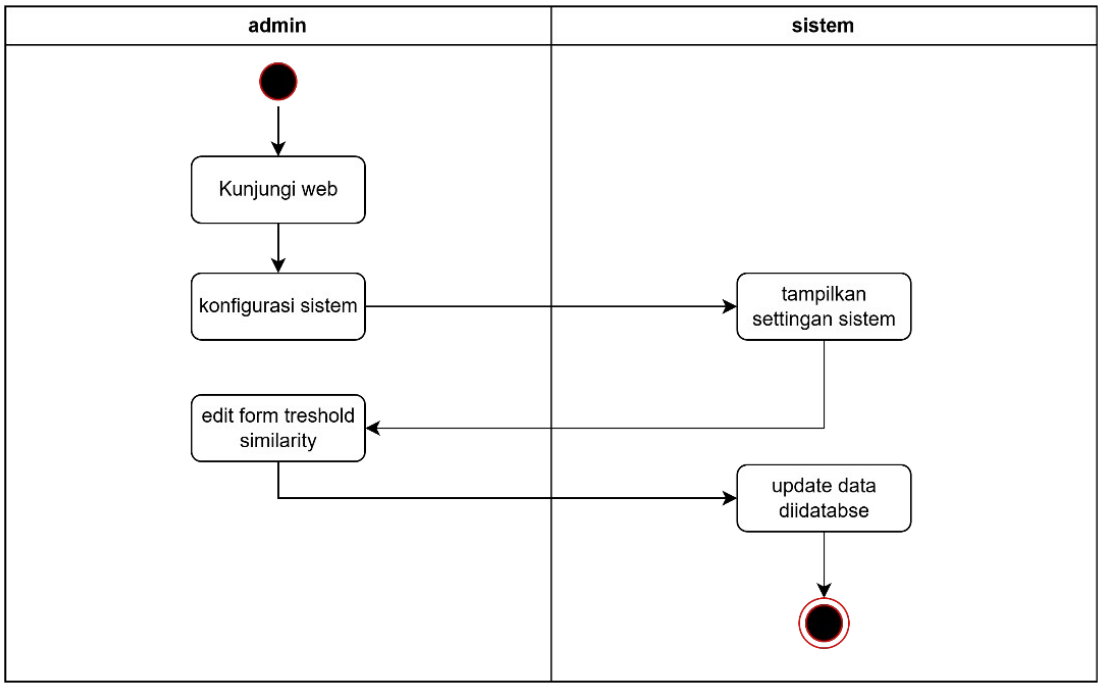


Gambar 3. 11 *Activity Diagram* mengatur *max delete distance*

Activity Diagram ini menggambarkan alur proses pengaturan konfigurasi sistem yang dilakukan oleh Admin. Proses diawali ketika Admin mengakses antarmuka aplikasi berbasis *web* dan menavigasi sistem dengan memilih fitur Konfigurasi Sistem. Sebagai respons, sistem menampilkan

antarmuka yang berisi daftar pengaturan sistem yang dapat dikelola. Selanjutnya, Admin melakukan penyuntingan nilai parameter *Max Delete Distance* melalui formulir konfigurasi yang tersedia. Setelah nilai baru dimasukkan, sistem memproses pembaruan dengan menyimpan perubahan tersebut ke dalam basis data, sehingga konfigurasi baru tersimpan secara persisten dan dapat langsung diterapkan oleh sistem. Proses ini menandai berakhirnya alur aktivitas pengaturan konfigurasi sistem.

f. *Activity Diagram* Mengatur *Treshold Similarity* Algoritma *Cosine Similarity* (Level Admin)

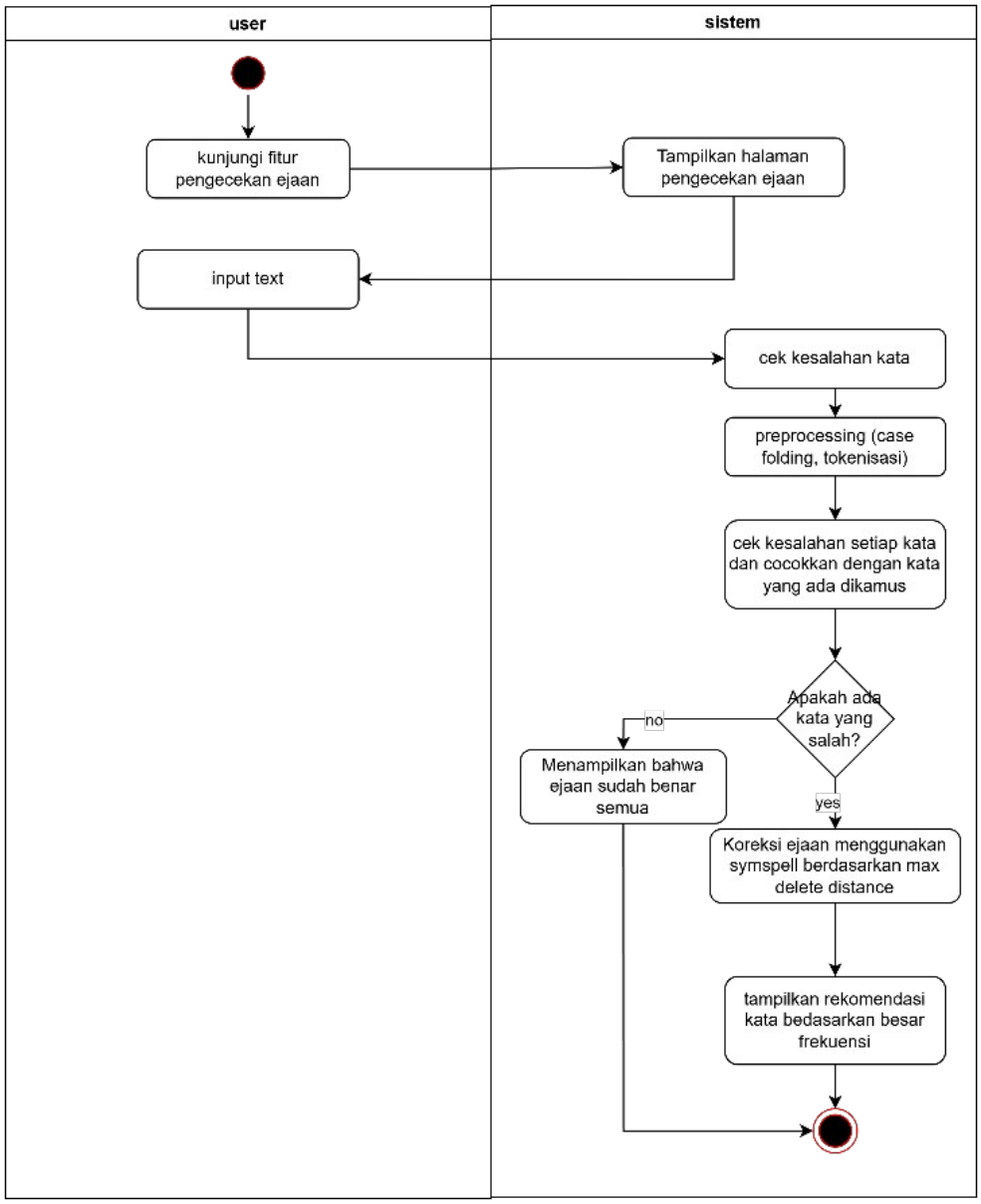


Gambar 3. 12 *Activity Diagram* mengatur *threshold similarity*

Activity Diagram ini menggambarkan alur proses pengelolaan parameter *Threshold Similarity* yang dilakukan oleh Admin pada sistem. Proses diawali ketika Admin mengakses antarmuka aplikasi berbasis *web* dan menavigasi sistem dengan

memilih menu Konfigurasi Sistem. Sebagai respons atas permintaan tersebut, sistem mengambil data konfigurasi yang sedang berlaku dan menampilkannya pada antarmuka pengaturan agar dapat ditinjau atau disesuaikan. Selanjutnya, Admin melakukan penyuntingan nilai parameter *Threshold Similarity* melalui formulir yang tersedia. Setelah nilai baru dimasukkan, sistem memproses pembaruan dengan menyimpan perubahan konfigurasi tersebut ke dalam basis data secara permanen. Alur aktivitas berakhir setelah sistem berhasil memperbarui data konfigurasi dan menerapkan nilai baru ke dalam proses sistem.

g. *Activity Diagram* Pengecekan Ejaan (Level Pengguna)

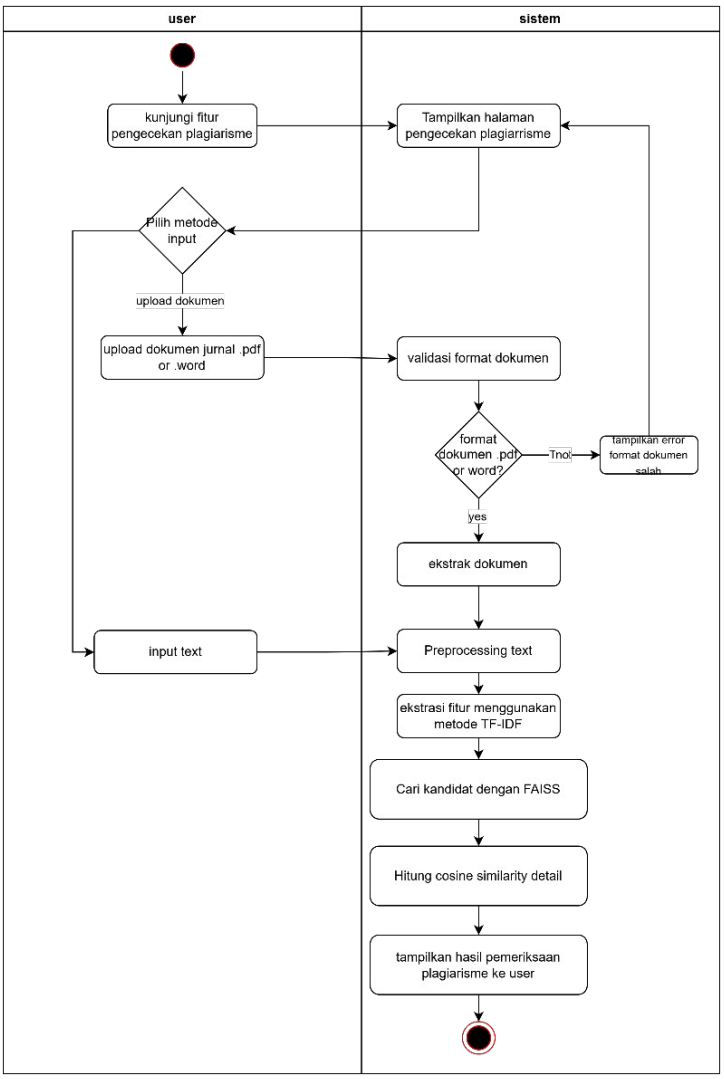


Gambar 3. 13 *Activity Diagram* pengecekan ejaan

Activity Diagram ini menggambarkan alur proses pengecekan ejaan dan koreksi kata yang dilakukan oleh sistem. Proses diawali ketika pengguna mengakses fitur pengecekan ejaan, kemudian sistem menampilkan antarmuka halaman pengecekan dan menerima *input* teks dari pengguna. Teks yang dimasukkan selanjutnya diproses melalui tahap *preprocessing*, yang meliputi *case folding* untuk menyeragamkan huruf serta

tokenisasi untuk memisahkan teks menjadi unit kata. Setelah itu, sistem melakukan verifikasi leksikal dengan mencocokkan setiap token kata terhadap basis data kamus. Apabila seluruh kata dinyatakan valid, sistem menampilkan notifikasi bahwa ejaan sudah benar. Sebaliknya, jika ditemukan kata yang tidak dikenali atau salah eja, sistem melanjutkan proses koreksi menggunakan algoritma SymSpell dengan memanfaatkan parameter *Max Delete Distance* untuk menghasilkan kandidat kata pengganti. Hasil koreksi kemudian ditampilkan kepada pengguna dalam bentuk rekomendasi kata yang diurutkan berdasarkan frekuensi kemunculan, sehingga pengguna memperoleh saran ejaan yang paling relevan.

h. *Activity Diagram* Pengecekan Plagiarisme Dokumen (Level Pengguna)



Gambar 3. 14 *Activity Diagram* pengecekan plagiarism dokumen

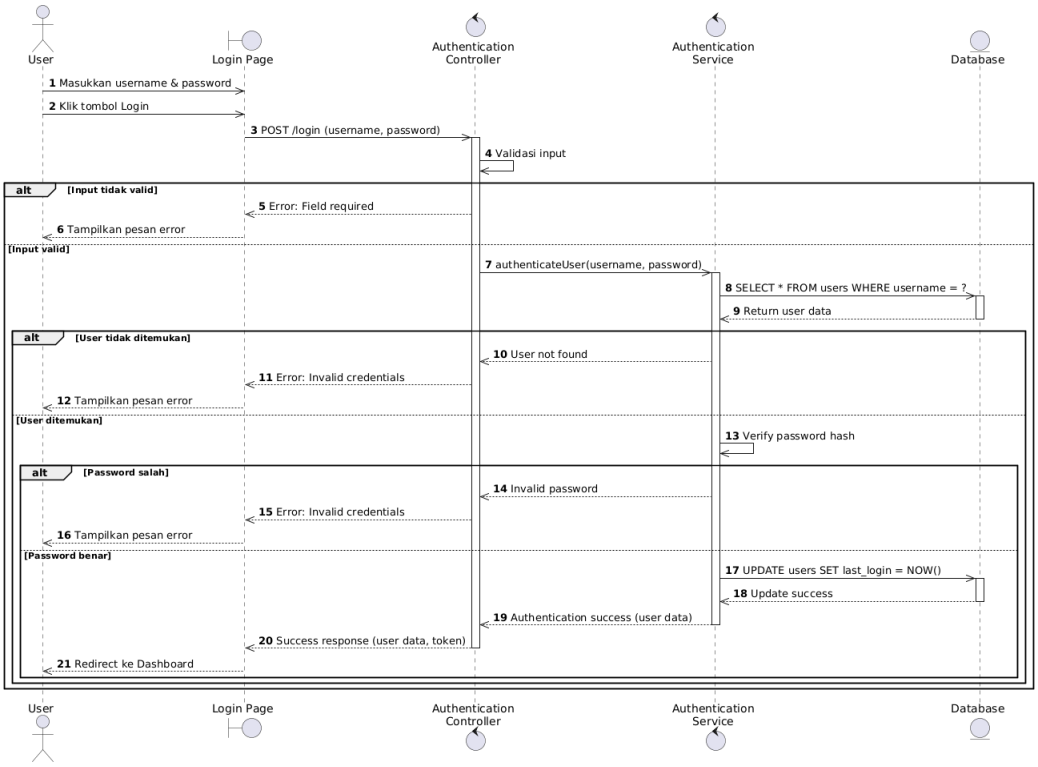
Activity Diagram ini menggambarkan alur proses pengecekan plagiarisme yang dilakukan oleh pengguna pada sistem. Proses diawali ketika pengguna mengakses fitur pengecekan plagiarisme melalui antarmuka sistem. Setelah sistem menampilkan halaman pengecekan, pengguna dihadapkan pada pilihan metode *input* data, yaitu mengunggah dokumen atau memasukkan teks secara langsung. Pada opsi unggah dokumen, sistem membatasi format file yang

diperbolehkan, yaitu berkas dengan ekstensi *.pdf* atau *.word*. Sistem selanjutnya melakukan validasi format dokumen, di mana apabila format tidak sesuai, sistem menampilkan pesan kesalahan dan mengarahkan pengguna kembali ke halaman awal. Jika format dokumen valid, atau apabila pengguna memilih metode *input* teks langsung, dokumen akan di ekstrasi menjadi teks. Tahap selanjutnya, data akan diproses melalui tahap *preprocessing* untuk membersihkan dan menormalisasi teks. Tahap berikutnya adalah ekstraksi fitur menggunakan metode TF-IDF untuk mengubah teks menjadi representasi vektor numerik, yang kemudian diproses menggunakan FAISS untuk memperoleh kandidat dokumen pembanding secara efisien. Selanjutnya, sistem menghitung tingkat kemiripan antar dokumen menggunakan algoritma *Cosine Similarity*. Proses ini diakhiri dengan penyajian hasil analisis plagiarisme kepada pengguna melalui antarmuka sistem.

3.6.4 *Sequence Diagram*

Sequence Diagram digunakan untuk memodelkan interaksi antar aktor dan objek dalam suatu sistem berdasarkan urutan waktu. Diagram ini menggambarkan bagaimana pesan atau komunikasi dikirim dan diterima antar komponen sistem secara berurutan untuk menjalankan suatu proses. Dengan menampilkan aliran interaksi, pemanggilan fungsi, serta respons yang terjadi, *Sequence Diagram* membantu memahami logika kerja sistem, tanggung jawab setiap komponen, dan urutan eksekusi proses secara lebih detail.

a. *Sequence Diagram Login*

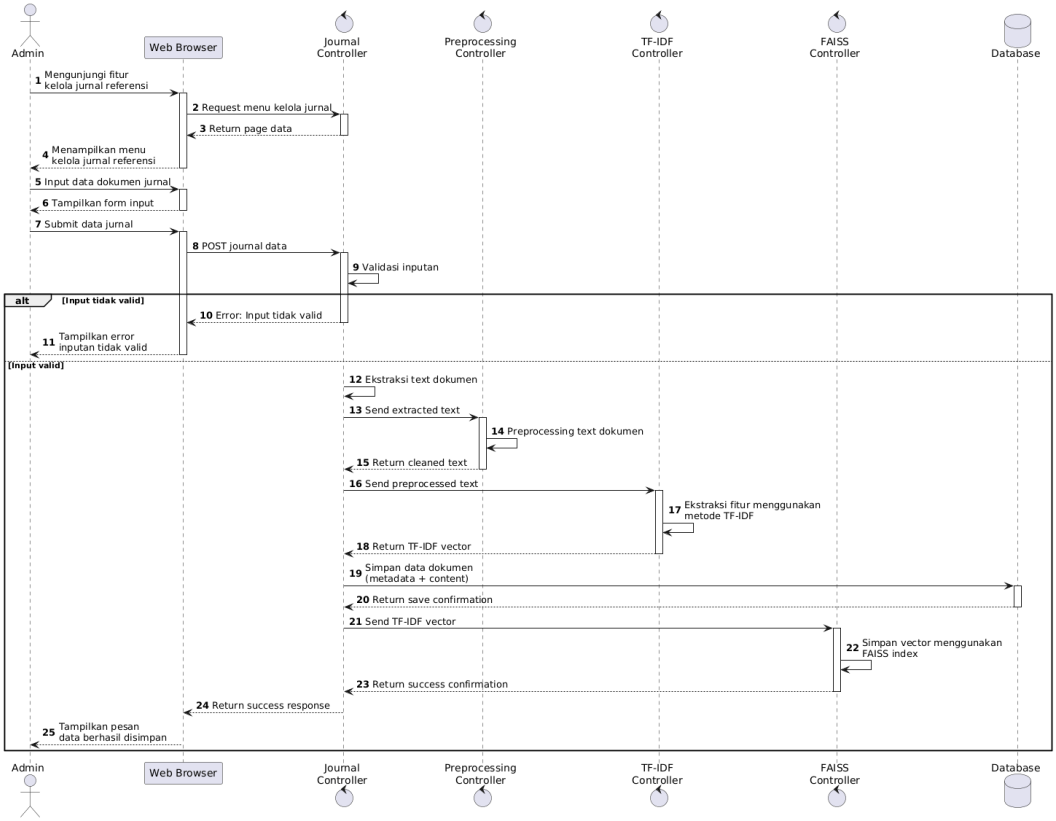


Gambar 3. 15 *Sequence Diagram login*

Sequence Diagram ini menggambarkan alur interaksi sistem pada proses *login* pengguna yang melibatkan aktor *User*, *Login Page*, *Authentication Controller*, *Authentication Service*, dan *Database*. Proses dimulai ketika pengguna memasukkan kredensial pada halaman *login*, yang kemudian dikirim ke *Authentication Controller* untuk dilakukan validasi awal. Jika data input tidak valid, sistem langsung mengembalikan pesan kesalahan ke antarmuka pengguna. Apabila valid, kontroler meneruskan proses autentikasi ke *Authentication Service* untuk melakukan verifikasi keberadaan pengguna pada basis data. Jika pengguna tidak ditemukan atau kata sandi tidak sesuai, sistem mengembalikan respons "*invalid credentials*" demi menjaga keamanan informasi. Sebaliknya, jika verifikasi berhasil, sistem

memperbarui data *login* terakhir pada basis data dan mengembalikan respons sukses berupa data pengguna serta token autentikasi, yang diakhiri dengan pengalihan pengguna ke halaman *dashboard* utama.

b. *Sequence Diagram Input* Jurnal Referensi (Level Admin)

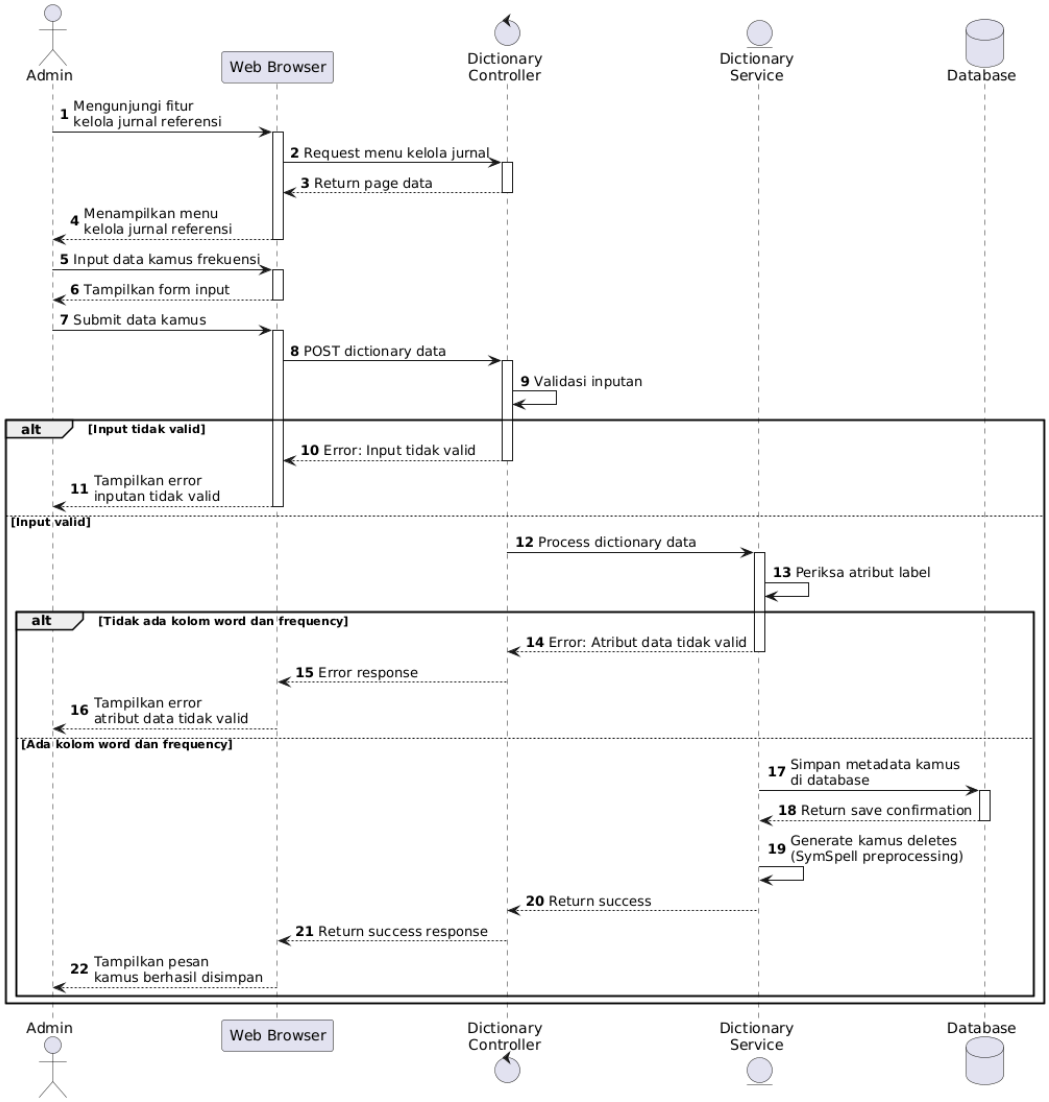


Gambar 3. 16 *Sequence Diagram input* jurnal referensi

Sequence Diagram input jurnal referensi menggambarkan alur interaksi sistem dalam proses pengelolaan dokumen jurnal oleh Admin. Proses dimulai ketika Admin mengirimkan dokumen jurnal melalui antarmuka *web* ke *Journal Controller*. Setelah validasi awal, sistem mengekstrak dan memproses teks melalui *Preprocessing Controller* untuk menghasilkan teks yang telah dibersihkan. Selanjutnya, teks diproses oleh TF-IDF *Controller* untuk membentuk representasi vektor dokumen. Pada tahap

akhir, sistem menerapkan mekanisme penyimpanan ganda, yaitu menyimpan metadata jurnal ke dalam basis data dan menyimpan vektor dokumen ke dalam FAISS untuk keperluan pencarian kemiripan. Proses berakhir setelah sistem memberikan notifikasi bahwa data jurnal berhasil disimpan.

c. *Sequence Diagram* Input Kamu Frekuensi (Level Admin)

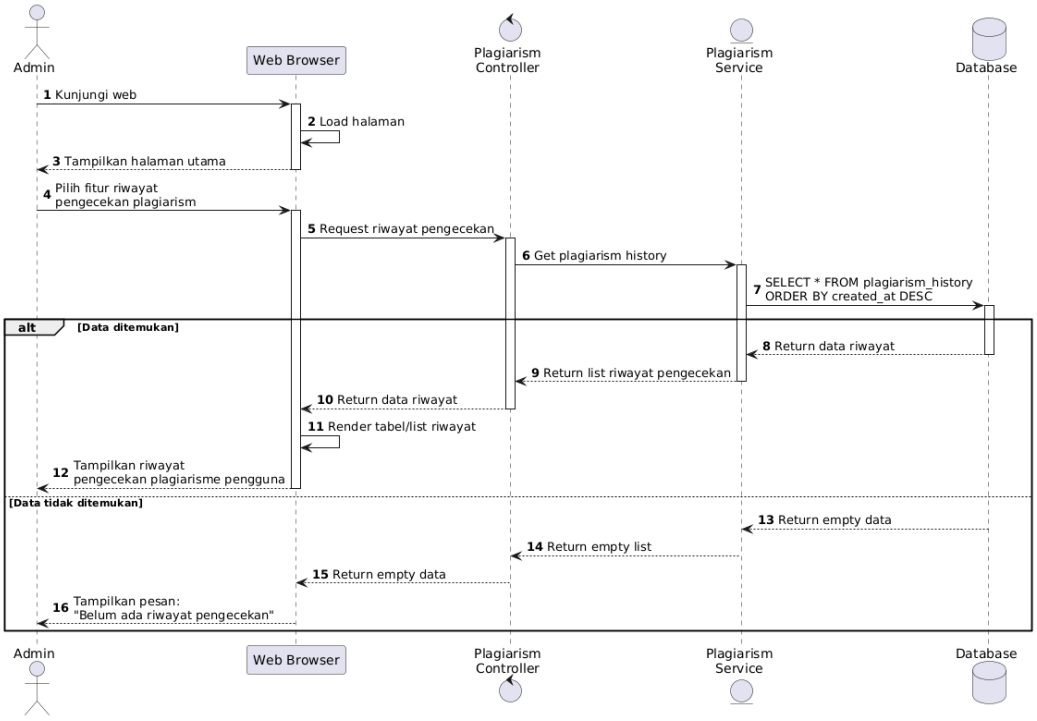


Gambar 3. 17 *Sequence Diagram input* kamu frekuensi

Sequence Diagram input data kamus menggambarkan alur interaksi sistem dalam proses pengelolaan kamus frekuensi oleh

Admin. Proses dimulai ketika Admin mengirimkan data kamus melalui antarmuka *web* ke *Dictionary Controller*, yang kemudian melakukan validasi awal terhadap format input. Apabila data tidak valid, sistem mengembalikan pesan kesalahan dan proses dihentikan. Jika valid, pemrosesan dilanjutkan ke *Dictionary Service* untuk memverifikasi kelengkapan atribut penting, yaitu *word* dan *frequency*. Apabila atribut tidak memenuhi ketentuan, sistem mengembalikan pesan galat terkait ketidaksesuaian struktur data. Sebaliknya, jika data dinyatakan valid, sistem menyimpan metadata kamus ke dalam basis data dan melakukan proses pembentukan kamus *deletes* sebagai tahap praproses algoritma SymSpell. Proses berakhir ketika sistem menampilkan notifikasi bahwa data kamus berhasil disimpan.

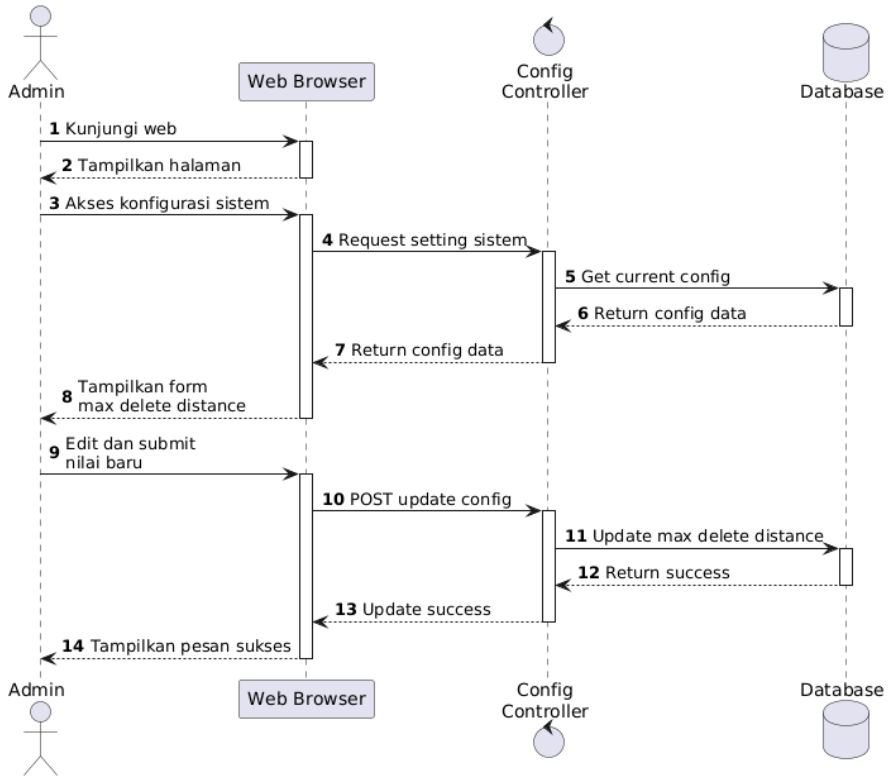
d. *Sequence Diagram* Monitoring Riwayat Penggunaan Aplikasi (Level Admin)



Gambar 3. 18 *Sequence Diagram* Monitoring Riwayat Penggunaan Aplikasi

Sequence Diagram monitoring riwayat plagiarisme merepresentasikan alur pengambilan dan penyajian data histori pengecekan oleh Admin. Proses diawali ketika Admin mengakses fitur riwayat pengecekan melalui antarmuka *web*, yang memicu permintaan data ke *Plagiarism Controller*. Kontroler kemudian mendelegasikan pengambilan data ke *Plagiarism Service* untuk mengeksekusi kueri basis data secara terurut menurun berdasarkan waktu pembuatan, sehingga data terbaru ditampilkan terlebih dahulu. Hasil kueri selanjutnya dikembalikan ke lapisan presentasi. Apabila data riwayat tersedia, sistem menampilkan daftar hasil pengecekan kepada Admin. Sebaliknya, jika tidak ditemukan data, sistem menampilkan pesan informasi bahwa riwayat pengecekan belum tersedia.

e. *Sequence Diagram* Mengatur *Max Delete Distance* (Level Admin)

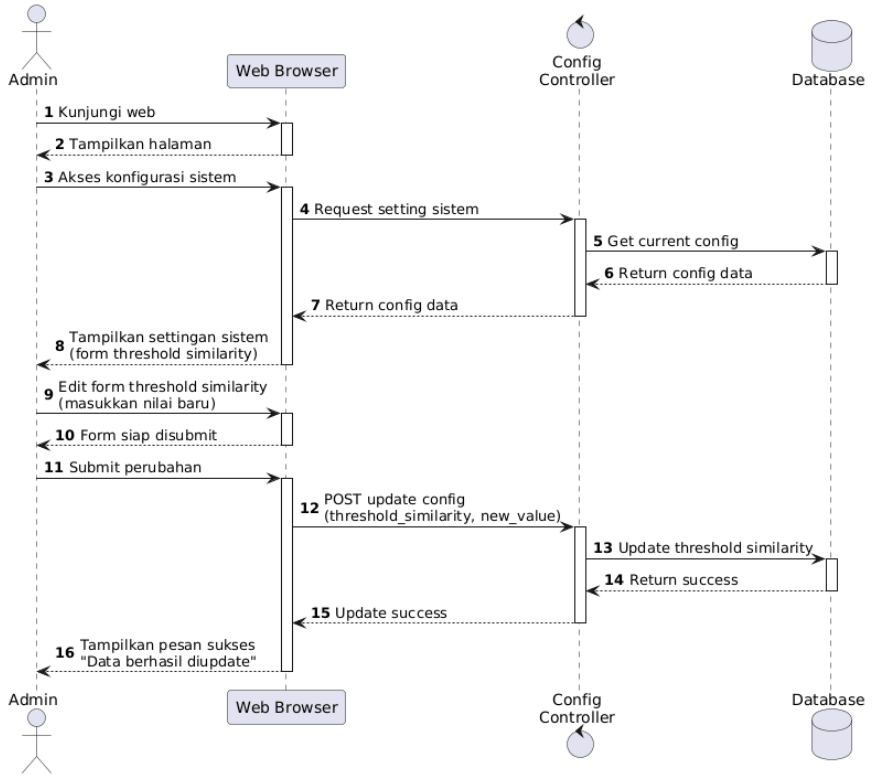


Gambar 3. 19 *Sequence Diagram* mengatur *max delete distance*

Sequence Diagram Max Delete Distance merepresentasikan alur pengelolaan parameter konfigurasi algoritma yang dilakukan oleh Administrator. Proses diawali dengan akses Administrator ke menu konfigurasi melalui antarmuka *web*, yang memicu permintaan pengambilan data konfigurasi ke *Config Controller*. Komponen ini bertugas menginisiasi proses *retrieval* ke basis data untuk memperoleh nilai parameter yang sedang aktif, termasuk *max delete distance*, yang selanjutnya ditampilkan pada formulir konfigurasi sebagai bahan evaluasi. Pada tahap berikutnya, Administrator melakukan penyesuaian nilai parameter dan mengirimkan perubahan tersebut ke sistem. *Config Controller* kemudian mengeksekusi proses pemutakhiran data pada basis data melalui mekanisme persistensi. Setelah pembaruan berhasil dikonfirmasi, sistem mengirimkan umpan

balik keberhasilan ke antarmuka pengguna, menandakan bahwa parameter *max delete distance* telah tersimpan secara permanen.

f. *Sequence Diagram* Mengatur *Treshold Similarity* (Level Admin)

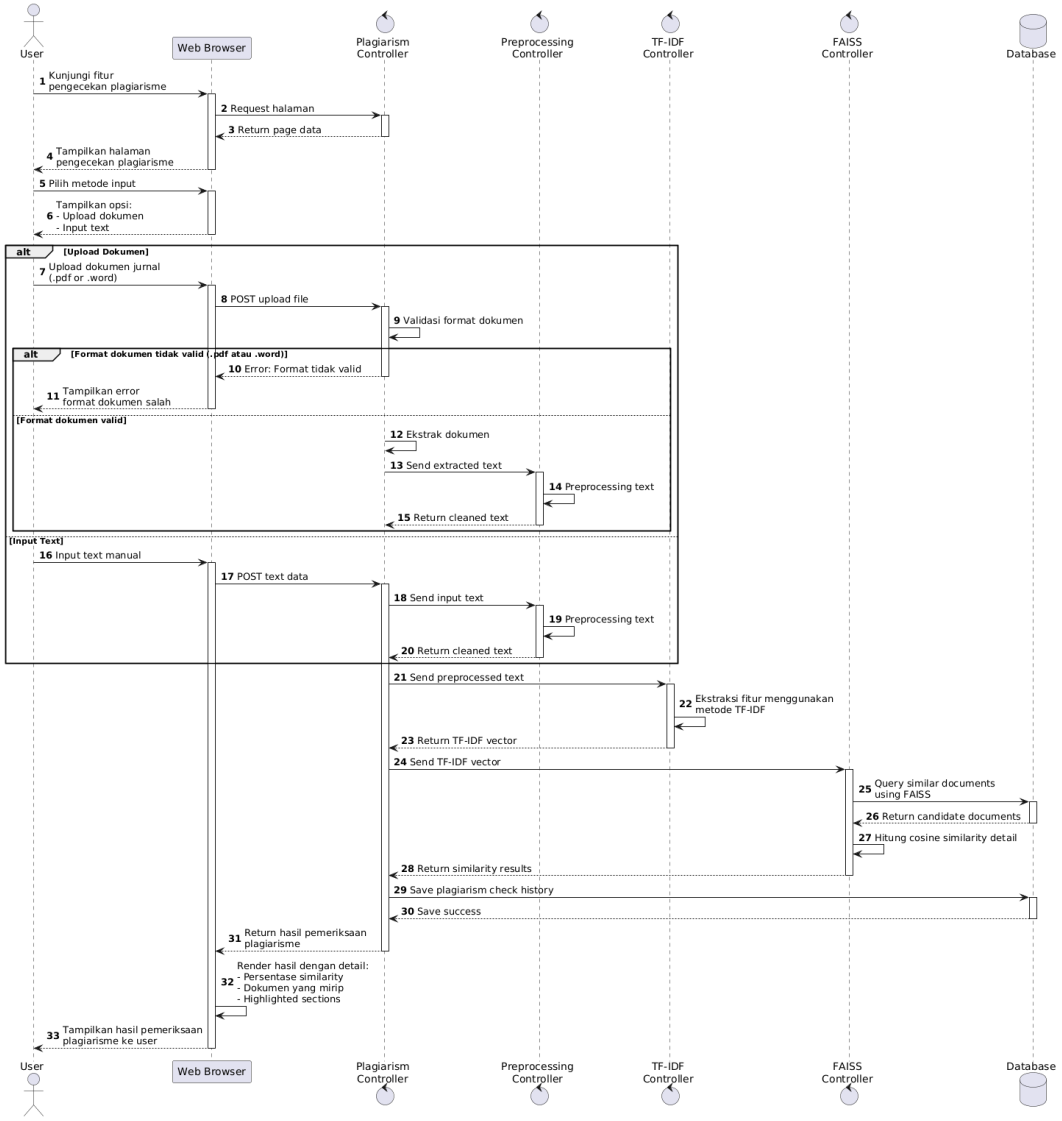


Gambar 3. 20 *Sequence Diagram* mengatur *treshold similarity*

Sequence Diagram Konfigurasi Threshold Similarity menunjukkan proses pengelolaan parameter ambang batas kemiripan pada sistem deteksi plagiarisme. Proses dimulai ketika Administrator mengakses menu konfigurasi, sehingga *Config Controller* mengambil nilai *threshold similarity* yang tersimpan di basis data dan menampilkannya pada antarmuka. Selanjutnya, Administrator memasukkan nilai baru dan mengirimkannya ke sistem, yang kemudian diproses oleh *Config*

Controller untuk diperbarui dan disimpan secara permanen di basis data. Setelah pembaruan berhasil, sistem mengirimkan umpan balik keberhasilan ke antarmuka, menandakan bahwa nilai ambang batas kemiripan telah diperbarui dan berlaku dalam proses analisis selanjutnya.

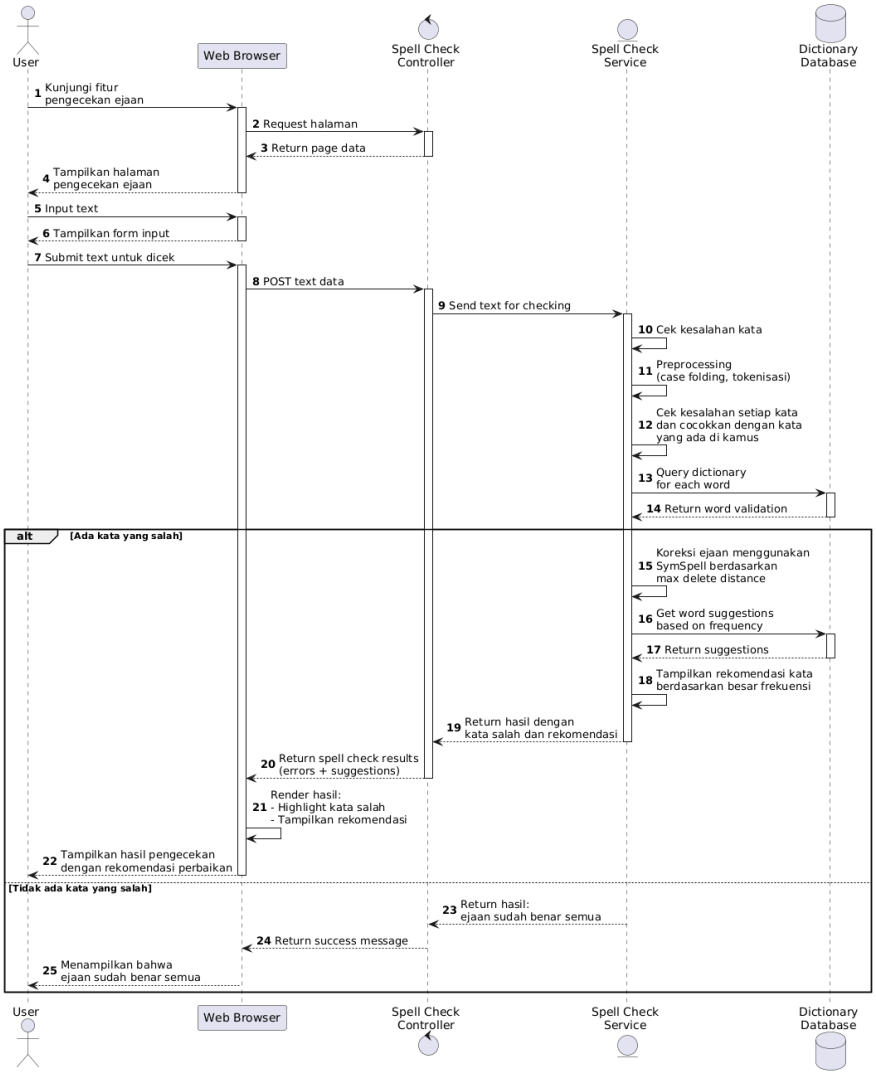
g. *Sequence Diagram* Pengecekan Plagiarisme (Level Pengguna)



Gambar 3. 21 *Sequence Diagram* pengecekan plagiarisme

Sequence Diagram Proses *Pengecekan Plagiarisme* merepresentasikan alur deteksi kemiripan dokumen yang diawali oleh pengguna melalui antarmuka *web* dengan dua metode input, yaitu unggah dokumen atau teks manual. Dokumen yang diunggah divalidasi formatnya dan diekstraksi menjadi teks, sedangkan teks manual langsung diproses. Seluruh input kemudian melalui tahap pra-pemrosesan untuk normalisasi dan pembersihan data, sebelum ditransformasikan ke dalam representasi vektor menggunakan metode TF-IDF. Vektor tersebut dianalisis menggunakan FAISS untuk menemukan dokumen referensi yang relevan, lalu dihitung tingkat kemiripannya secara presisi menggunakan *Cosine Similarity*. Hasil analisis disimpan sebagai riwayat pengecekan dalam basis data dan ditampilkan kepada pengguna dalam bentuk persentase kemiripan, daftar sumber terkait, serta visualisasi segmen teks yang terindikasi plagiat.

h. *Sequence Diagram* Pengecekan Ejaan (Level Pengguna)



Gambar 3. 22 *Sequence Diagram* pengecekan ejaan

Sequence Diagram Proses Pengecekan Ejaan (Spell Check) menggambarkan alur validasi ortografi teks yang diinisiasi oleh pengguna melalui antarmuka *web*. Teks input dikirim ke *Spell Check Controller* dan diteruskan ke *Spell Check Service* untuk dilakukan pra-pemrosesan berupa *case folding* dan tokenisasi. Setiap token kemudian divalidasi terhadap basis data kamus. Apabila ditemukan kata yang tidak terdaftar, sistem mengeksekusi algoritma SymSpell dengan mempertimbangkan parameter *max delete distance* untuk menghasilkan dan

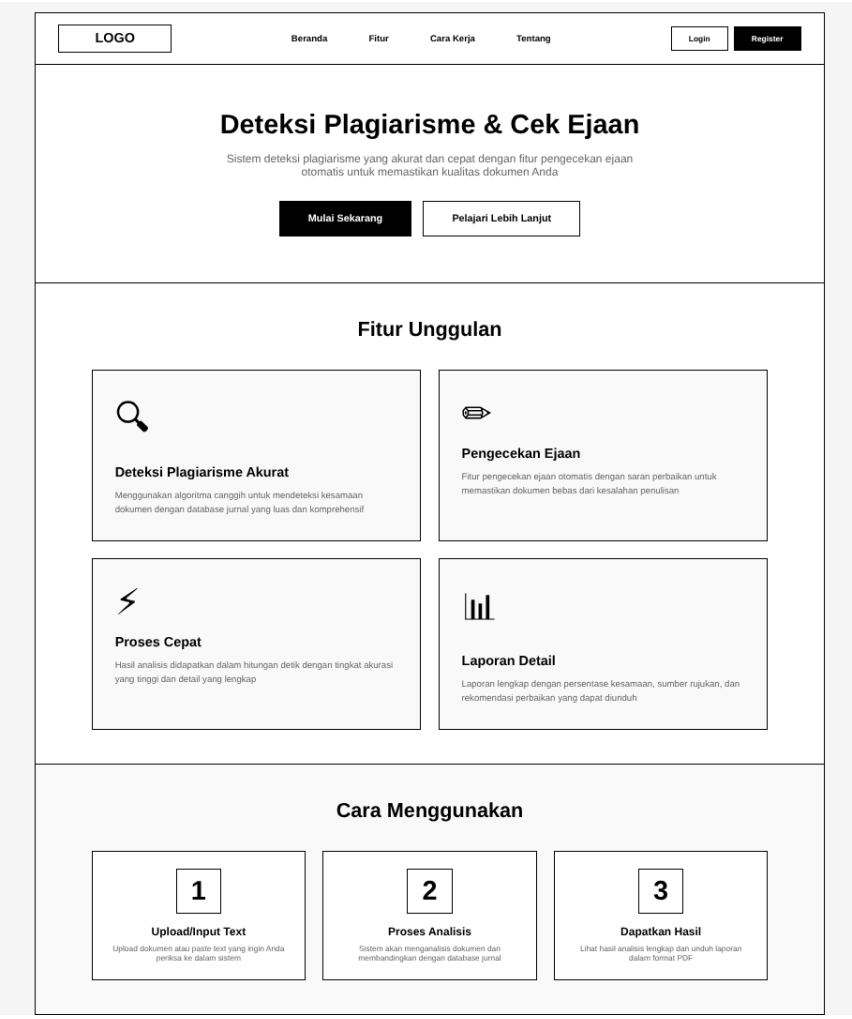
memberi peringkat kandidat koreksi berdasarkan frekuensi kemunculan. Hasil analisis, berupa daftar kata salah dan rekomendasinya, dikembalikan ke antarmuka untuk divisualisasikan. Sebaliknya, jika seluruh token valid, sistem langsung mengembalikan status bahwa ejaan teks telah sesuai.

3.7. Analisis dan Perancangan Antarmuka (*User Interface*)

Perancangan antarmuka merupakan suatu tahapan sistematis yang melibatkan proses perencanaan, pengembangan, dan pengaturan komponen visual serta fungsional dalam sebuah sistem, dengan tujuan untuk memastikan keterpaduan antara aspek estetika, fungsionalitas, dan kemudahan penggunaan bagi pengguna.

1. *Landing page*

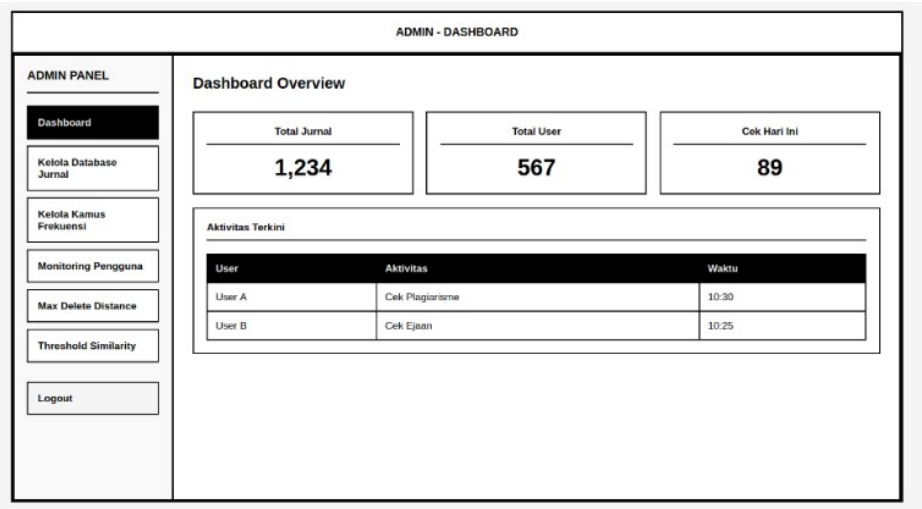
Pada halaman ini ditampilkan rancangan antarmuka *Landing Page* yang berfungsi sebagai gerbang awal interaksi pengguna dengan sistem. Halaman ini dirancang secara informatif, terstruktur, dan berorientasi pada kemudahan penggunaan (*usability*) agar pengguna dapat dengan cepat memahami tujuan, ruang lingkup, serta kapabilitas sistem yang disediakan.



Gambar 3. 23 Antarmuka *landing page*

2. Halaman *Dashboard* Admin

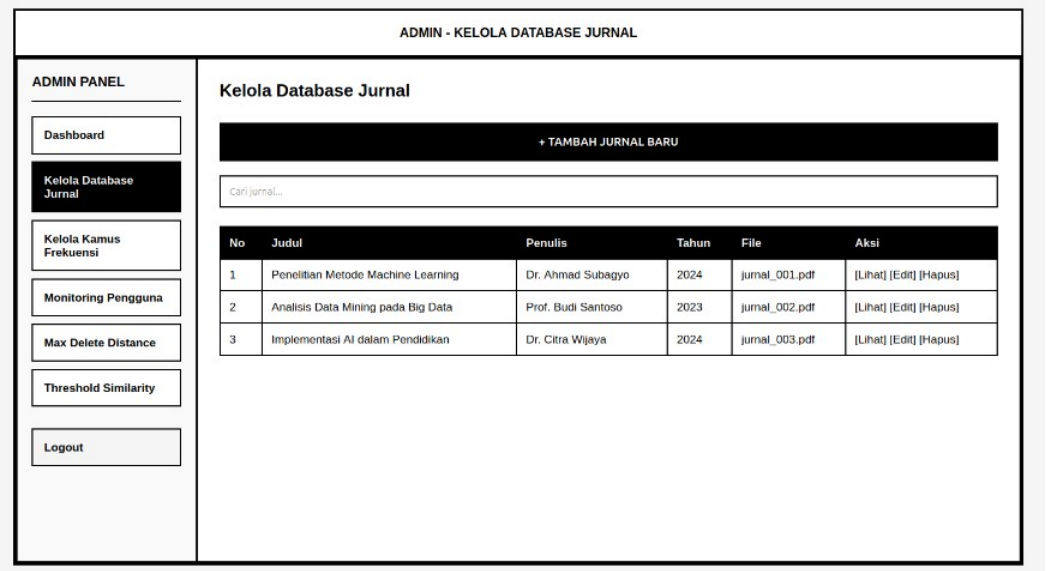
Halaman *dashboard* admin merupakan halaman utama yang berfungsi sebagai pusat kendali bagi administrator dalam mengelola dan memantau seluruh aktivitas sistem. Halaman ini dirancang untuk menyajikan ringkasan informasi penting secara cepat, terstruktur, dan mudah dipahami sehingga administrator dapat melakukan pengambilan keputusan secara efektif.



Gambar 3. 24 Antarmuka *dashboard* admin

3. Halaman Kelola *Database* Jurnal

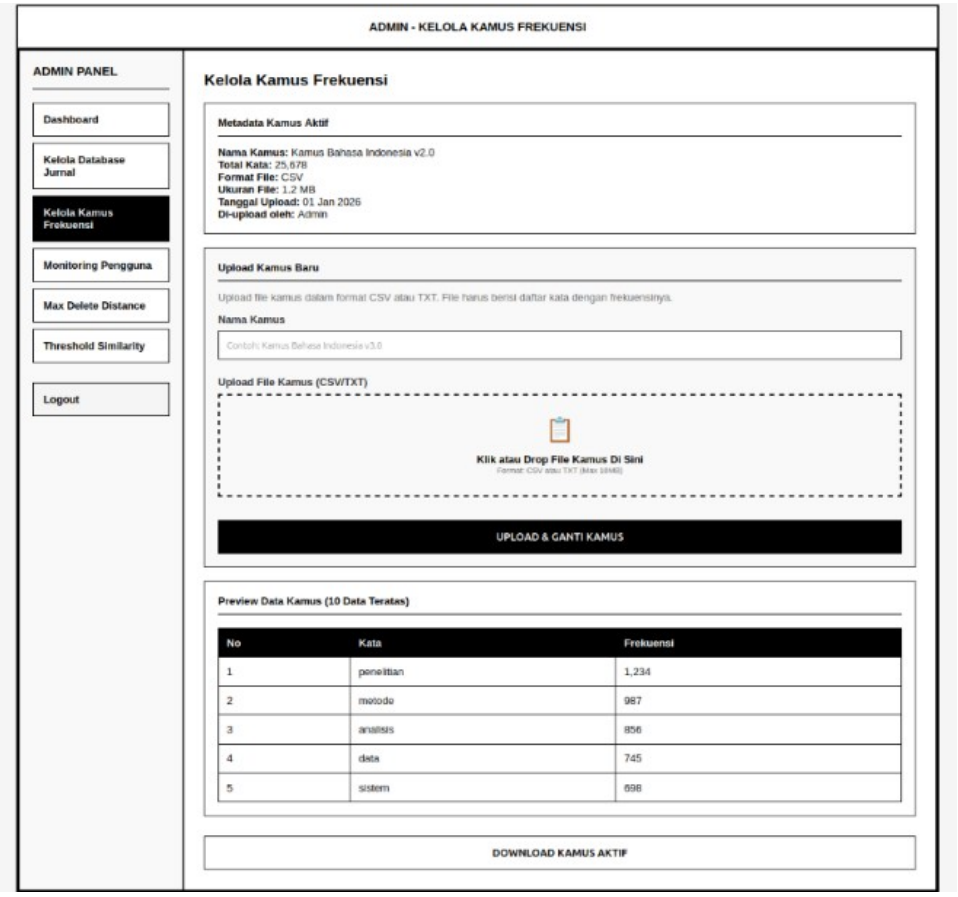
Halaman kelola *database* jurnal merupakan bagian dari modul administrator yang berfungsi untuk mengelola seluruh data jurnal yang digunakan dalam sistem. Halaman ini dirancang untuk mendukung pengelolaan data jurnal secara terstruktur, sistematis, dan efisien, sehingga administrator dapat menjamin ketersediaan, konsistensi, serta kualitas data jurnal yang menjadi dasar dalam proses analisis sistem.



Gambar 3. 25 Antarmuka kelola *database* jurnal

4. Halaman Kelola Kamu Frekuensi

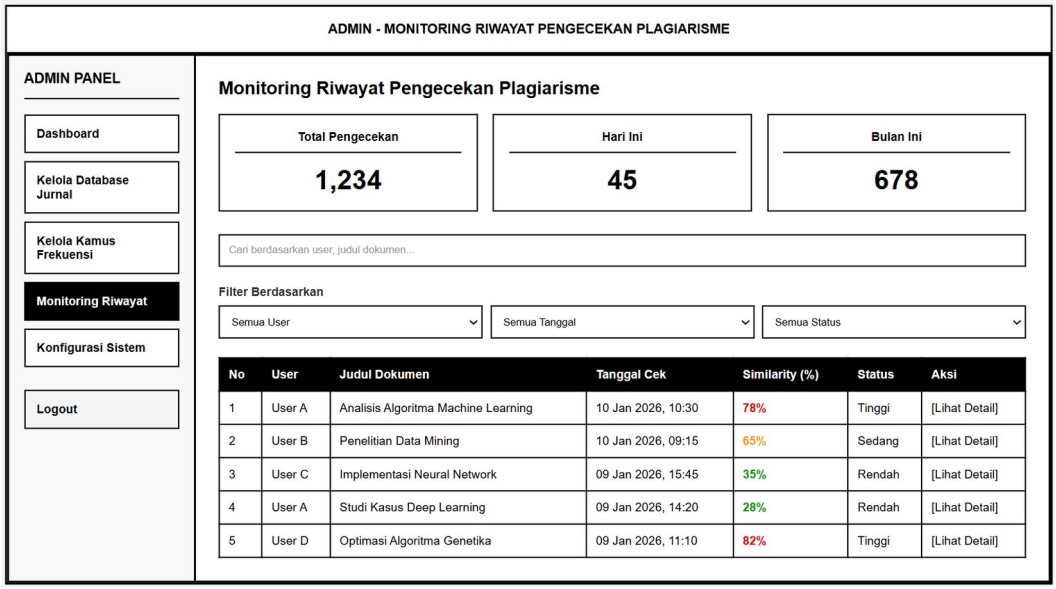
Halaman kelola kamus frekuensi merupakan bagian dari modul administrator yang berfungsi untuk mengelola kamus kata beserta nilai frekuensinya yang digunakan dalam proses analisis sistem. Halaman ini menyediakan informasi metadata kamus aktif, fasilitas unggah dan penggantian kamus dalam format CSV atau TXT, serta pratinjau data kata untuk verifikasi awal. Perancangan halaman ini bertujuan untuk memastikan keakuratan, konsistensi, dan keterbaruan kamus frekuensi sebagai komponen pendukung utama dalam proses pemrosesan teks dan analisis linguistik sistem.



Gambar 3. 26 Antarmuka kelola kamus frekuensi

5. Halaman Riwayat Pengecekan Plagiarisme Pengguna

Halaman monitoring riwayat pengecekan plagiarisme merupakan bagian dari modul administrator yang berfungsi untuk memantau dan mengevaluasi seluruh aktivitas pengecekan plagiarisme yang dilakukan dalam sistem. Halaman ini menyajikan ringkasan statistik jumlah pengecekan, baik secara keseluruhan maupun berdasarkan periode waktu tertentu, serta dilengkapi dengan fitur pencarian dan penyaringan data. Informasi yang ditampilkan mencakup identitas pengguna, judul dokumen, waktu pengecekan, persentase kemiripan, dan status hasil analisis. Perancangan halaman ini bertujuan untuk mendukung fungsi pengawasan, audit, dan pengambilan keputusan berbasis data terkait penggunaan dan kinerja sistem deteksi plagiarisme.

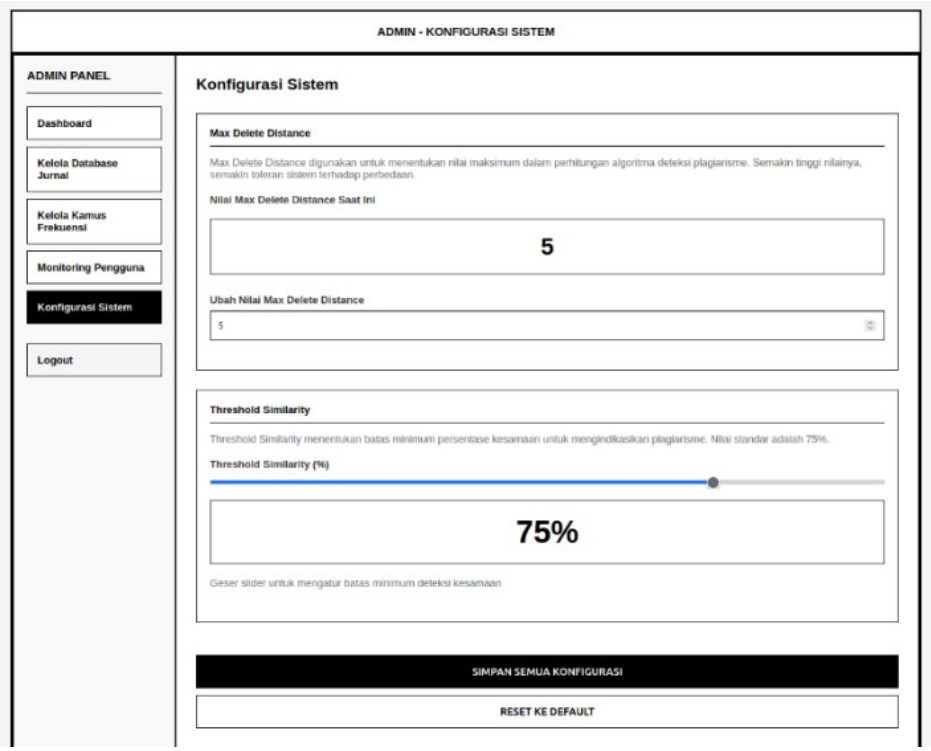


Gambar 3. 27 Antarmuka riwayat pengecekan plagiarism

6. Halaman Konfigurasi Sistem

Halaman konfigurasi sistem merupakan bagian dari modul administrator yang berfungsi untuk mengatur parameter utama dalam proses deteksi plagiarisme. Pada halaman ini, administrator dapat menentukan nilai *Max Delete Distance*

sebagai batas toleransi perbedaan karakter dalam algoritma SymSpell, serta mengatur *Threshold Similarity* yang merepresentasikan persentase minimum kesamaan untuk mengindikasikan plagiarisme. Pengaturan ini memungkinkan penyesuaian tingkat sensitivitas dan akurasi sistem sesuai kebutuhan operasional. Seluruh perubahan konfigurasi dapat disimpan atau dikembalikan ke nilai default guna menjaga konsistensi dan stabilitas kinerja sistem.

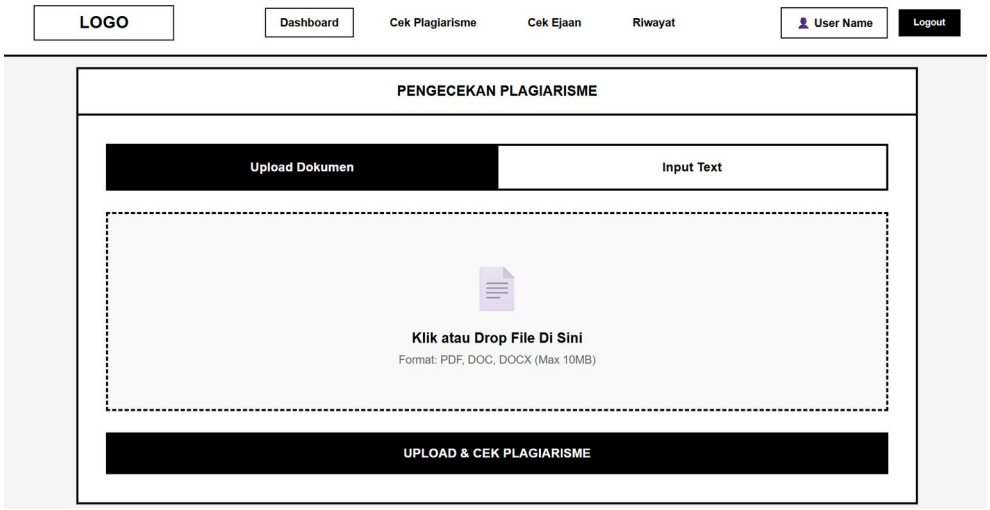


Gambar 3. 28 Antarmuka konfigurasi sistem

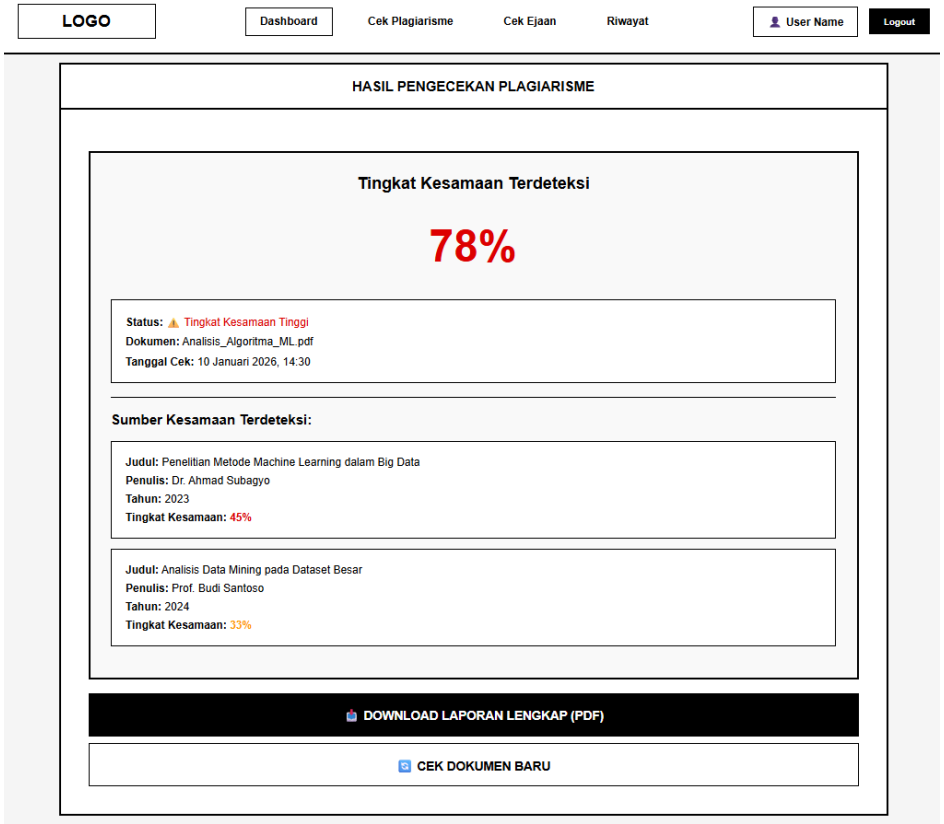
7. Halaman Pengecekan Plagiarisme

Halaman pengecekan plagiarisme pada sisi pengguna berfungsi sebagai antarmuka utama untuk melakukan analisis kesamaan dokumen. Pada halaman ini, pengguna dapat memasukkan data berupa unggahan dokumen atau teks secara langsung, yang selanjutnya diproses oleh sistem menggunakan algoritma pendeteksian plagiarisme. Hasil analisis disajikan

dalam bentuk persentase tingkat kemiripan, daftar sumber pembanding, serta penandaan bagian teks yang terindikasi memiliki kesamaan. Penyajian ini bertujuan untuk memberikan informasi yang objektif, terukur, dan mudah dipahami terkait tingkat orisinalitas dokumen yang diperiksa.



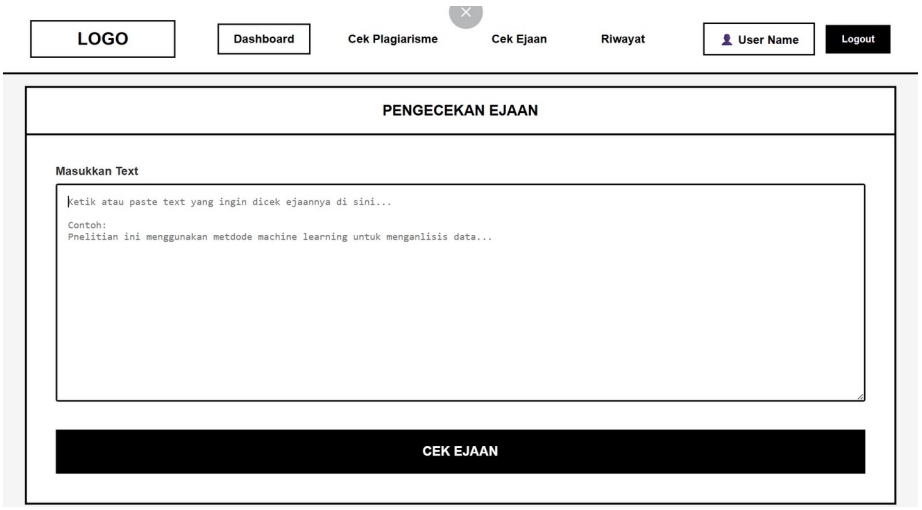
Gambar 3. 29 Antarmuka pengecekan plagiarisme



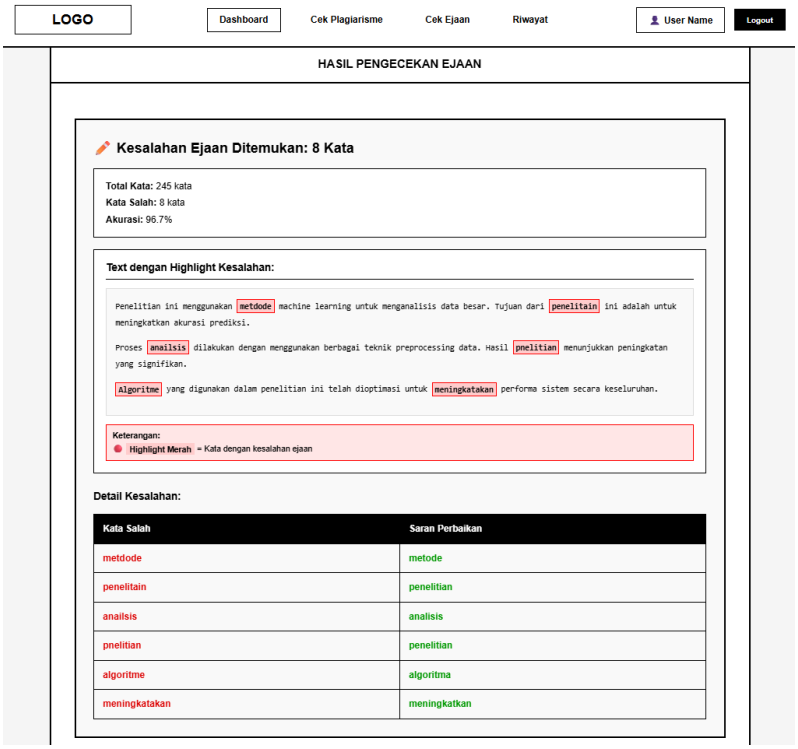
Gambar 3. 30 Antarmuka hasil pengecekan plagiarism

8. Halaman Pengecekan Ejaan

Halaman pengecekan ejaan pada sisi pengguna berfungsi sebagai sarana untuk mengevaluasi ketepatan penulisan teks secara otomatis. Pengguna dapat memasukkan teks atau dokumen yang akan diperiksa, kemudian sistem menggunakan algoritma SymSpell untuk melakukan analisis dengan memanfaatkan kamus frekuensi sebagai acuan validasi kata. Hasil pemeriksaan disajikan dalam bentuk penandaan kata yang teridentifikasi sebagai kesalahan ejaan beserta rekomendasi perbaikannya. Fitur ini dirancang untuk membantu pengguna meningkatkan kualitas kebahasaan dan ketepatan penulisan dokumen secara sistematis dan efisien.



Gambar 3. 31 Antarmuka pengecekan ejaan



Gambar 3. 32 Antarmuka hasil pengecekan ejaan

3.8. Studi Kasus

3.8.1. Studi Kasus Pengecekan Ejaan Menggunakan Algoritma Symspell

Sistem ini menerima input berupa teks yang dikirimkan oleh pengguna melalui fitur pengeceka ejaan. Pada skenario ini,

pengguna menginput teks "*Sistem ini dikembagkan untuk mendetksi plagirisme pada dokumen ilmiah.*". Algoritma Sysmspell akan mengoreksi kata-kata yang salah eja sehingga diperoleh teks yang sesuai dengan kaidah bahasa Indonesia. Proses koreksi melalui beberapa tahapan utama, mulai dai tokenisasi, pencocokan dengan kamus, proses *delete distance*, dan pencocokkan kata.

1. Tokenisasi

Dalam aplikasi yang dikembangkan, proses tokenisasi dilakukan melalui beberapa tahapan. Pertama, sistem menerima input teks dari pengguna dalam bentuk paragraf. Teks tersebut kemudian diproses melalui normalisasi awal, yaitu mengubah seluruh karakter menjadi huruf kecil (*lowercase*) serta menghilangkan tanda baca yang tidak diperlukan. Setelah itu, teks dipecah menjadi unit-unit linguistik yang lebih kecil, yaitu kata dan sub-kata.

Tabel 3. 5

Tokenisasi

No	Kata Asli
1	Sistem
2	ini
3	dikembagkan
4	untuk
5	mendetksi
6	plagirisme
7	pada
8	dokumen
9	ilmiah

2. Pencocokan dengan Kamus

Setelah proses tokenisasi, setiap kata hasil pemecahan teks dibandingkan dengan kamus frekuensi kata bahasa Indonesia yang digunakan sebagai acuan ejaan baku. Proses ini bertujuan untuk menentukan apakah suatu kata ditulis dengan benar atau mengandung kesalahan ejaan.

Setiap kata dibandingkan dengan kamus frekuensi kata bahasa Indonesia.

a. Jika kata ditemukan di kamus, maka kata dianggap benar
b. Jika kata tidak ditemukan, maka dilakukan proses koreksi menggunakan SymSpell

Berdasarkan hasil pencocokan dengan kamus, sistem mendeteksi beberapa kata yang tidak terdaftar dalam kamus frekuensi, yaitu:

Tabel 3. 6 Kata salah

No.	**Kata Salah**
1.	dikembagkan
2.	mendetksi
3.	plagirisme

Kata-kata tersebut kemudian ditandai sebagai kesalahan ejaan dan menjadi masukan pada tahap koreksi ejaan menggunakan algoritma SymSpell untuk menghasilkan bentuk kata yang sesuai dengan kaidah bahasa Indonesia.

3. Proses *Delete Distance* dan Pencocokan Kata.

Dalam melakukan perbaikan, SymSpell menggunakan kamus sebagai acuan kata yang dianggap benar. Untuk proses *delete distance* digunakan salah satu kata *typo* yaitu “mendetksi”. Langkah-langkahnya adalah sebagai berikut:

a. Menghapus satu huruf pada setiap kata yang terdapat dalam kamus. Hasil penghapusan pada tahap pertama kemudian digunakan kembali untuk dilakukan proses penghapusan huruf selanjutnya dengan cara yang sama. Proses ini dilakukan hingga dua kali penghapusan sesuai dengan nilai *delete distance* yang digunakan, yaitu 2. Seluruh bentuk kata, baik kata asli maupun kata hasil penghapusan huruf, disimpan oleh sistem dan digunakan sebagai referensi pada proses pencocokan kata.

Tabel 3. 7 Daftar kata hasil penghapusan kata pada kamus

Kata Asli	***Delete Distance* 1 (Hapus 1 Huruf)**	***Delete Distance* 2 (Hapus 2 Huruf)**
mendeteksi	endeteksi, mndeteksi, medeteksi, meneteksi, mendteksi, mendeteksi, mendeteksi, mendeteksi, mendeteksi, mendeteks	ndeteksi, edeteksi, eneteksi, endetksi, endeteksi, endeteks, mdeteksi, mneteksi, mndteksi, mndeteksi, mndeteks, medteksi, medeteksi, medeteks, menetksi, meneteksi, meneteks, mendeksi, mendteksi, mendetksi, mendetksi
pendeteksi	endeteksi, pndeteksi, pedeteksi, peneteksi, pendteksi, pendeteksi, pendeteksi, pendeteksi,	ndeteksi, edeteksi, eneteksi, endetksi, endeteksi, endeteks, pdeteksi, pneteksi, pndteksi, pndeteksi, pndeteks, pedteksi, pedeteksi, pedeteks, penetksi, peneteksi, peneteks, pendeksi, pendteksi, pendetksi, pendetks

Tabel 3. 7 Daftar kata hasil penghapusan kata pada kamus

Kata Asli	*Delete Distance* 1 (Hapus 1 Huruf)	*Delete Distance* 2 (Hapus 2 Huruf)
	pendeteks	
terdeteksi	erdeteksi, trdeteksi, tedeteksi, tereteksi, terdteksi, terdeteksi, terdeteksi, terdeteksi, terdeteks	rdeteksi, edeteksi, ereteksi, erdetksi, erdeteksi, erdeteks, tdeteksi, tretksi, trdetksi, trdeteksi, trdeteks, tedteksi, tedeteksi, tedeteks, terdetksi, terdetks, terdeksi, terdteksi

b. Melakukan pemeriksaan terhadap kata input dengan mencocokkannya ke dalam kamus. Sebagai ilustrasi, salah satu kata salah dari inputan pengguna yaitu “mendetksi” dilakukan proses penghapusan huruf terhadap kata tersebut.

Tabel 3. 8 Hasil *delete distance* kata yang salah

Kata Salah (*Typo*)	*Delete Distance* 1 (Hapus 1 Huruf)	*Delete Distance* 2 (Hapus 2 Huruf)
mendetksi	endetksi, mndetksi, medetksi, menetksi, mendtksi, mendeksi,	ndetksi, edetksi, enetksi, endetks, mdetksi, mnetksi, mndetks, medetks, menetks, mendksi, mendeki,

Tabel 3. 8 Hasil *delete distance* kata yang salah

Kata Salah (*Typo*)	***Delete Distance* 1 (Hapus 1 Huruf)**	***Delete Distance* 2 (Hapus 2 Huruf)**
	mendetki, mendetks	mendeks

c. Selanjutnya adalah mencari kata yang sama antara hasil penghapusan kata “mendetksi” dan hasil penghapusan dari kata-kata pada kamus yang dilakukan diproses pertama.

Tabel 3. 9 Hasil pencocokan kombinasi penghapusan karakter antara kata input dan kamus frekuensi

Jenis Kata	**Kata**	**Delete Distance 1 (Hapus 1 Huruf)**	**Delete Distance 2 (Hapus 2 Huruf)**
Kata Benar (Kamus)	**mendeteksi**	endeteksi, mndeteksi, medeteksi, meneteksi, mendteksi, mendetksi, mendeteks	ndeteksi, edeteksi, eneteksi, endetksi, endeteks, mdeteksi, mneteksi, mndteksi, medteksi, menetksi, mendeksi
Kata Benar (Kamus)	**pendeteksi**	endeteksi, pndeteksi, pedeteksi, peneteksi, pendteksi, pendeteksi, pendeteks	ndeteksi, edeteksi, eneteksi, endetksi, endeteks, pdeteksi, pneteksi, pndteksi, pedteksi, penetksi, pendeksi
Kata	**terdetek**	erdeteksi,	rdeteksi, edeteksi,

Tabel 3. 9 Hasil pencocokan kombinasi penghapusan karakter antara kata input dan kamus frekuensi

Jenis Kata	Kata	Delete Distance 1 (Hapus 1 Huruf)	Delete Distance 2 (Hapus 2 Huruf)
Benar (Kamus)	**si**	trdeteksi, tedeteksi, tereteksi, terdteksi, terdeteksi, terdeteks	ereteksi, erdetksi, erdeteks, tdeteksi, trdetksi, tedteksi, terdetksi, terdeksi
Kata Salah (Input User)	**mendetksi**	endetksi, mndetksi, medetksi, menetksi, mendtksi, mendeksi, mendetki, mendetks	ndetksi, edetksi, enetksi, endetks, mdetksi, mnetksi, mndetks, medetks, menetks, mendksi, mendeki, mendeks

Berdasarkan tabel gabungan yang disajikan, dapat diketahui bahwa kata salah **"mendetksi"** menghasilkan sejumlah *delete variant* yang memiliki kecocokan langsung dengan *delete variant* dari kata kamus **"mendeteksi"**, khususnya pada variasi seperti "*mendetksi*", "*mendeksi*", dan "*mendetks*" pada tahap penghapusan pertama. Kecocokan tersebut menjadi dasar bagi algoritma **SymSpell** dalam menentukan kandidat koreksi ejaan. Meskipun kata **"pendeteksi"** dan **"terdeteksi"** juga menunjukkan kedekatan secara struktural, kata **"mendeteksi"** dipilih sebagai hasil

koreksi utama karena memiliki jarak penghapusan yang paling kecil serta frekuensi kemunculan tertinggi dalam kamus. Proses yang sama diterapkan pada kata-kata salah lainnya dalam sistem.

3.8.2. Studi Kasus Pegecekan Plagiarisme Menggunakan Algoritma TF-IDF dan *Cosine Similarity*

Pada studi kasus ini, dilakukan pengujian tingkat kemiripan (*similarity*) antara satu dokumen uji dengan beberapa dokumen pembanding menggunakan metode TF-IDF (*Term Frequency–Inverse Document Frequency*) dan *Cosine Similarity*.

Pada skenario ini, pengguna memasukkan teks *"sistem menggunakan tf-idf untuk plagiarisme"* sebagai dokumen uji. Selanjutnya, algoritma TF-IDF dan *Cosine Similarity* digunakan untuk membandingkan teks tersebut dengan dokumen-dokumen referensi yang tersimpan dalam basis data sistem. Adapun kumpulan dokumen referensi yang digunakan dalam studi kasus ini disajikan pada Tabel 3.10.

Tabel 3. 10 *Database* referensi

No.	***Database* Referensi**
1.	sistem mendeteksi plagiarisme pada dokumen teks.
2.	algoritma tf-idf digunakan untuk mendeteksi plagiarisme.

Proses membandingkan dokumen melalui beberapa tahapan utama, mulai dai *preprocessing*, pembentukan *term frequency*, perhitungan *inverse document frequency*, representasi vektor, dan perhitungan kemiripan menggunakan *Cosine Similarity*.

1. *Pre-processing*

Tahap *pre-processing* merupakan tahapan awal dalam proses pengolahan data yang bertujuan untuk menghasilkan dataset yang bersih, terstruktur, dan siap digunakan pada proses analisis selanjutnya. Pada penelitian ini, tahapan *pre-processing* dilakukan terhadap teks input pengguna maupun dokumen referensi dalam basis data agar memiliki format yang seragam sebelum dilakukan perhitungan TF-IDF dan *Cosine Similarity*. Tahapan *pre-processing* yang dilakukan meliputi *cleaning*, *case folding*, *tokenizing*, *filtering* (*stopword removal*), dan *stemming*. Adapun hasil dari *pre-processing* dapat dilihat pada Tabel 3.11.

Tabel 3. 11 Hasil *pre-processing*

No	Sumber Dokumen	Token Hasil *Pre-processing*
1	Dokumen Referensi 1	sistem, deteksi, plagiarisme, dokumen, teks
2	Dokumen Referensi 2	algoritma, tfidf, guna, deteksi, plagiarisme
3	Input Pengguna	sistem, guna, tfidf, plagiarisme

2. Perhitungan *Term Frequency* dan *Document Frequency*

Berdasarkan hasil *pre-processing* terhadap ketiga dokumen, diperoleh nilai *term frequency* (TF) untuk setiap kata, sebagaimana disajikan pada Tabel 3.12. Perhitungan ini menggunakan Rumus 2.3.

Tabel 3. 12 Perhitungan term frequency

Term	**D1**	**D2**	**D3(Uji)**
sistem	0.2	0	0.25
deteksi	0.2	0.2	0
plagiarisme	0.2	0.2	0.25
dokumen	0.2	0	0
teks	0.2	0	0
algoritma	0	0.2	0
tfidf	0	0.2	0.25
guna	0	0.2	0.25

Document Frequency (DF) merupakan jumlah dokumen dalam kumpulan dokumen (*corpus*) yang mengandung suatu term tertentu. Nilai DF digunakan untuk mengetahui seberapa sering sebuah kata muncul pada seluruh dokumen referensi dan dokumen uji. Dalam metode TF-IDF, nilai DF berperan penting karena menjadi dasar dalam perhitungan *Inverse Document Frequency* (IDF), yang berfungsi untuk menurunkan bobot kata-kata umum dan meningkatkan bobot kata-kata yang bersifat lebih spesifik.

Tabel 3. 13 Perhitungan *document frequency*

Term	**DF**
sistem	2
deteksi	2

Tabel 3. 13 Perhitungan *document frequency*

Term	**DF**
plagiarisme	3
dokumen	1
teks	1
algoritma	1
tfidf	2
guna	2

3. Perhitungan *Inverse Document Frequency*

Perhitungan IDF dilakukan berdasarkan nilai *Document Frequency* (DF) yang telah diperoleh pada tahap sebelumnya. Semakin besar nilai DF suatu term, maka semakin kecil nilai IDF yang dihasilkan. Sebaliknya, term yang hanya muncul pada sedikit dokumen akan memiliki nilai IDF yang lebih tinggi. Pada penelitian ini, perhitungan IDF dilakukan terhadap seluruh term hasil *pre-processing* yang terdapat pada dokumen input pengguna maupun dokumen referensi. Perhitungan IDF menggunakan Rumus 2.4.

Tabel 3. 14 Perhitungan IDF

Term	**IDF**
sistem	0.176
deteksi	0.176
plagiarisme	0
dokumen	0.477
teks	0.477

Tabel 3. 14 Perhitungan IDF

Term	**IDF**
algoritma	0.477
tfidf	0.176
guna	0.176

4. Perhitungan TF-IDF

Perhitungan *Term Frequency–Inverse Document Frequency* (TF-IDF) dilakukan untuk memberikan bobot pada setiap *term* yang muncul dalam masing-masing dokumen. Pembobotan ini bertujuan untuk menunjukkan tingkat kepentingan suatu term dalam sebuah dokumen terhadap keseluruhan kumpulan dokumen (*corpus*). Nilai TF-IDF diperoleh dengan mengalikan nilai *Term Frequency* (TF) dan *Inverse Document Frequency* (IDF), sebagaimana ditunjukkan pada Rumus 2.2.

Berdasarkan hasil perhitungan menggunakan Rumus 2.2, diperoleh nilai TF-IDF untuk setiap term pada dokumen D1, D2, dan D3 yang disajikan pada Tabel 3.15.

Tabel 3. 15 Perhitungan TF-IDF

Term	**D1**	**D2**	**D3(Uji)**
sistem	0.035	0	0.044
deteksi	0.035	0.035	0
plagiarisme	0	0	0
dokumen	0.095	0	0
teks	0.095	0	0
algoritma	0	0.095	0
tfidf	0	0.035	0.044

guna	0	0.035	0.044

5. Perhitungan Kemiripan Menggunakan *Cosine Similarity*

Pada tahap ini dilakukan perhitungan tingkat kemiripan antara dokumen uji dan dokumen referensi menggunakan metode *Cosine Similarity*. Proses perhitungan kemiripan dilakukan menggunakan Rumus 2.1, di mana semakin besar nilai *Cosine Similarity* yang dihasilkan, maka semakin tinggi tingkat kemiripan antara kedua dokumen yang dibandingkan. Hasil perhitungan kemiripan antara dokumen uji (D3) dengan dokumen referensi (D1 dan D2) disajikan pada tabel berikut.

Tabel 3. 16 Perhitungan *Cosine Similarity*

Perbandingan	***Similarity***
D3,D1	0.141
D3,D2	0.359

Berdasarkan hasil perhitungan kemiripan menggunakan metode *Cosine Similarity*, diperoleh nilai kemiripan antara dokumen uji (D3) dengan dokumen referensi D1 sebesar 0,141 dan dengan dokumen referensi D2 sebesar 0,359. Nilai tersebut menunjukkan bahwa dokumen uji memiliki tingkat kemiripan yang lebih tinggi terhadap dokumen D2 dibandingkan dengan dokumen D1. Dengan demikian, dapat disimpulkan bahwa konten pada dokumen uji lebih mendekati dokumen referensi D2. Hasil ini membuktikan bahwa penerapan algoritma TF-IDF dan *Cosine Similarity* mampu mengidentifikasi tingkat kemiripan antar dokumen secara objektif dan efektif, sehingga dapat digunakan sebagai dasar dalam proses deteksi plagiarisme pada dokumen teks.

3.9. Pengujian Kinerja Algoritma

3.9.1. Skenario Pengujian Kinerja Algoritma SymSpell dalam Pengecekan Ejaan

Rencana pengujian kinerja algoritma SymSpell bertujuan untuk mengevaluasi kemampuan sistem dalam mendeteksi dan memperbaiki kesalahan ejaan kata berbahasa Indonesia secara akurat. Pengujian ini dirancang untuk mengetahui tingkat ketepatan algoritma SymSpell dalam menghasilkan rekomendasi kata yang sesuai dengan kata baku.

Pengujian kinerja algoritma SymSpell direncanakan dilakukan dengan memberikan sejumlah kata atau kalimat yang mengandung kesalahan ejaan ke dalam sistem. Sistem kemudian akan menghasilkan rekomendasi perbaikan ejaan berdasarkan kamus frekuensi bahasa Indonesia yang telah dibangun.

Dataset pengujian direncanakan berupa kumpulan kalimat berbahasa Indonesia yang mengandung kesalahan ejaan, di mana setiap kalimat disertai dengan versi kalimat yang telah diperbaiki sesuai kaidah bahasa Indonesia. Penyusunan dataset dilakukan berdasarkan Kamus Besar Bahasa Indonesia (KBBI) serta pola kesalahan ejaan yang umum ditemukan dalam teks pengguna.

Dataset tersebut direncanakan digunakan sebagai *ground truth* untuk membandingkan hasil koreksi ejaan yang dihasilkan oleh sistem terhadap kalimat referensi yang telah dinyatakan benar.

Evaluasi kinerja algoritma SymSpell direncanakan menggunakan pendekatan perbandingan langsung antara hasil koreksi sistem dan kata referensi (kata benar). Akurasi koreksi digunakan untuk mengukur tingkat keberhasilan algoritma dalam menghasilkan kata yang benar.

(3.1)

Output yang diharapkan dari rencana pengujian ini adalah diperolehnya nilai akurasi koreksi kata yang dapat menggambarkan kinerja algoritma SymSpell dalam mendeteksi dan memperbaiki kesalahan ejaan secara efektif.

Pada skenario pengujian ini, digunakan sebanyak 100 kata yang telah diidentifikasi mengandung kesalahan ejaan. Berdasarkan hasil pemrosesan menggunakan algoritma SymSpell, diperoleh 89 kata yang berhasil dikoreksi dengan tepat sesuai dengan ejaan yang seharusnya. Sementara itu, 11 kata lainnya tidak berhasil dikoreksi secara benar, baik karena tidak ditemukan dalam kamus referensi maupun karena sistem menghasilkan rekomendasi koreksi yang kurang tepat.

Dengan demikian, tingkat akurasi koreksi ejaan yang dihasilkan oleh algoritma SymSpell pada skenario pengujian ini adalah sebesar 89%, yang diperoleh dari perbandingan antara jumlah kata yang berhasil dikoreksi dengan benar terhadap total kata yang diuji.

3.9.2. Skenario Pengujian Kinerja Algoritam *Cosine Similarity* dan TF-IDF dalam Pengecekan Plagiarisme

Pengujian kinerja algoritma dilakukan dengan membandingkan hasil keluaran terhadap data pembanding (*ground truth*) yang ditentukan secara manual. Setiap pasangan dokumen akan diproses menggunakan algoritma TF-IDF dan *Cosine Similarity* untuk memperoleh nilai kemiripan. Data uji direncanakan terdiri atas beberapa kategori tingkat kemiripan, yaitu:

1. Teks identik atau salinan langsung
2. Teks dengan parafrase ringan
3. Teks dengan parafrase berat
4. Teks dengan topik yang berbeda

Pada tahap ini, pengujian tidak dilakukan setelah sistem web selesai dikembangkan, tetapi secara terpisah untuk mengevaluasi performa algoritma secara mandiri. Pengujian menggunakan data *dummy* yang dijalankan melalui *platform* Google Colab. Hasil perhitungan algoritma kemudian dibandingkan dengan perhitungan manual yang dilakukan menggunakan Microsoft Excel sebagai data pembanding (*ground truth*).

Secara metodologis, tahap ini termasuk *algorithm validation* atau *unit testing*, yaitu pengujian komponen inti sistem secara independen agar hasil evaluasi benar-benar mencerminkan kinerja algoritma tanpa dipengaruhi proses integrasi atau antarmuka. Setelah dinyatakan akurat, barulah dilakukan pengujian sistem secara keseluruhan (*system testing*).

Pengujian ini dirancang untuk mengetahui sejauh mana algoritma mampu mengklasifikasikan dokumen ke dalam kategori plagiarisme dan tidak plagiarisme berdasarkan nilai kemiripan yang dihasilkan. Pada skenario ini digunakan contoh data yang ditunjukkan pada Tabel 3.17.

Tabel 3. 17 Contoh data dokumen plagiarisme

Dokumen	**Manual**	**Similarity**	**Prediksi Algoritma**
D1	Plagiarisme	0.91	Plagiarisme

Tabel 3. 17 Contoh data dokumen plagiarisme

Dokumen	Manual	Similarity	Prediksi Algoritma
D2	Plagiarisme	0.85	Plagiarisme
D3	Plagiarisme	0.79	Plagiarisme
D4	Plagiarisme	0.68	Plagiarisme
D5	Plagiarisme	0.63	Tidak Plagiarisme
D6	Tidak Plagiarisme	0.55	Plagiarisme
D7	Tidak Plagiarisme	0.47	Tidak Plagiarisme
D8	Tidak Plagiarisme	0.39	Tidak Plagiarisme
D9	Tidak Plagiarisme	0.33	Tidak Plagiarisme
D10	Tidak Plagiarisme	0.21	Tidak Plagiarisme

Pada pengujian ini digunakan batas ambang (*threshold*) sebesar 0,6 sebagai dasar untuk mengklasifikasikan dokumen ke dalam kategori plagiarisme atau tidak. Berdasarkan hasil perbandingan antara perhitungan algoritma dan perhitungan manual (*ground truth*), kemudian disusun *confusion matrix* yang ditunjukkan pada Tabel 3.18.

Tabel 3. 18 Skenario *confusion matrix*

Aktual/Prediksi	Plagiarisme	Tidak Plagiarisme	Total Aktual
Sistem Plagiarisme	4 (TP)	1 (FP)	5
Sistem Tidak	1 (FN)	4 (TN)	5

Plagiarisme			
Total Prediksi	5	5	10

Berdasarkan *confusion matrix* yang telah diperoleh, selanjutnya dilakukan perhitungan metrik evaluasi untuk mengukur kinerja sistem dalam mendeteksi plagiarisme.

1. Akurasi mengukur seberapa akurat algoritma yang digunakan dapat mendeteksi plagiarism. Akurasi dihitung menggunakan persamaan 2.5.

2. *Precision* mengukur ketepatan sistem dalam mengidentifikasi dokumen yang dikategorikan sebagai plagiarisme. *Precision* dihitung menggunakan persamaan 2.6.

3. *Recall* mengukur kemampuan sistem dalam mendeteksi seluruh dokumen yang benar-benar termasuk kategori plagiarisme. *Recall* dihitung menggunakan persamaan 2.7.

4. *F1-Score* digunakan untuk memperoleh keseimbangan antara nilai *precision* dan *recall. F1-Score* dihitung menggunakan persamaan 2.8.

3.10. Pengujian *User Acceptance Testing*

User Acceptance Testing digunakan untuk mengevaluasi sistem analisis teks terpadu berbasis web yang dikembangkan untuk pengecekan ejaan menggunakan algoritma SymSpell

serta deteksi plagiarisme menggunakan metode TF-IDF dan *Cosine Similarity*.

Tabel 3. 19 Pertanyaan *user acceptance testing*

No	Pernyataan	SS	S	N	TS	STS
1	Apakah sistem analisis teks mudah digunakan oleh pengguna?					
2	Apakah antarmuka sistem jelas dan mudah dipahami?					
3	Apakah fitur pengecekan ejaan membantu dalam menemukan kesalahan penulisan kata?					
4	Apakah saran perbaikan ejaan yang diberikan sistem sesuai dengan konteks penulisan?					
5	Apakah proses pengecekan ejaan berjalan dengan cepat dan lancar?					
6	Apakah hasil deteksi plagiarisme mudah dipahami oleh pengguna?					
7	Apakah persentase tingkat plagiarisme yang ditampilkan sistem jelas dan informatif?					
8	Apakah integrasi antara fitur pengecekan ejaan dan deteksi plagiarisme berjalan dengan baik?					
9	Apakah sistem membantu meningkatkan kualitas penulisan karya ilmiah?					
10	Apakah sistem layak digunakan sebagai alat bantu analisis teks ilmiah berbasis web?					

Pengujian *User Acceptance Testing* (UAT) pada penelitian ini menggunakan skala Likert lima tingkat sebagai alat ukur untuk mengetahui tingkat penerimaan pengguna terhadap sistem yang dikembangkan. Skala Likert digunakan karena mampu menggambarkan persepsi dan penilaian responden secara kuantitatif terhadap pernyataan yang diajukan. Setiap kategori jawaban memiliki bobot nilai tertentu yang digunakan dalam proses perhitungan dan analisis hasil pengujian.

Tabel 3. 20 Kriteria jawaban

Kategori	**Keterangan**	**Bobot**
SS	Sangat Setuju	5
S	Setuju	4
N	Netral	3
TS	Tidak Setuju	2
STS	Sangat Tidak Setuju	1

Hasil pengujian *User Acceptance Testing* (UAT) dihitung menggunakan Rumus 2.5. Selanjutnya, persentase hasil pengujian tersebut diinterpretasikan berdasarkan kriteria penilaian skala Likert sebagaimana disajikan pada Tabel 3.8.

Tabel 3. 21 Kriteria penilaian skala likert

Persentase Jumlah Skor	**Kriteria**
20,00% – 36,00%	Tidak Baik
36,01% – 52,00%	Kurang Baik
52,01% – 68,00%	Cukup

Tabel 3. 21 Kriteria penilaian skala likert

Persentase Jumlah Skor	**Kriteria**
68,01% – 84,00%	Baik
84,01% – 100%	Sangat Baik

Pengujian *User Acceptance Testing* (UAT) dengan menggunakan skala Likert lima tingkat memberikan dasar evaluasi yang sistematis dan terukur dalam menilai tingkat penerimaan pengguna terhadap sistem yang dikembangkan. Melalui pembobotan pada setiap kategori jawaban dan perhitungan persentase skor, hasil pengujian dapat diinterpretasikan secara objektif berdasarkan kriteria penilaian skala Likert yang telah ditetapkan. Interpretasi tersebut digunakan untuk menentukan kualitas dan kelayakan sistem, sehingga dapat disimpulkan apakah sistem telah memenuhi kebutuhan dan harapan pengguna.

DAFTAR PUSTAKA

Arianti, T., Fa'izi, A., Adam, S., dan Wulandari, M., 2022, Perancangan Sistem Informasi Perpustakaan Menggunakan Diagram UML (Unified Modelling Language), *Jurnal Ilmiah Komputer Terapan dan Informasi*, 1, 1, 19–25.

Arief, S.F., dan Sugiarti, Y., 2022, Literature Review: Analisis Metode Perancangan Sistem Informasi Akademik Berbasis Web, *Jurnal Ilmiah Ilmu Komputer Fakultas Ilmu Komputer Universitas Al Asyariah Mandar*, 8, 2, 87–93.

Audah, H.A., Yuliawati, A., dan Alfina, I., 2023, A Comparison Between Symspell and a Combination of Damerau-Levenshtein Distance with the Trie Data Structure, *10th International Conference on Advanced Informatics: Concept, Theory and Application (ICAICTA)*, Lombok.

Azhari, R.Y., 2022, Web Service Framework: Flask dan FastAPI, *Technology and Informatics Insight Journal*, 1, 1, 2830–3210.

Darmanto, D., Pradasari, N.I., dan Wahyudi, E., 2024, Sistem Deteksi Plagiarisme Tugas Akhir Mahasiswa Berbasis Natural Language Processing Menggunakan Algoritma Jaro-Winkler dan TF-IDF, *Smart Comp: Jurnalnya Orang Pintar Komputer*, 13, 1, 202–211.

Fauzi, R., Iqbal, M., dan Haryanti, T., 2022, Design and Implementation of a Final Project Plagiarism Detection System Using Cosine Similarity Method, *IJAIT (International Journal of Applied Information Technology)*, 5, 2, 1–16.

Ferdiansyah, M.H., dan Nuryana, I.K.D., 2023, Analisis Perbandingan Metode Burkhard Keller Tree dan Symspell

dalam Spell Correction Bahasa Indonesia, *Journal of Informatics and Computer Science (JINACS)*, 4, 3, 305–313.

Gasim, I., 2025, Evaluating Hunspell, Symspell, Norvig, and N-Gram Spellcheckers for Azerbaijani Text, *International Journal of Innovative Science and Research Technology*, 10, 5, 2574–2577.

Hagos, D.H., Battle, R., dan Rawat, D.B., 2024**,** Recent Advances in Generative AI and Large Language Models: Current Status, Challenges, and Perspectives, *IEEE Transactions on Artificial Intelligence*, 5, 12, 1–21.

Halim, J., dan Lasut, D., 2024, Document Plagiarism Detection Application Using Web-Based TF-IDF and Cosine Similarity Methods, *Bit-Tech (Binary Digital – Technology)*, 7, 2, 203–213.

Khandelwal, U., 2024, FastAPI vs. the Competition: A Security Feature Showdown with a Proposed Model for Enhanced Protection, *International Journal of Scientific Research in Engineering and Management*, 8, 6, 1–4.

Muis, A., dan Muhammad, F., 2023, Pelatihan Text Mining Menggunakan Bahasa Pemrograman Python, *Relawan Jurnal Indonesia*, 3, 1, 36–46.

Nasution, dan Iswari, L., 2021, Penerapan React JS pada Pengembangan Frontend Aplikasi Startup Ubaform, *Jurnal Automata*, 2, 2, 1–8.

Ong, R., 2024, What Is Faiss (Facebook AI Similarity Search), *datacamp.com*, diakses tanggal 14 Januari 2026.

Pawestri, S., dan Suyanto, Y., 2024, Analisis Perbandingan Metode Similarity untuk Kemiripan Dokumen Bahasa

Indonesia pada Deteksi Kemiripan Teks Bahasa Indonesia, *Jurnal Media Informatika Budidarma*, 8, 3, 1440–1450.

Pratiwi, M.A., dan Aisya, N., 2021, Fenomena Plagiarisme Akademik di Era Digital, *Publishing Letters*, 1, 2, 16–33.

Prianto, C., Markuci, D., dan Pane, S.F., 2023, Implementasi Spelling Corrector untuk Mengatasi Typographical Error pada Fitur Pencarian Aplikasi Kamus Istilah Informatika, *Jurnal Teknologi Informasi: Jurnal Keilmuan dan Aplikasi Bidang Teknik Informatika*, 17, 1, 20–30.

Qi, P., Zhang, Y., Zhang, Y., Bolton, J., dan Manning, C.D., 2020, Stanza: A Python Natural Language Processing Toolkit for Many Human Languages, *arXiv*, Abs/2003.0, 101–108.

Ramdany, S.W., Kaidar, S.A., Aguchino, B., Amelia, C., dan Putri, A., 2024, Penerapan UML Class Diagram dalam Perancangan Sistem Informasi Perpustakaan Berbasis Web, *Journal of Industrial and Engineering System (JIES)*, 5, 1, 30–41.

Risparyanto, A., 2020, Turnitin sebagai Alat Deteksi Plagiarisme, *Jurnal Perpustakaan*, 11, 2, 126–135.

Rosdiana, L.A., 2020, Kesalahan Penggunaan Ejaan Bahasa Indonesia (EBI) pada Karya Ilmiah Mahasiswa, *Bahtera Indonesia: Jurnal Penelitian Pendidikan Bahasa dan Sastra Indonesia*, 5, 1, 1–11.

Saeed, A.A.M., dan Taqa, A.Y., 2022, A Proposed Approach for Plagiarism Detection in Article Documents, *Journal*, 6, 2, 568–578.

Sari, L.P., 2023, Cosine Similarity-Based Plagiarism Detection on Electronic Documents, *Journal of Computer Science Application and Engineering (JOSAPEN)*, 1, 2, 44–48.

Shafiera, E., 2022, Pengaruh Penerapan Spelling Correction Menggunakan Metode Symspell pada Incident Categorization, *Skripsi*, Teknik Informatika, Universitas Islam Negeri Syarif Hidayatullah, Jakarta.

Silalahi, E., Silalahi, D., Tarigan, M.I., dan Sinaga, R.V., 2024, Deteksi Plagiarisme sebagai Peningkatan Integritas Akademik, *Kaizen: Jurnal Pengabdian pada Masyarakat*, 3, 1, 27–33.

Sukowati, I., dan Suciptaningsih, O.A., 2024, Literatur Review: Plagiarisme dalam Penulisan Karya Ilmiah: Memahami, Mencegah, dan Menangani, *JIIP – Jurnal Ilmiah Ilmu Pendidikan*, 7, 2, 1473–1477.

Syam, A.A., M, G.H., Salim, A., Surianto, D.F., dan B, M.F., 2024, Analisis Teknik Preprocessing pada Sentimen Masyarakat Terkait Konflik Israel–Palestina Menggunakan Support Vector Machine, *Jurnal Ilmiah Penelitian dan Pembelajaran Informatika*, 9, 3, 1464–1472.

Tanuwijaya, W., Setiawan, C.E., Irsyad, H., dan Rahman, A., 2025, Implementasi TF-IDF dan Cosine Similarity untuk Penyaringan Dokumen Berita Program Makan Siang Gratis Pemerintah Indonesia, *Journal of Information System, Computer Science, and Information Technology*, 6, 2, 322–334.

Tolegenova, A., 2024, Automatic Error Correction: Evaluating Performance of Spell Checker Tools, *Suleyman Demirel University Bulletin Natural and Technical Sciences*, 58, 1, 15–21.

Widianto, A., Pebriyanto, E., Fitriyanti, F., dan Marna, M., 2024, Document Similarity Using Term Frequency–Inverse Document Frequency Representation and Cosine

Similarity, *Journal of Dinda: Data Science, Information Technology, and Data Analytics*, 4, 2, 149–153.

Wulandari, Nofiyani, dan Hasugian, H., 2023, User Acceptance Testing (UAT) pada Electronic Data Preprocessing guna Mengetahui Kualitas Sistem, *JMIK (Jurnal Mahasiswa Ilmu Komputer)*, 4, 1, 20–27.

Zaky Ramadhan, Muhammad, dan Lhaksmana, K.M., 2020, Improving Document Retrieval with Spelling Correction for Weak and Fabricated Indonesian-Translated Hadith, *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, 4, 3, 551–557.

Zhu, Y., Yuan, H., Wang, S., Liu, J., Liu, W., Deng, C., Chen, H., Liu, Z., Dou, Z., dan Wen, J.-R., 2025, Large Language Models for Information Retrieval: A Survey, *ACM Transactions on Information Systems*, 44, 1, 1–54.

LAMPIRAN

Lampiran 1 Bimbingan dosen pembiming 1

Lampiran 2 Bimbingan dosen pembimbing 2